

Computergraphik I

Clipping | Rasterisierung | Anti-Aliasing



Prof. Dr. Michael Guthe

Sommersemester 2026

AI 5: Graphische Datenverarbeitung
Universität Bayreuth

Clipping (Zuschneiden)

- Abschneiden von Zeichnungsteilen, die außerhalb eines Bereiches liegen
- Beispiel:
 - Gegeben: Linie von Punkt P_1 nach P_2
 - Gesucht: Linienteil, der in der Zeichenfläche liegt
 - Wird durch P_1' und P_2' definiert



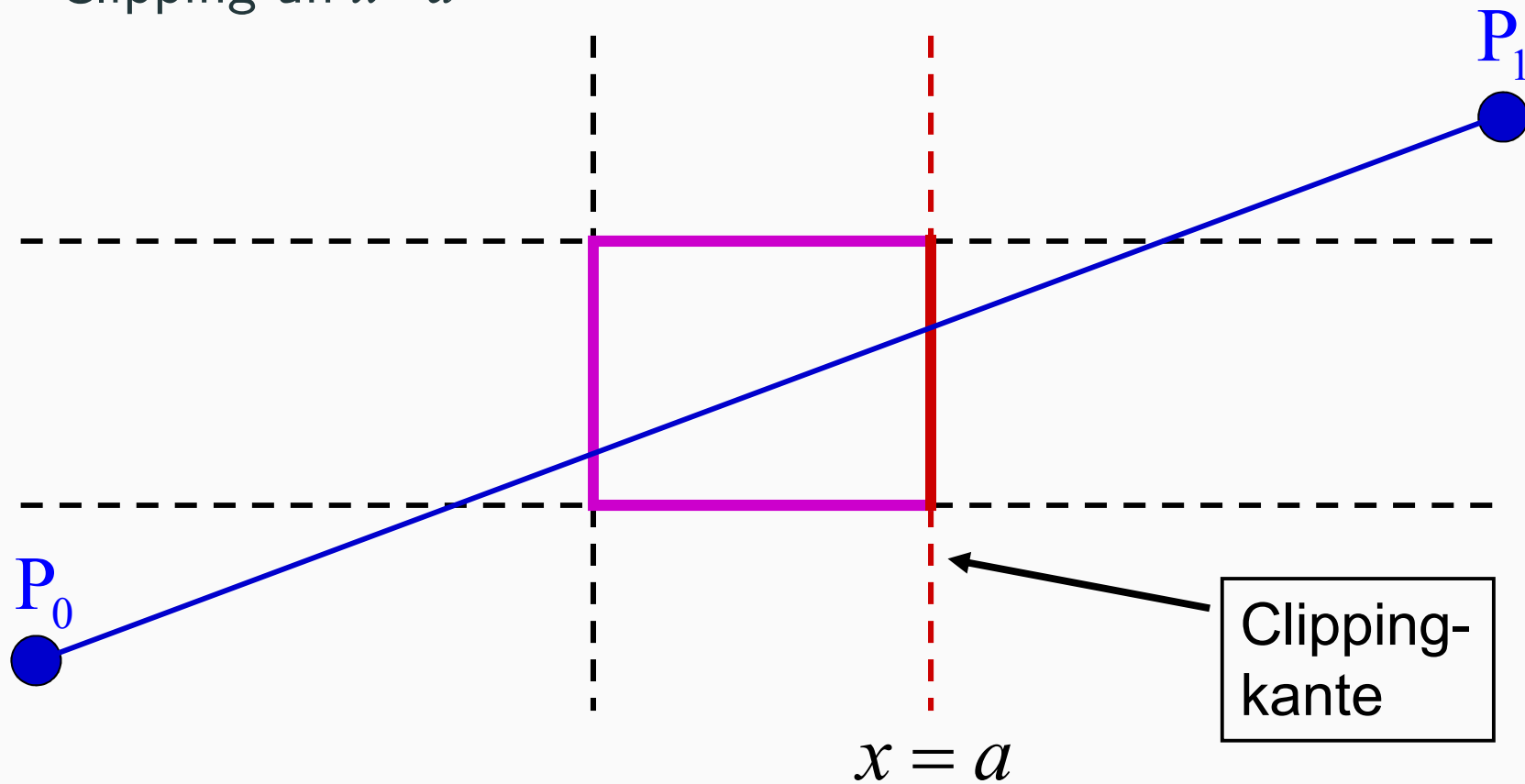
Implementierung von Clipping

- Clipping muss automatisch erfolgen
 - Kein Absturz, wenn P_i außerhalb des Bildschirms liegt
- Mögliche Implementierungen
 - Berechne P_1' , P_2' und ersetze Linie(P_1 , P_2) durch Linie(P_1' , P_2')
 - Zeichne Linie in (größere) temporäre Zeichenfläche und kopiere dann mit BitBlock Transfer
 - Abfrage bei setPixel

Clipping von Linien (1)

Beispiel:

- Clipping an $x=a$



Clipping von Linien (2)

Parametrische Liniengleichung:

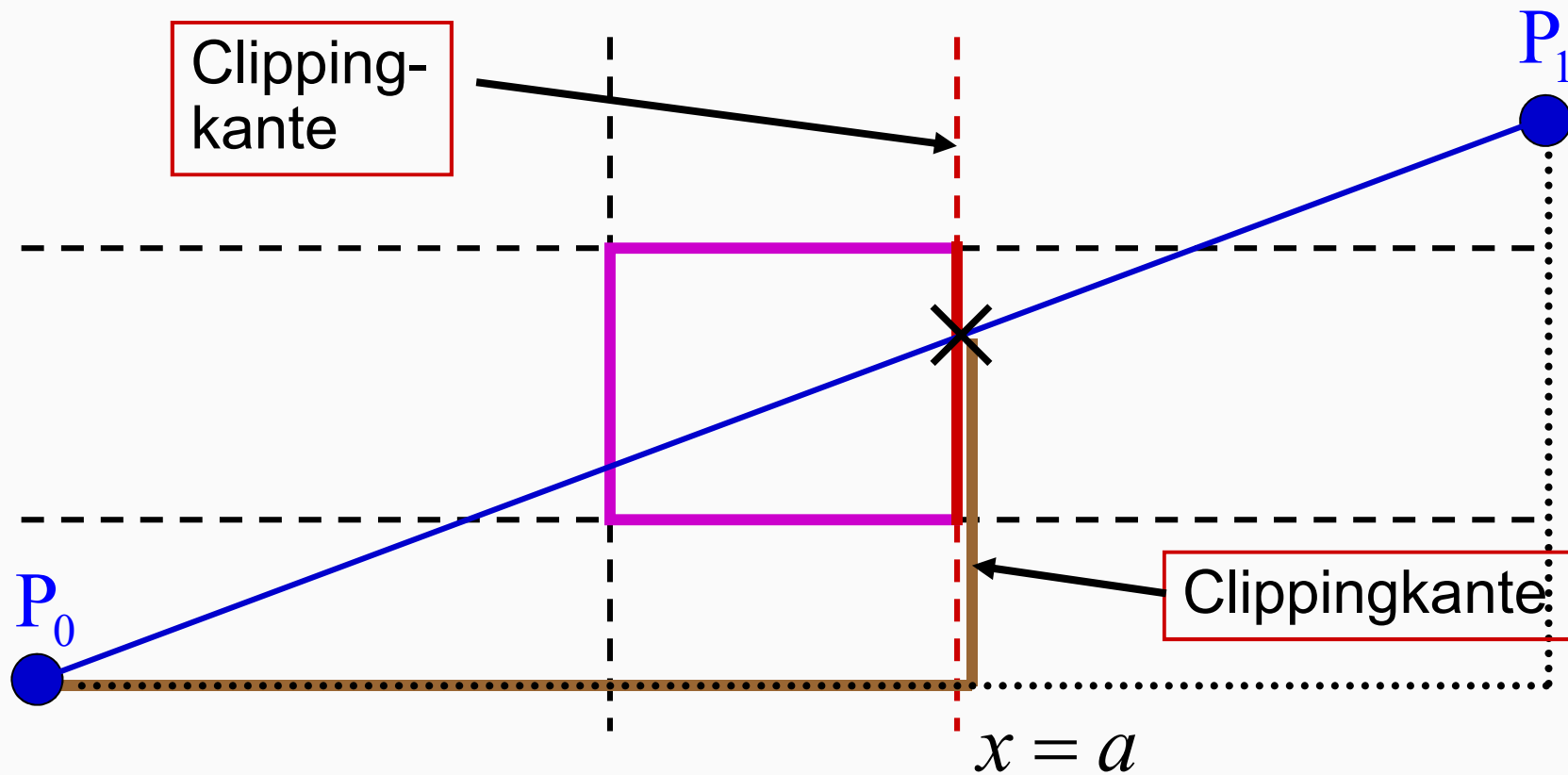
$$P(t) = P_0 + t(P_1 - P_0) \quad 0 \leq t \leq 1,$$

wobei

$$P(0) = P_0 \quad P(1) = P_1$$

Clipping von Linien (3)

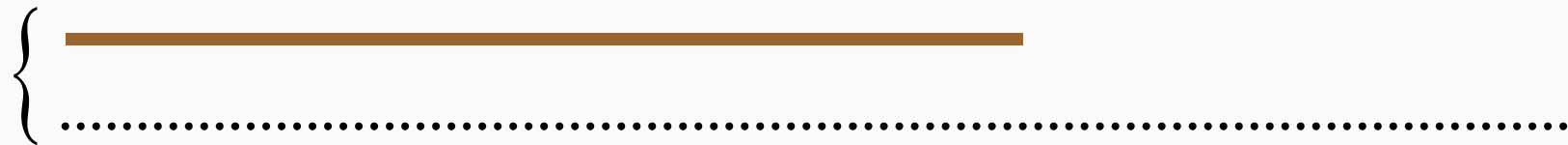
Benutze Strahlensatz:



Clipping von Linien (4)

Benutze Strahlensatz:

- Verwende Verhältnis dieser beiden Linien:



- d.h.: $t' = \frac{a - x_0}{x_1 - x_0}$
- Sonderfälle:
 - kein Schnittpunkt, wenn $t' \leq 0$, bzw. $t' \geq 1$
 - vertikale/horizontale Linie hat keinen Schnittpunkt

Liang-Barsky-Algorithmus (1)

Verallgemeinerung:

- Jede Clippingkante wird als implizite Gerade dargestellt:

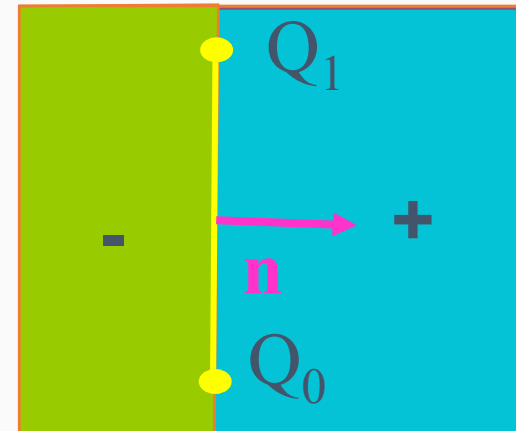
$$Q_0 = (x_0, y_0) \quad Q_1 = (x_1, y_1)$$

$$\mathbf{n} = (\Delta y, -\Delta x) = (y_1 - y_0, x_0 - x_1)$$

$$P = (x, y)$$

$$E(P) = \mathbf{n} \cdot P - \mathbf{n} \cdot Q_0$$

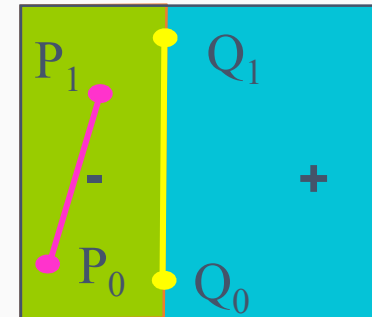
- Konvention:
 - Normale zeigt ins Innere!



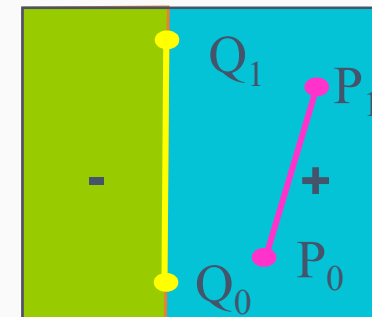
Liang-Barsky-Algorithmus (2)

- Nun gibt es 3 Fälle:

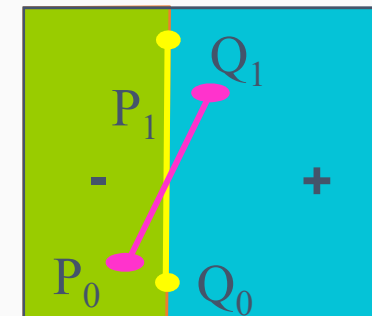
1.) P_0 und P_1 liegen außen :
 $E(P_0) \leq 0 \quad E(P_1) \leq 0$



2.) P_0 und P_1 liegen innen :
 $E(P_0) \geq 0 \quad E(P_1) \geq 0$



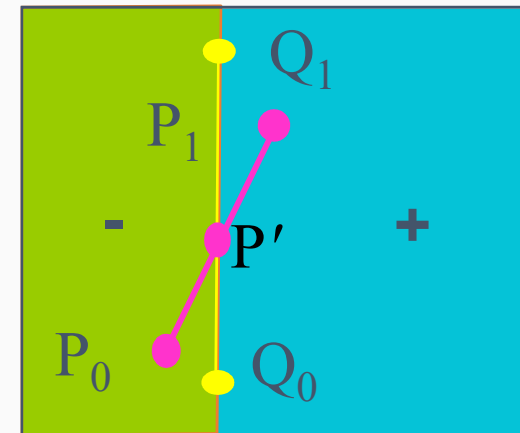
3.) P_0 und P_1 auf versch. Seiten :
 $E(P_0) < 0 \quad E(P_1) > 0$, bzw.
 $E(P_0) > 0 \quad E(P_1) < 0$



Liang-Barsky-Algorithmus (3)

- Bei 3) müssen wir Schnittpunkt P' berechnen:
- Einsetzen der parametrischen in die implizite Gleichung liefert:

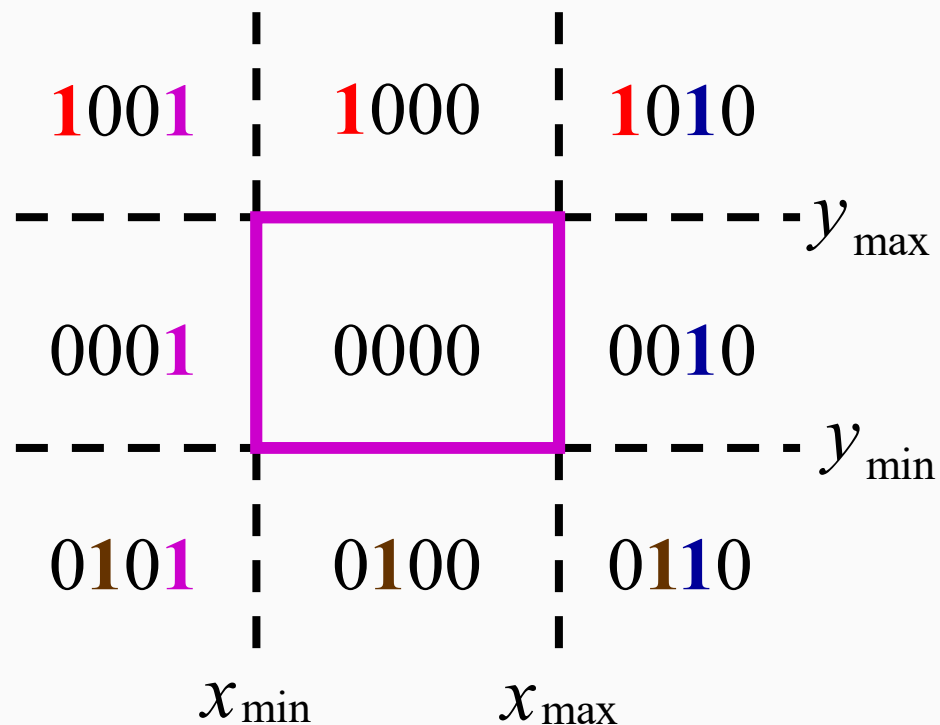
$$\begin{aligned} E(P_0 + t'(P_1 - P_0)) &= 0 \\ \Leftrightarrow \mathbf{n} \cdot (P_0 + t'(P_1 - P_0)) - \mathbf{n} \cdot Q_0 &= 0 \\ \Leftrightarrow \mathbf{n} \cdot P_0 + t' \mathbf{n} \cdot (P_1 - P_0) - \mathbf{n} \cdot Q_0 &= 0 \\ \Leftrightarrow t' = \frac{\mathbf{n} \cdot Q_0 - \mathbf{n} \cdot P_0}{\mathbf{n} \cdot (P_1 - P_0)} = \frac{\mathbf{n} \cdot (Q_0 - P_0)}{\mathbf{n} \cdot (P_1 - P_0)} \\ \Rightarrow P' = P_0 + \frac{\mathbf{n} \cdot (Q_0 - P_0)}{\mathbf{n} \cdot (P_1 - P_0)} (P_1 - P_0) \end{aligned}$$



Cohen-Sutherland-Algorithmus (1)

Schnelles identifizieren von Trivialfällen:

1. Definiere „Region Outcodes“



Cohen-Sutherland-Algorithmus (2)

2. Prüfe Vorzeichen:

- Bit 1 $\leftarrow \text{sign}(y_{\max} - y)$
- Bit 2 $\leftarrow \text{sign}(y - y_{\min})$
- Bit 3 $\leftarrow \text{sign}(x_{\max} - x)$
- Bit 4 $\leftarrow \text{sign}(x - x_{\min})$

Cohen-Sutherland-Algorithmus (3)

3. Klassifikation der Endpunkte:

- Betrachte $C_0 \wedge C_1$:
 - Was sagt uns dies?
 - $C_0 \wedge C_1 \neq 0 \Rightarrow$ komplett außerhalb
Beide Endpunkte horizontal oder vertikal außerhalb
- Betrachte $C_0 \vee C_1$:
 - $C_0 \vee C_1 = 0 \Rightarrow$ komplett innerhalb

Cohen-Sutherland-Algorithmus (4)

Bedeutung der Outcodes:

Bit_1	Bit_2	Bit_3	Bit_4
---------	---------	---------	---------

- Bit 1 = $t \Rightarrow$ über dem Fenster
- Bit 2 = $t \Rightarrow$ unter dem Fenster
- Bit 3 = $t \Rightarrow$ rechts vom Fenster
- Bit 4 = $t \Rightarrow$ links vom Fenster

Cohen-Sutherland-Algorithmus (5)

Clipping, falls nicht Trivialfall:

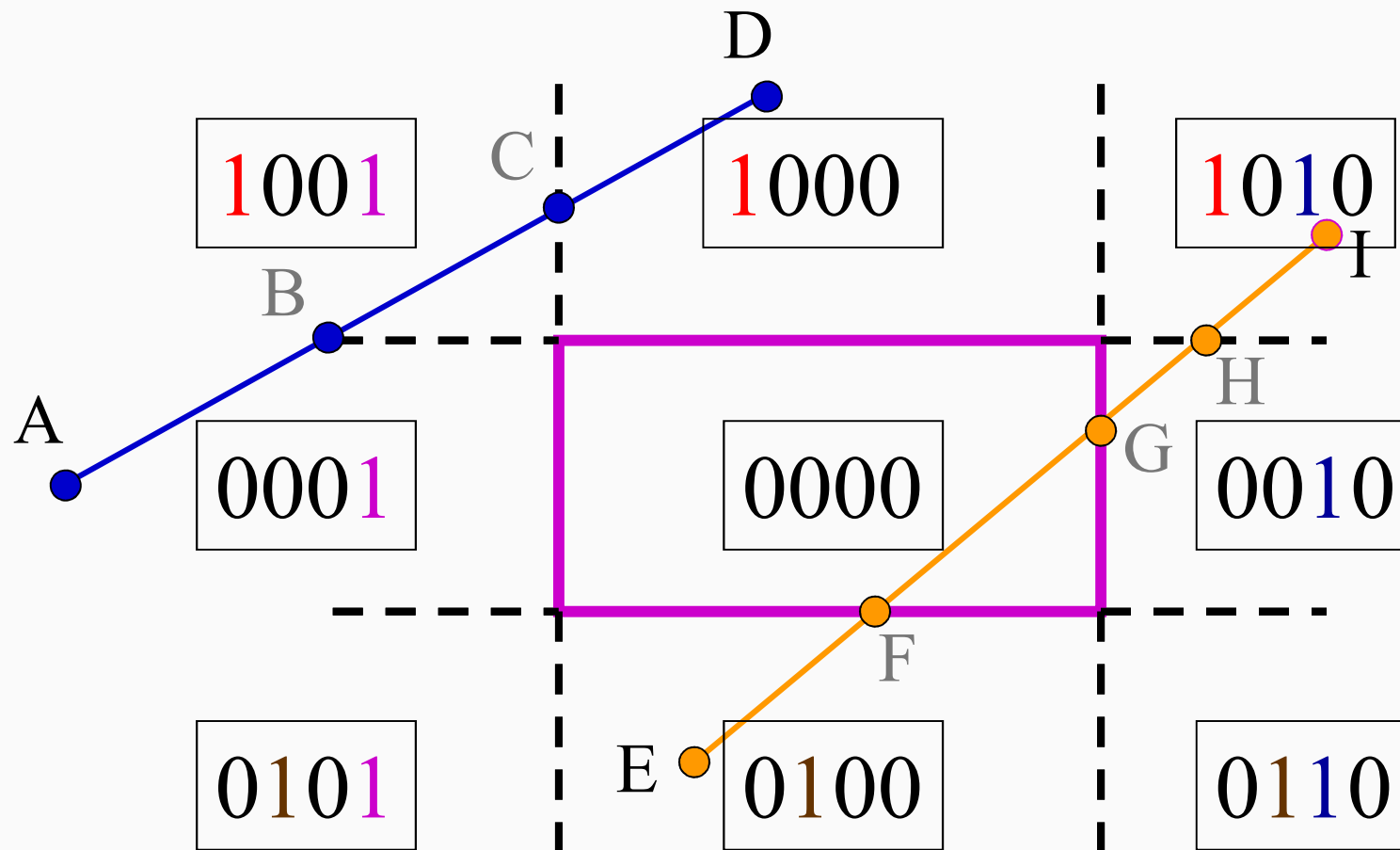
$\{C_0 \wedge C_1 = 0\} \Rightarrow$ Endpunkte eventuell
nicht im Fenster

4. Clippe an Kante i , für die $(C_0 \vee C_1)_i \neq 0$

5. Ermittle neue Outcodes (zurück zu 3.)

Cohen-Sutherland-Algorithmus (6)

Beispiel:



Cohen-Sutherland-Algorithmus (7)

Beispiel:

- Initiale Outcodes:

- $OC(D)=1000$; $OC(A)=0001$

$$1000 \wedge 0001 = 0000 \quad \checkmark$$

$$1000 \vee 0001 = \textcircled{1}001$$

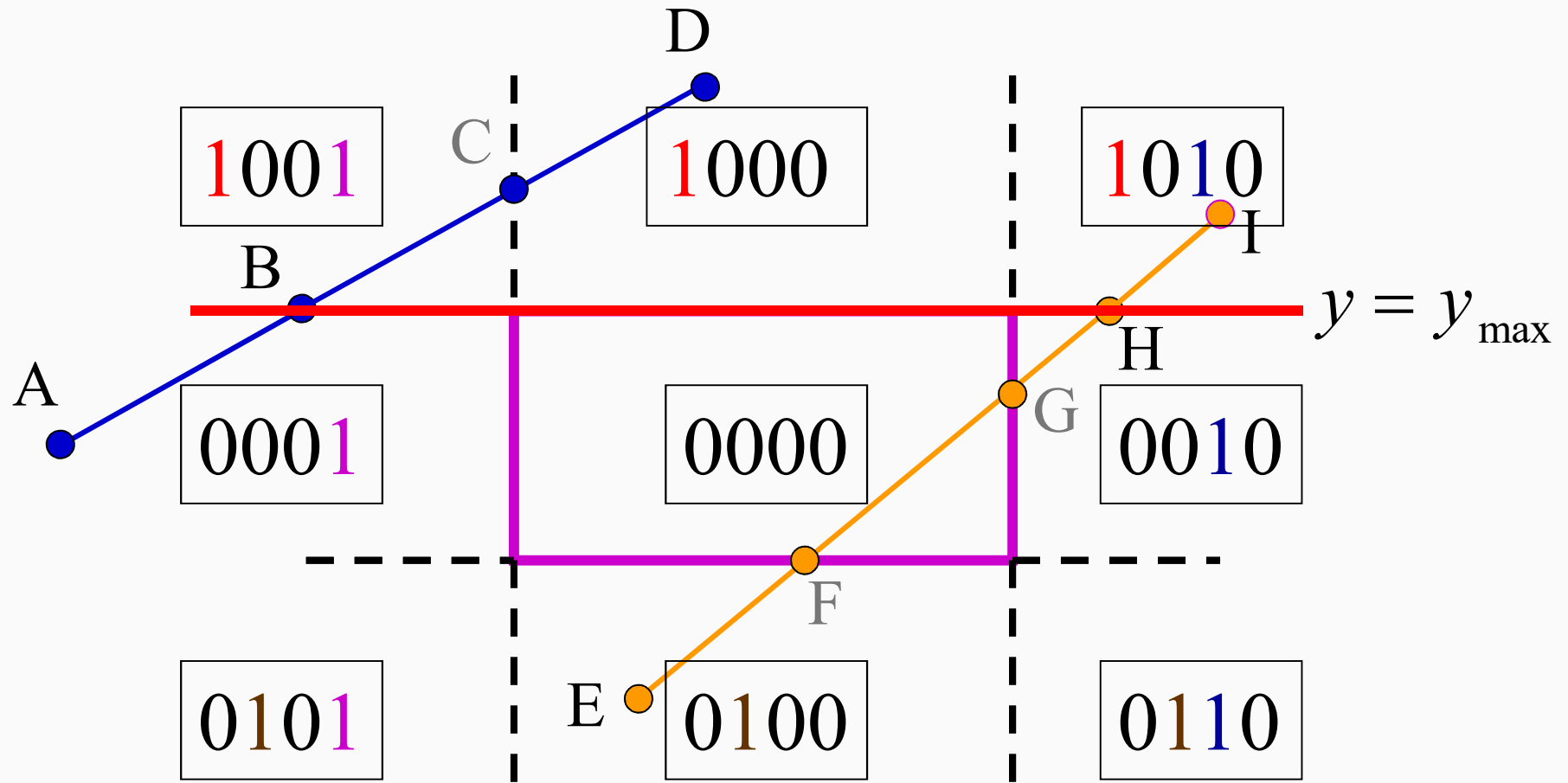
- $OC(E)=0100$; $OC(I)=1010$

$$0100 \wedge 1010 = 0000 \quad \checkmark$$

$$0100 \vee 1010 = \textcircled{1}110$$

Cohen-Sutherland-Algorithmus (8)

Beispiel:



Cohen-Sutherland-Algorithmus (9)

Beispiel:

- Neue Outcodes:
- $OC(B)=0001$; $OC(A)=0001$

$$0001 \wedge 0001 = 0001 \quad \times$$

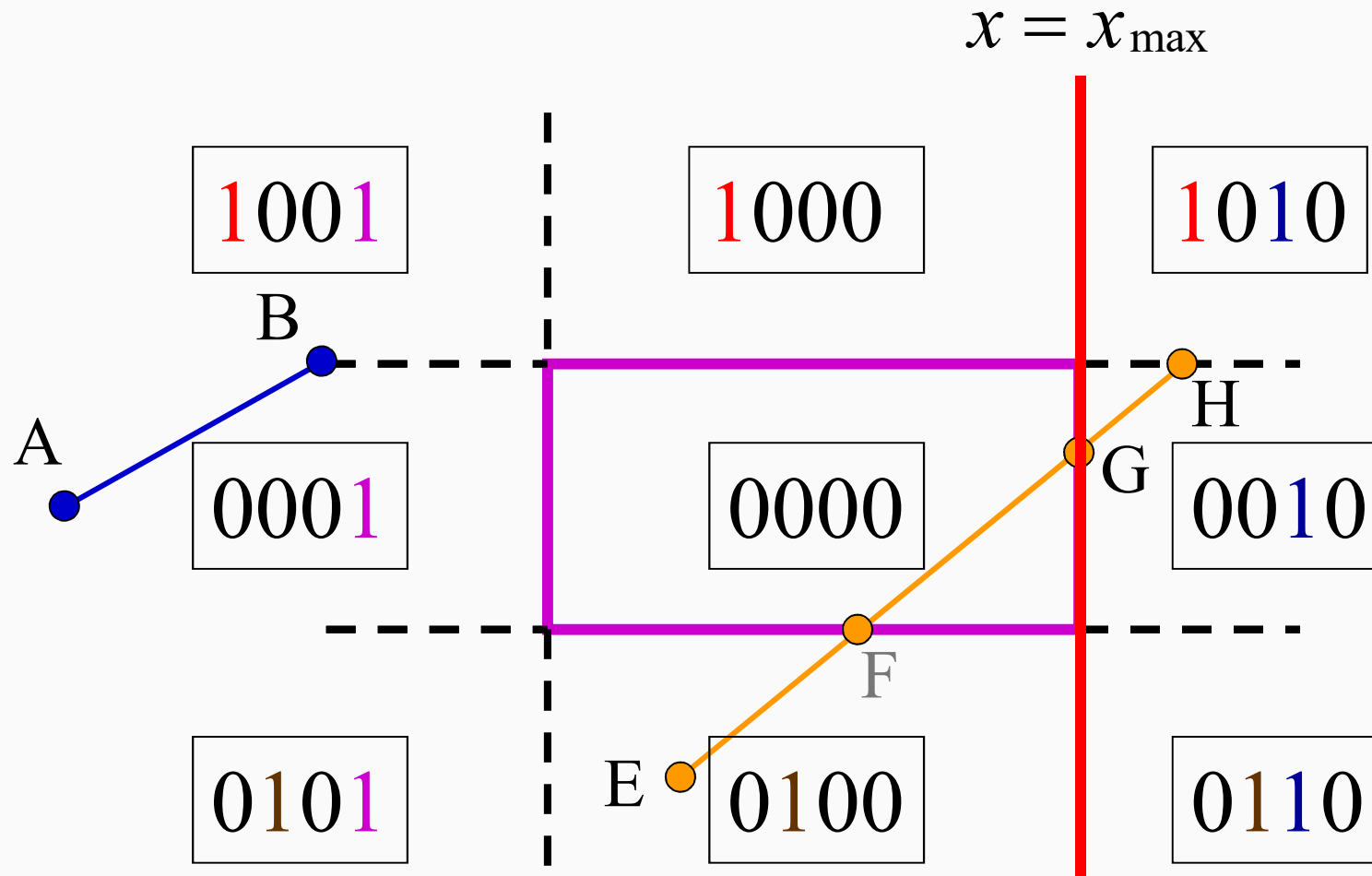
- $OC(E)=0100$; $OC(H)=0010$

$$0100 \wedge 0010 = 0000 \quad \checkmark$$

$$0100 \vee 0010 = 01\textcircled{1}0$$

Cohen-Sutherland-Algorithmus (10)

Beispiel:



Cohen-Sutherland-Algorithmus (11)

Beispiel:

- Neue Outcodes:

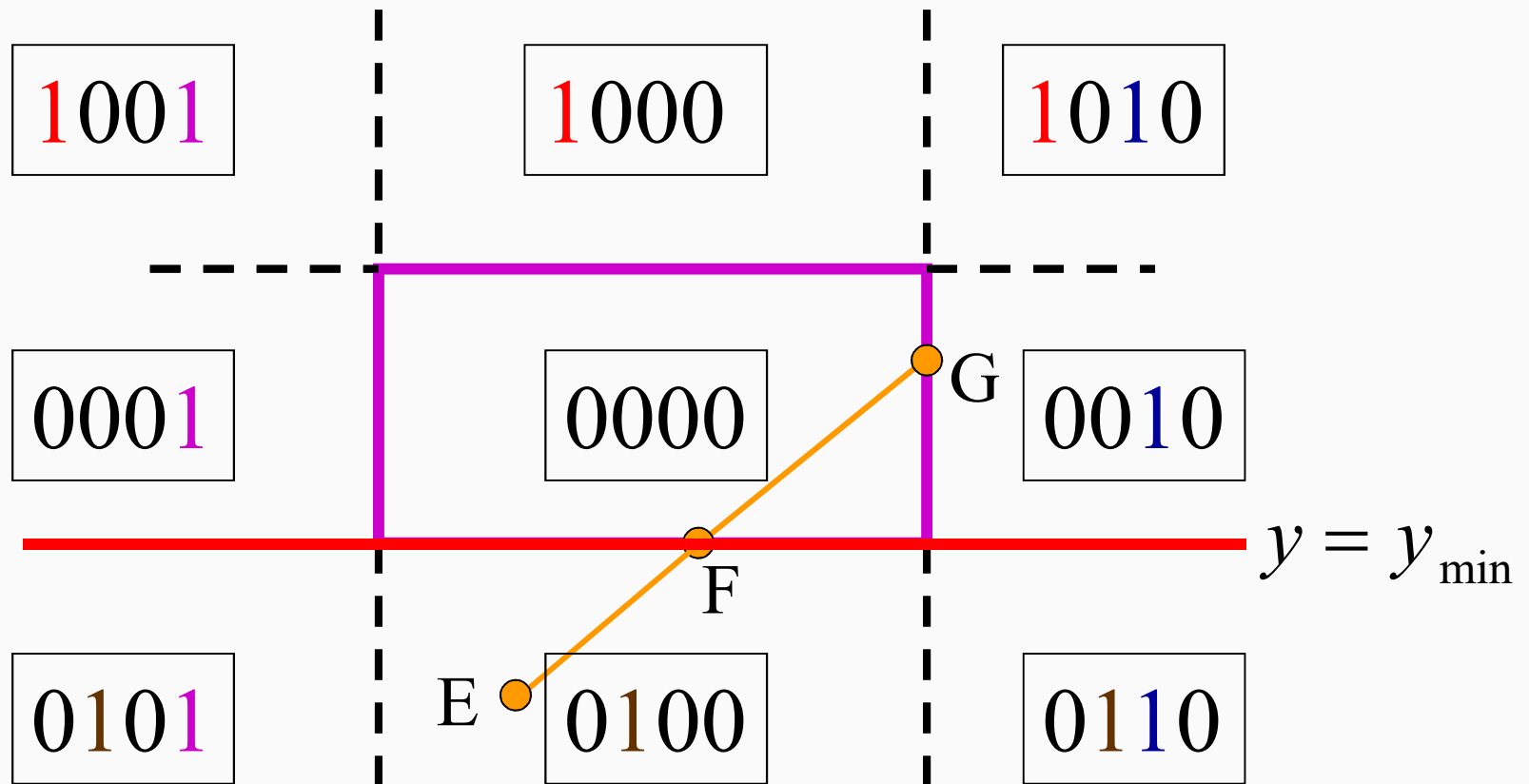
- $OC(E)=0100$; $OC(G)=0000$

$$0100 \wedge 0000 = 0000 \quad \checkmark$$

$$0100 \vee 0000 = 0100$$

Cohen-Sutherland-Algorithmus (12)

Beispiel:



Cohen-Sutherland-Algorithmus (13)

Beispiel:

- Neue Outcodes:

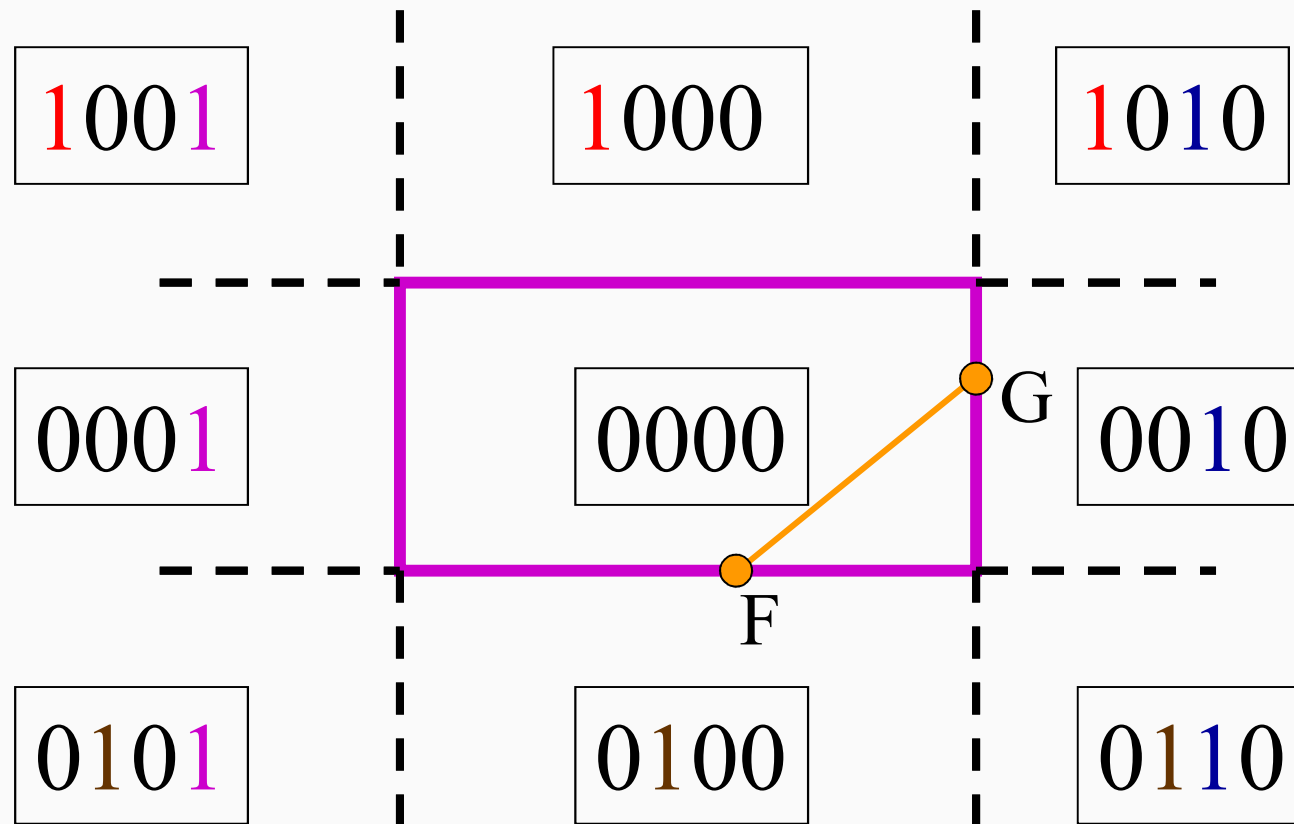
- $OC(F)=0000$; $OC(G)=0000$

$$0000 \wedge 0000 = 0000 \checkmark$$

$$0000 \vee 0000 = 0000 \checkmark$$

Cohen-Sutherland-Algorithmus (14)

Beispiel:

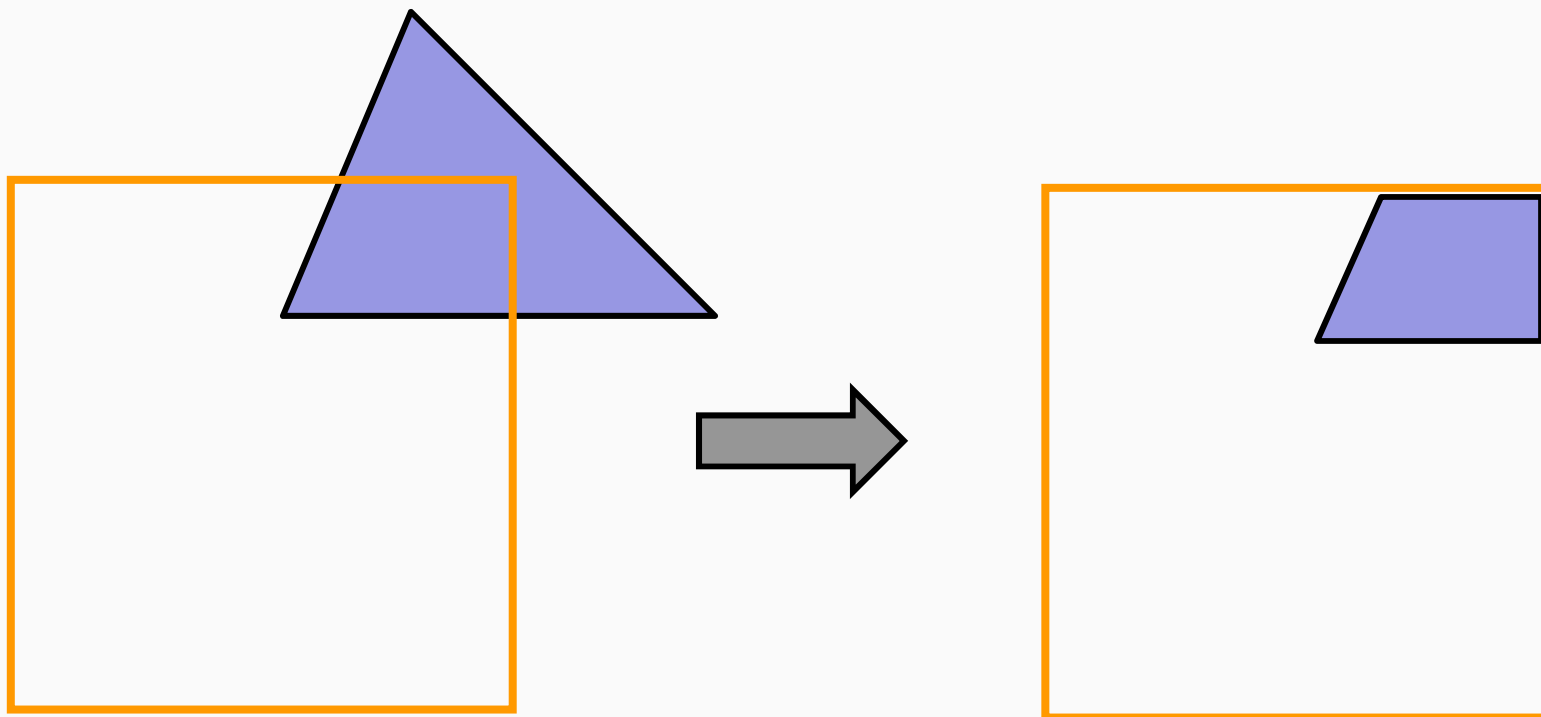


Cohen-Sutherland-Algorithmus (15)

- Wann ist der Algorithmus gut?
 - Wenn er in den meisten Fällen **trivial akzeptiert**, bzw. **verwirft**
 - Wenn das Fenster groß im Verhältnis zur Linie ist
 - Wenn das Fenster klein im Verhältnis zur Linie ist
- D.h., er funktioniert in den **Extremfällen** gut, wenn die meisten Entscheidungen trivial sind.

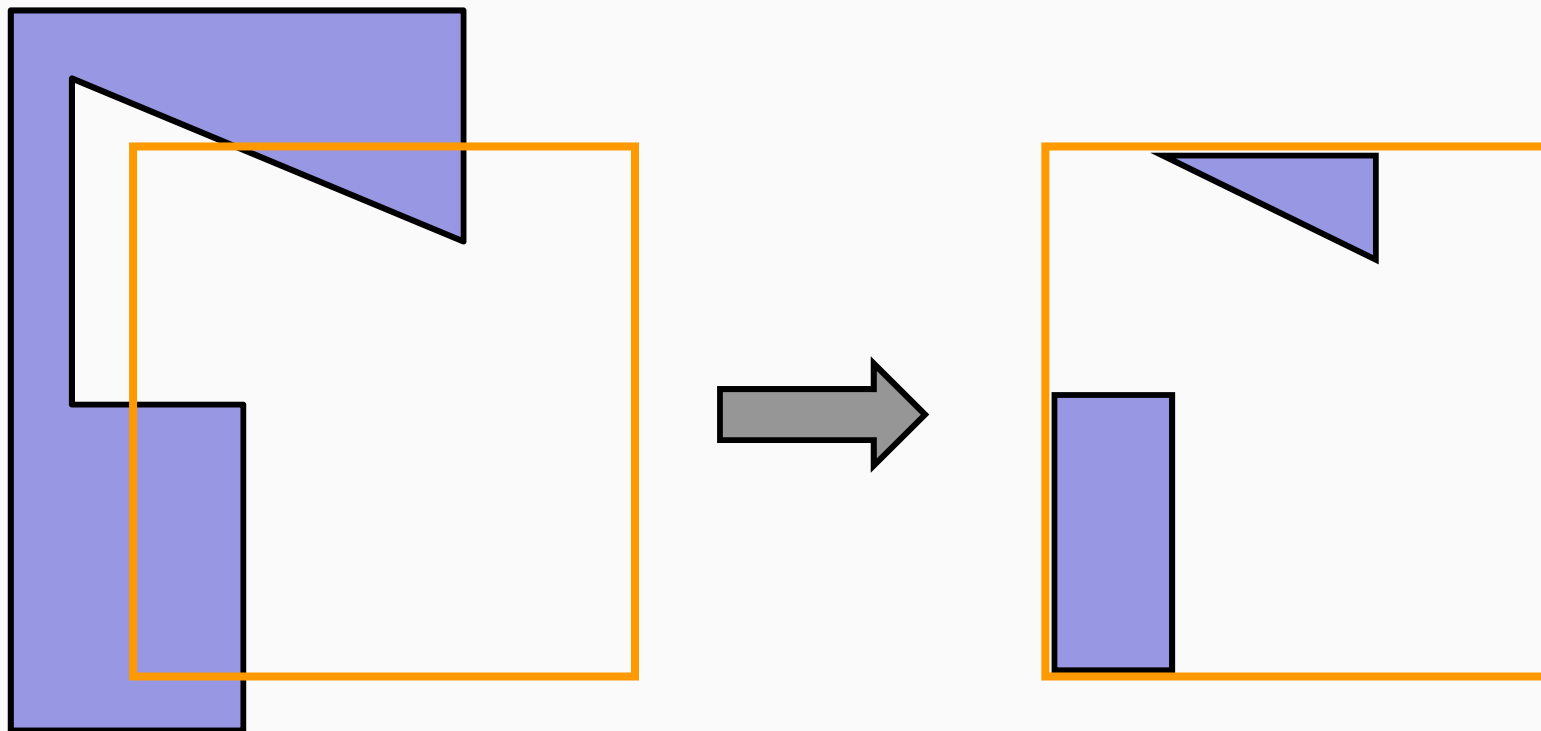
Polygon Clipping (1)

Beispiel: einfaches Polygon



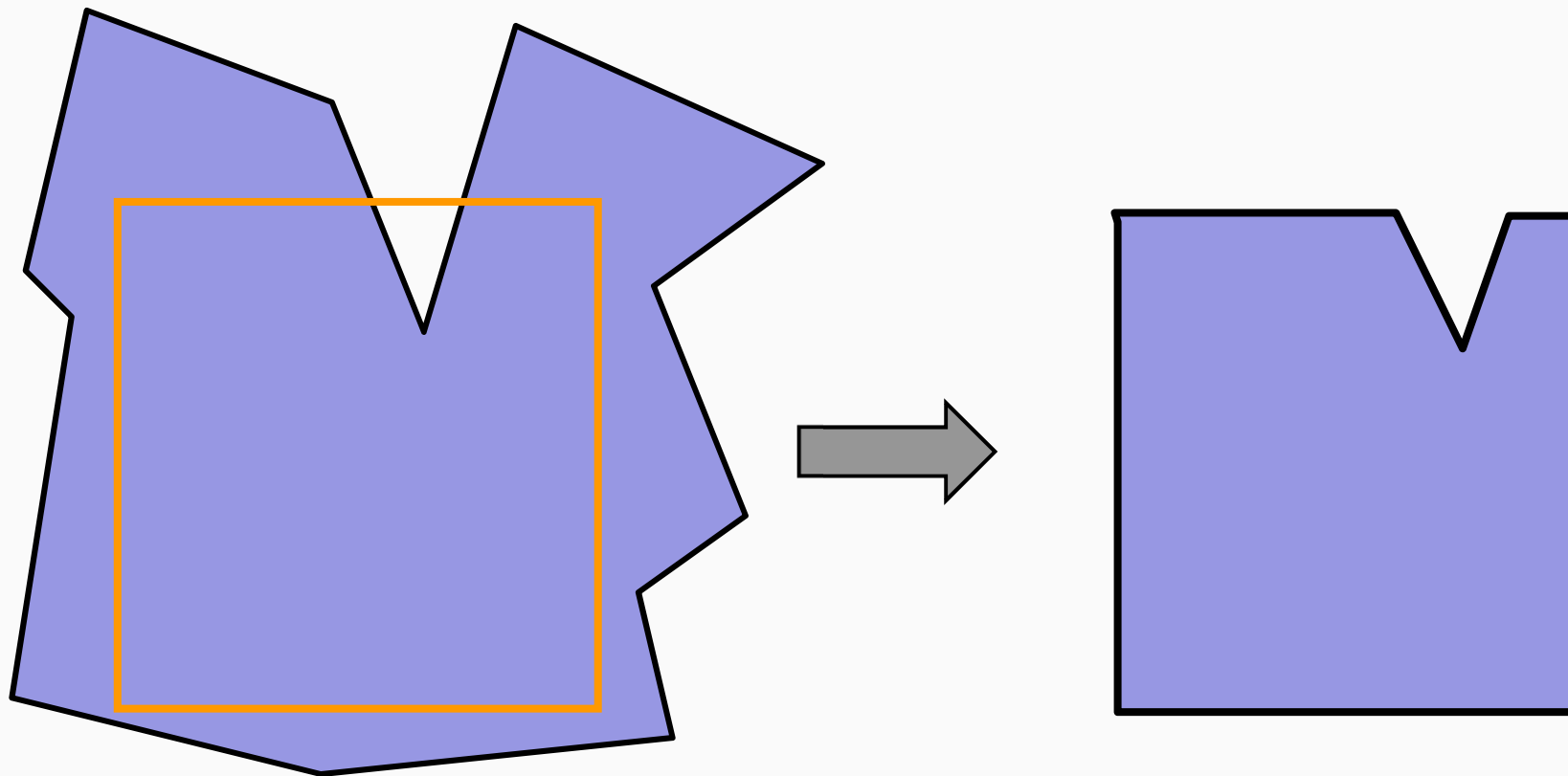
Polygon Clipping (2)

Beispiel: mehrere Teile



Polygon Clipping (3)

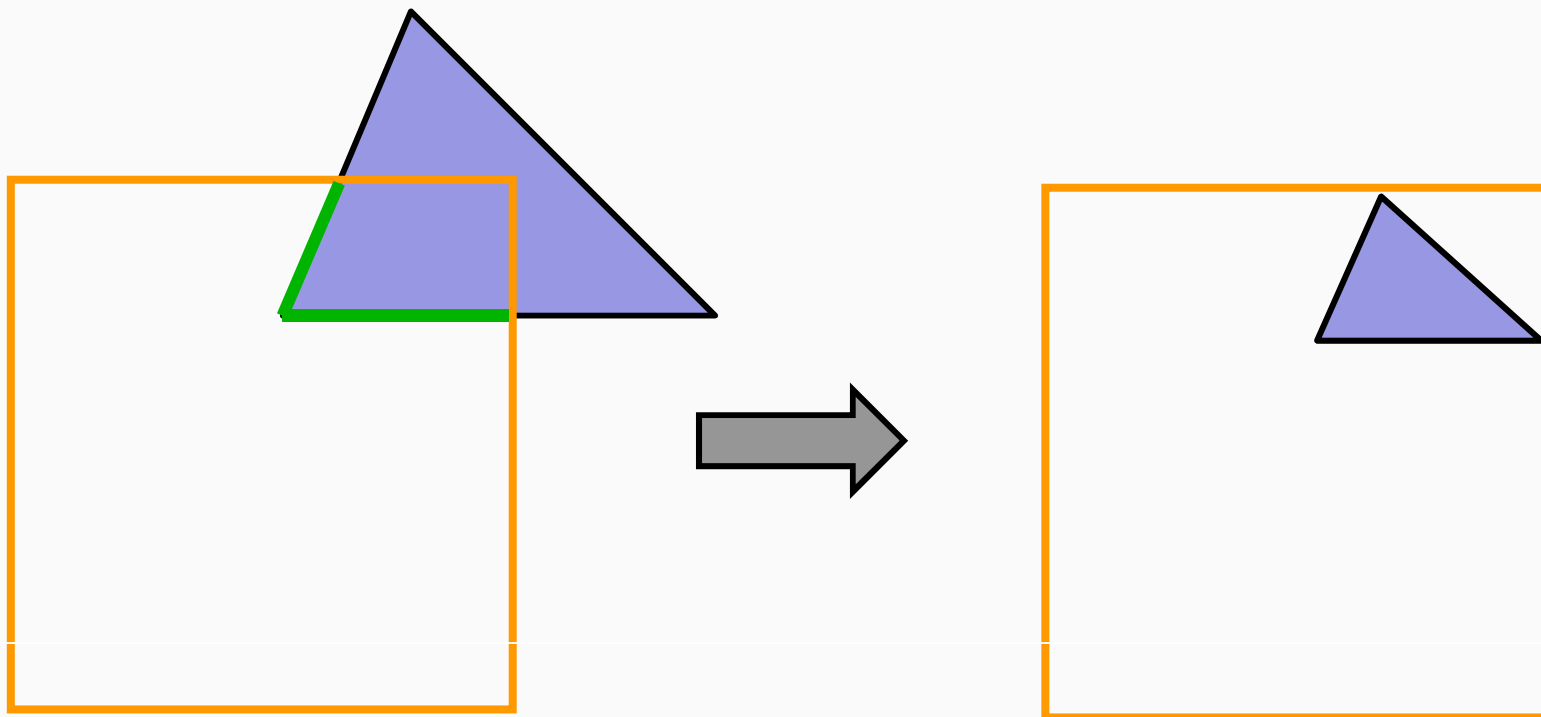
Beispiel: komplexes nichtkonvexes Polygon



Polygon Clipping (4)

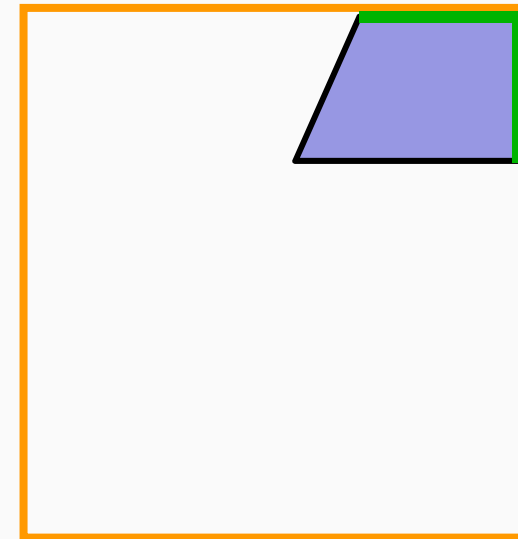
Polygon Clipping $\neq$ Clipping der Liniensegmente!

Polygon Clipping = Clipping der Liniensegmente!

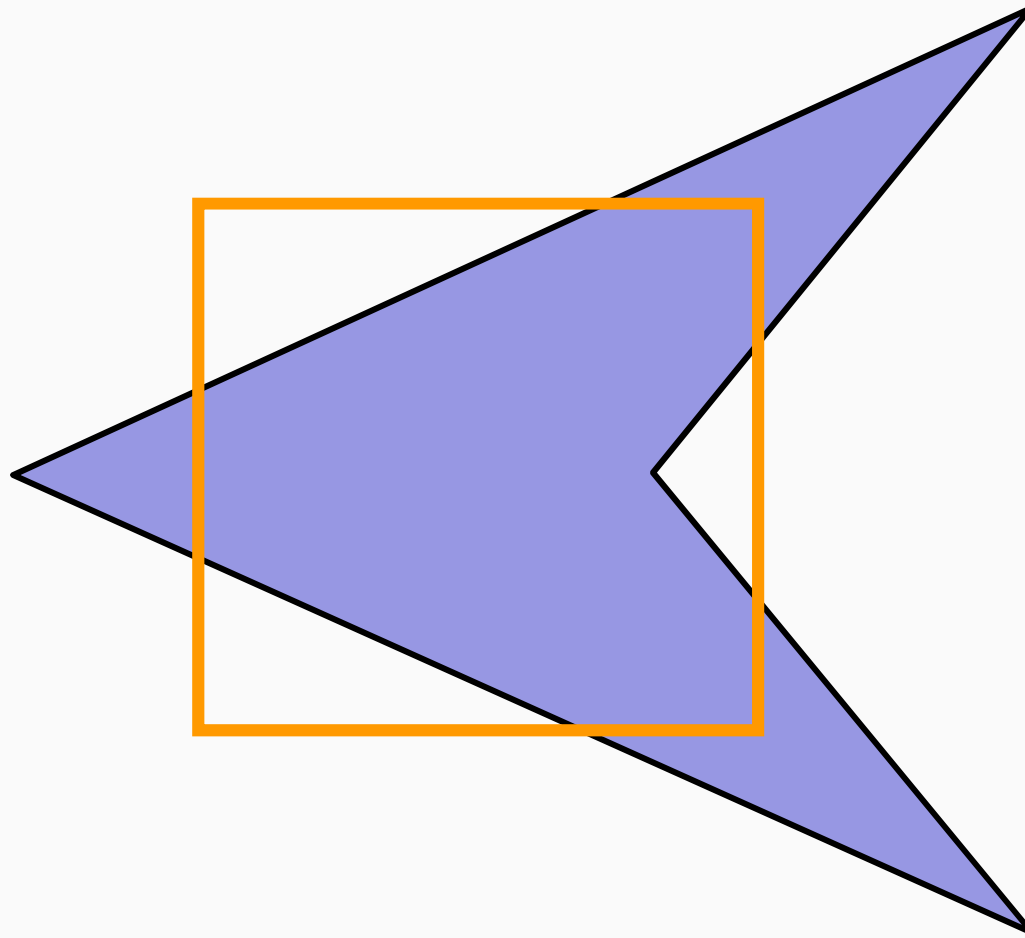


Polygon Clipping (5)

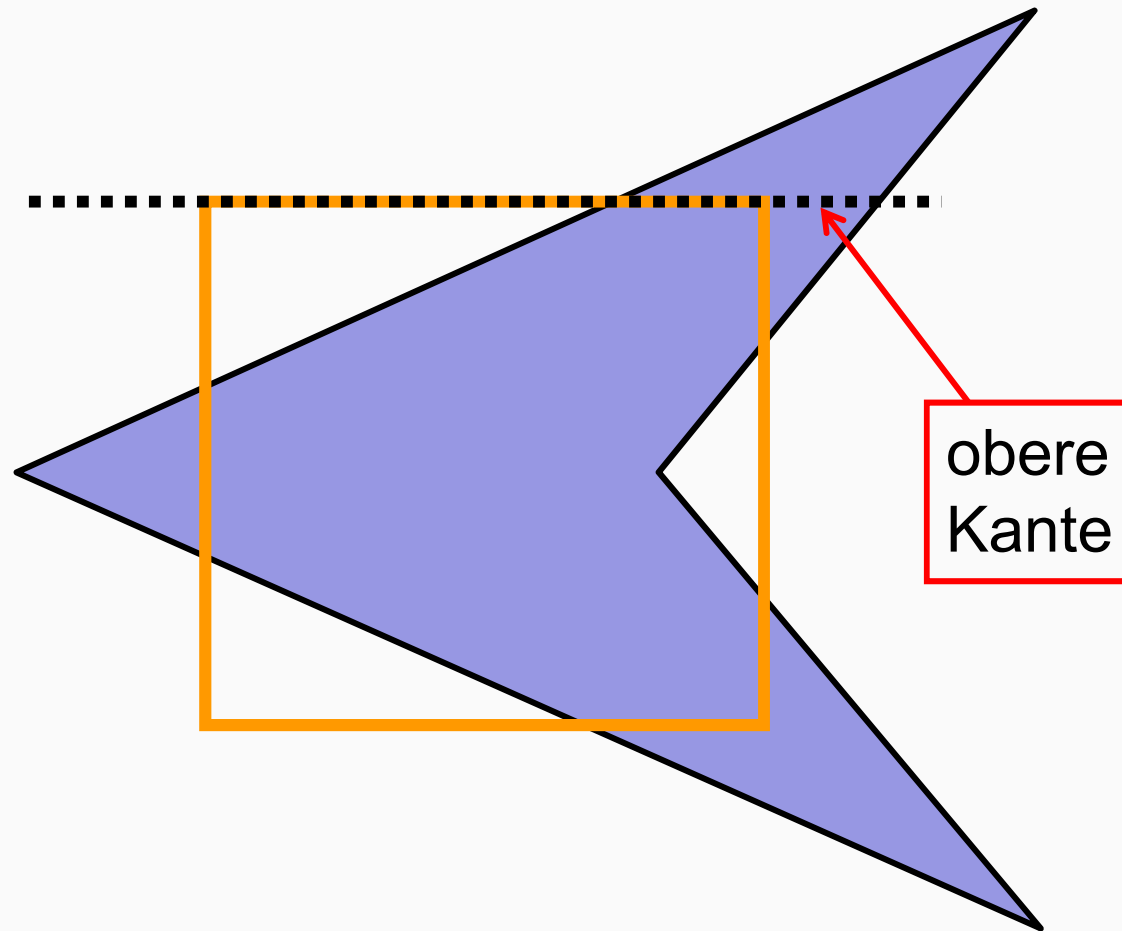
- Algorithmus muss als **Ergebnis** wieder **geschlossene Polygone** liefern.
 - Nur durch Einbeziehung von von Teilen der Fensterbegrenzung möglich.
 - Einfachste Methode: Umkehrung der Schleifenschachtelung:
 - Das gesamte Polygon wird zunächst jeweils an einer Fenstergrenze geclippt.
 - Dies ist die wesentliche Idee des Sutherland-Hodgman-Algorithmus.



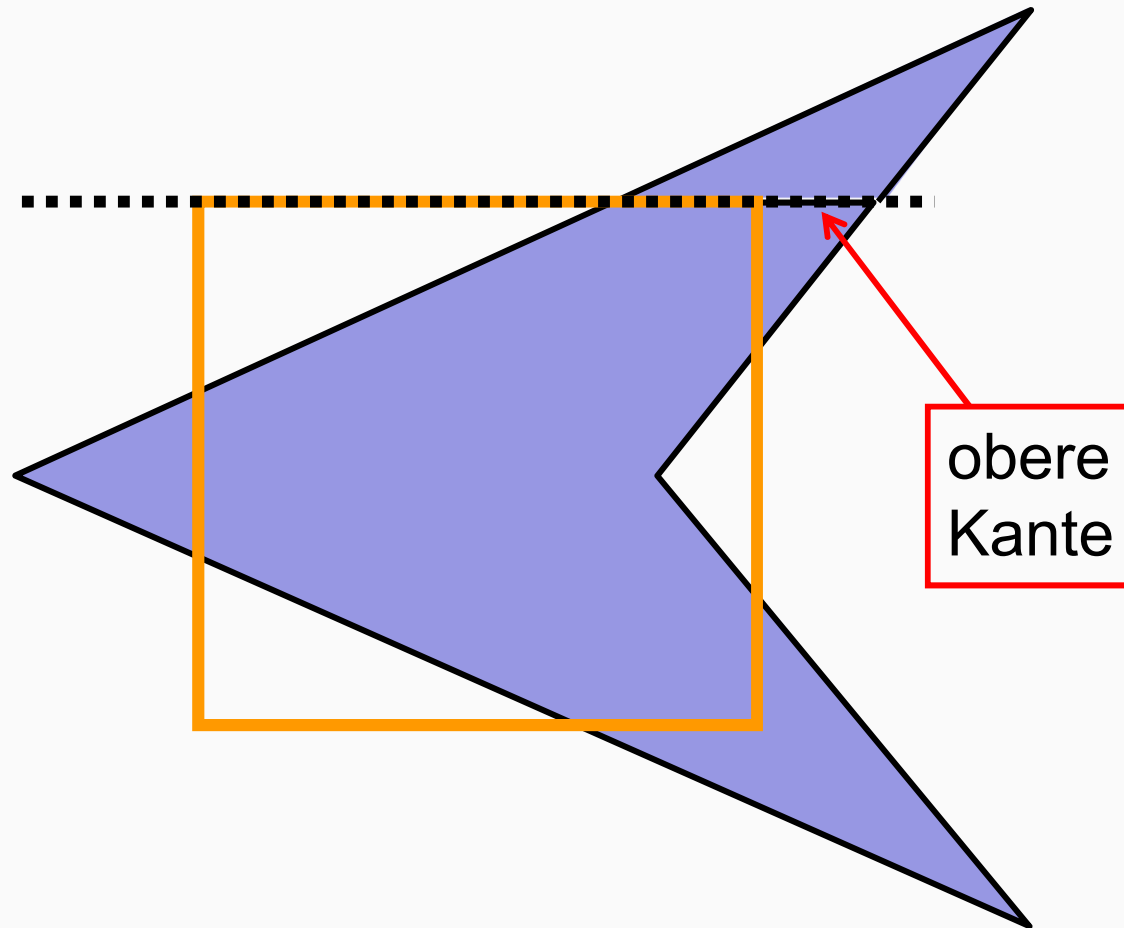
Sutherland-Hodgman-Algorithmus (1)



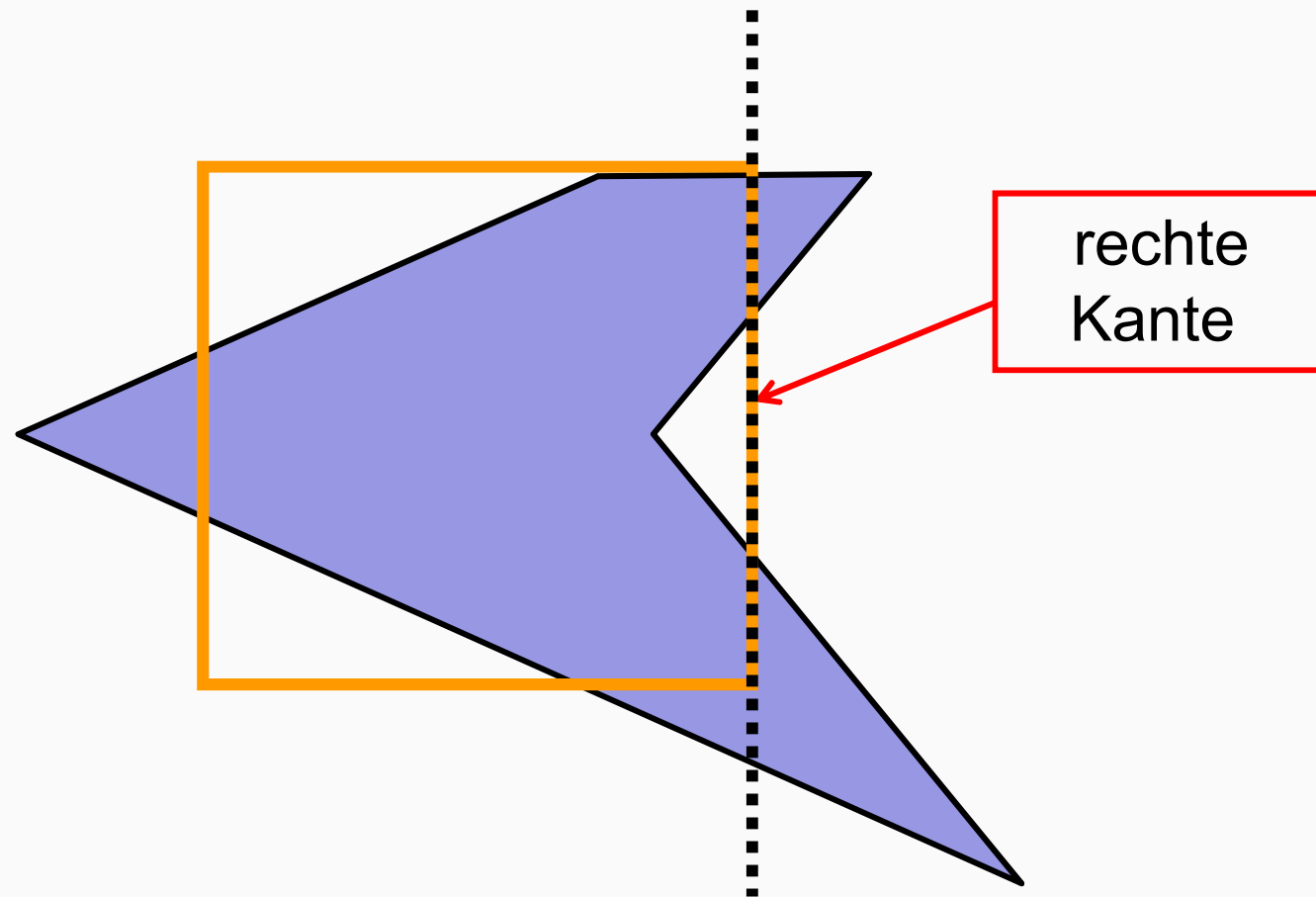
Sutherland-Hodgman-Algorithmus (2)



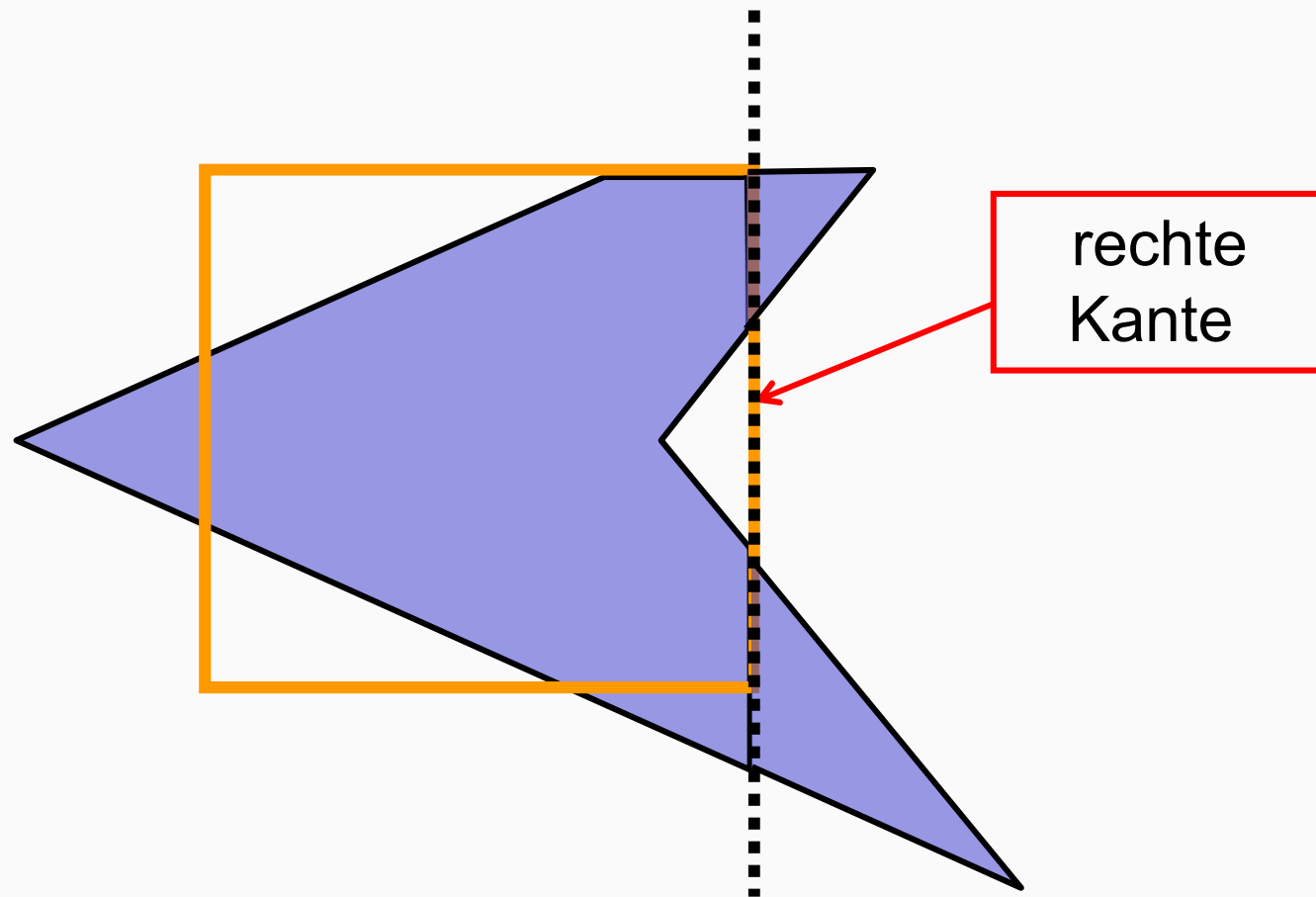
Sutherland-Hodgman-Algorithmus (3)



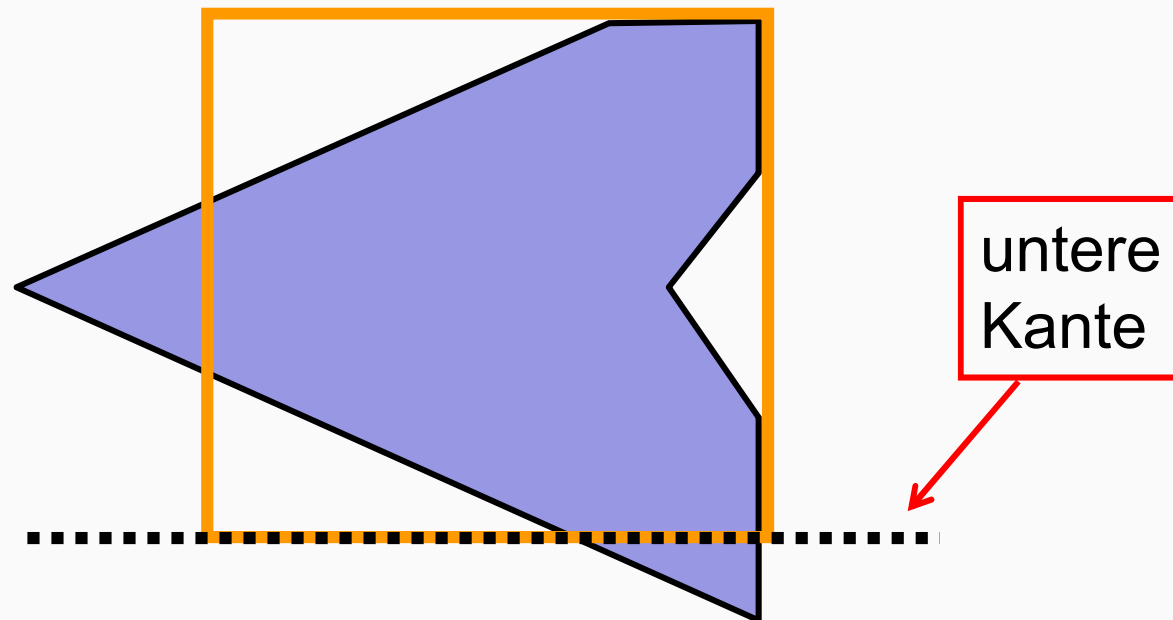
Sutherland-Hodgman-Algorithmus (4)



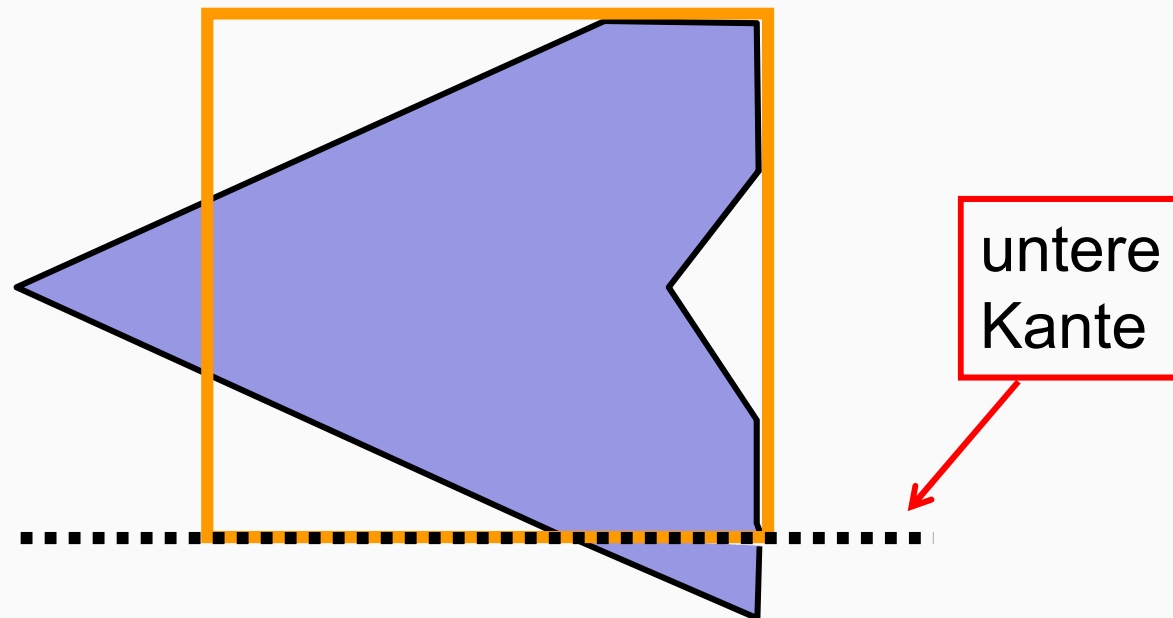
Sutherland-Hodgman-Algorithmus (5)



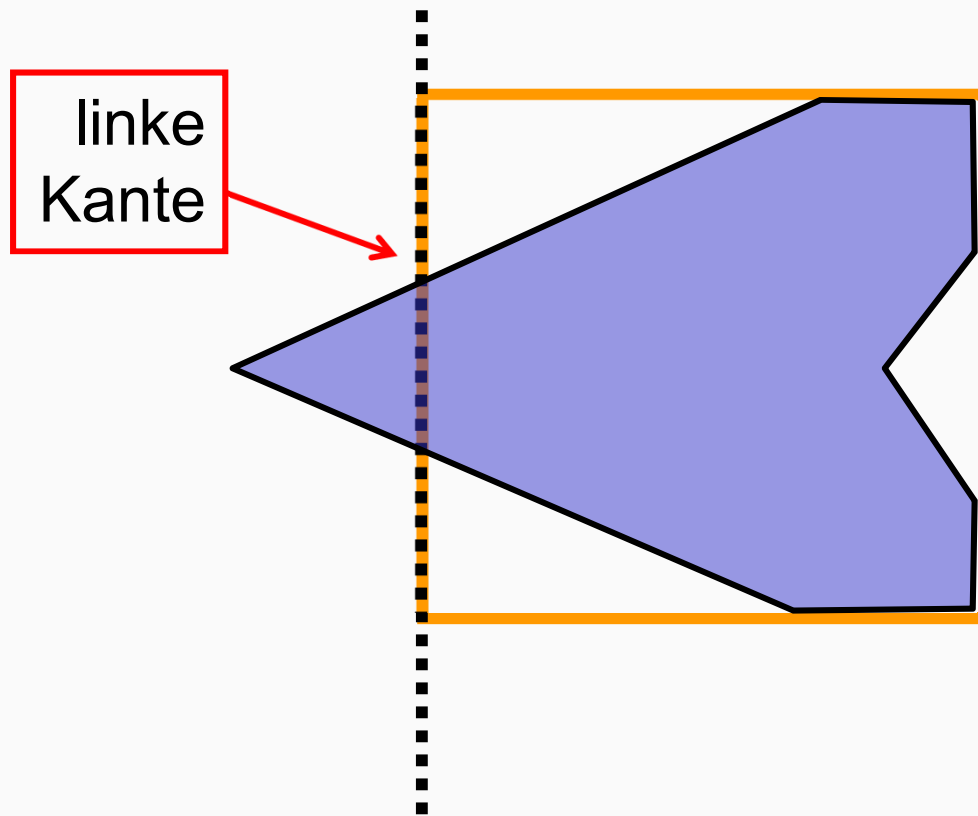
Sutherland-Hodgman-Algorithmus (6)



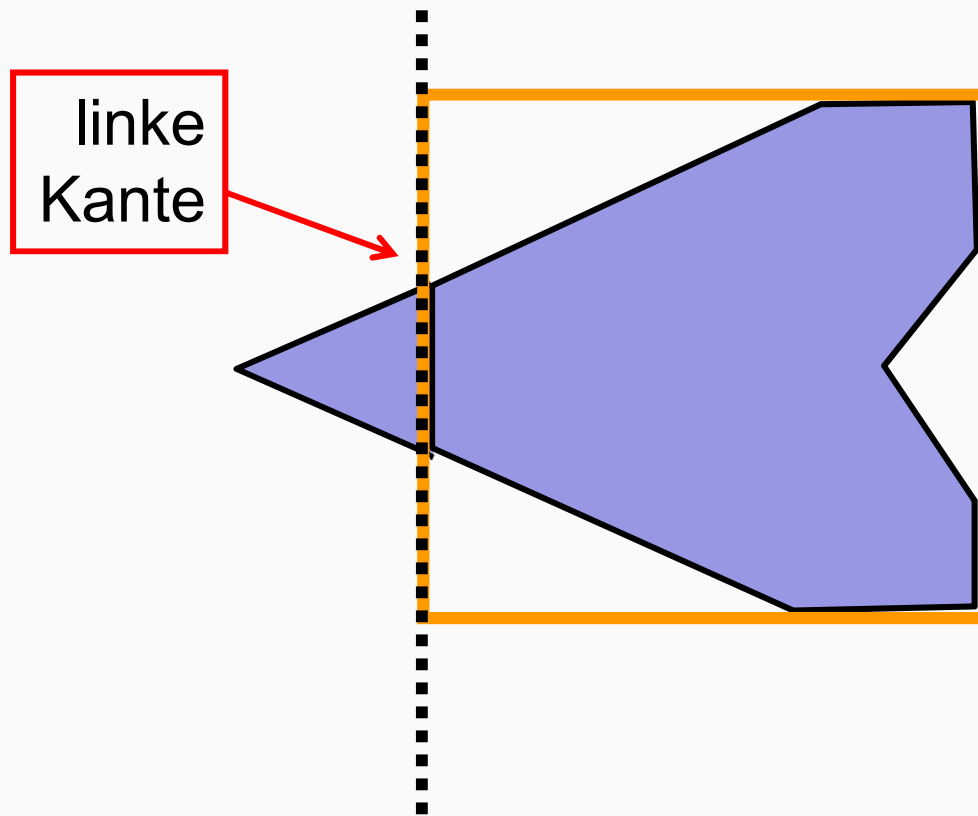
Sutherland-Hodgman-Algorithmus (7)



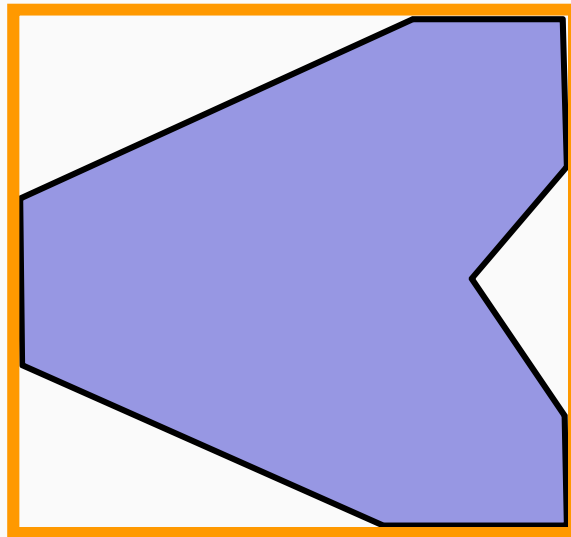
Sutherland-Hodgman-Algorithmus (8)



Sutherland-Hodgman-Algorithmus (9)



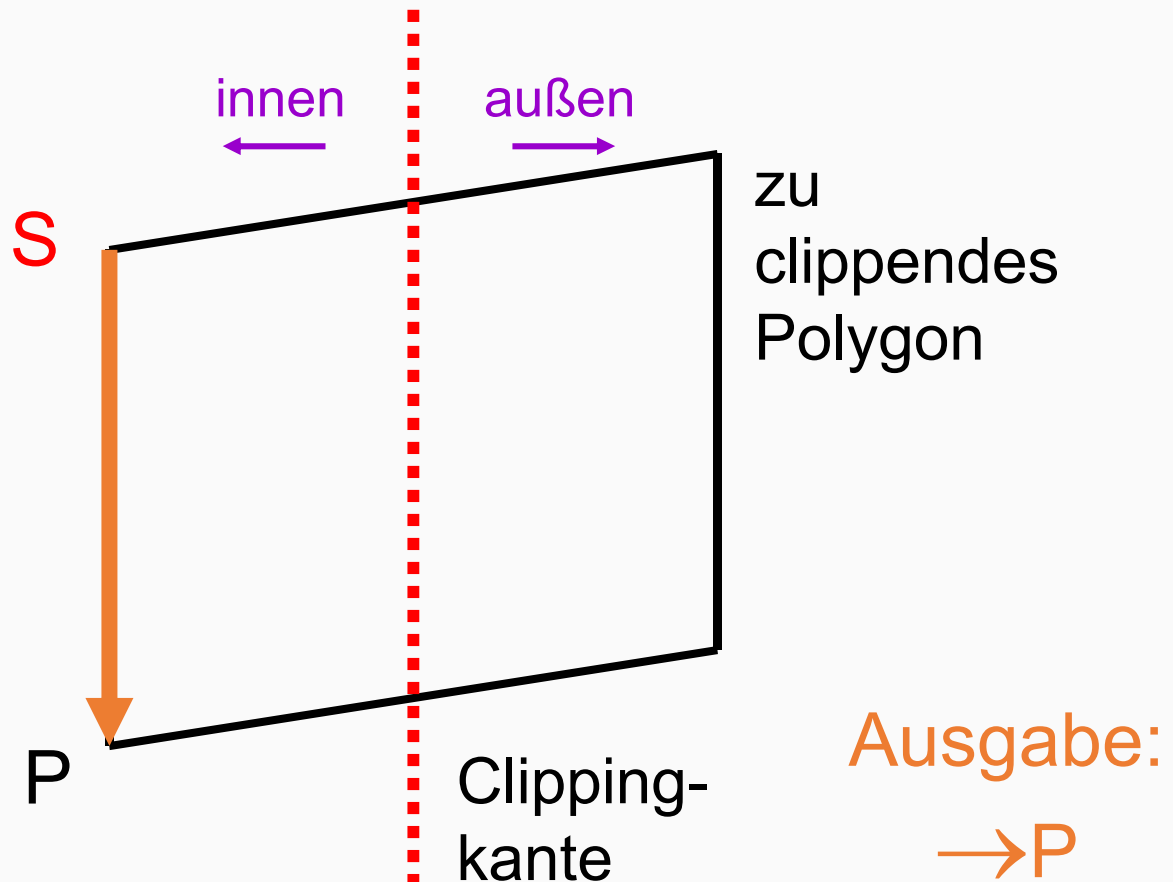
Sutherland-Hodgman-Algorithmus (10)



Sutherland-Hodgman-Algorithmus (11)

- Fall 1:
 - S und P innen

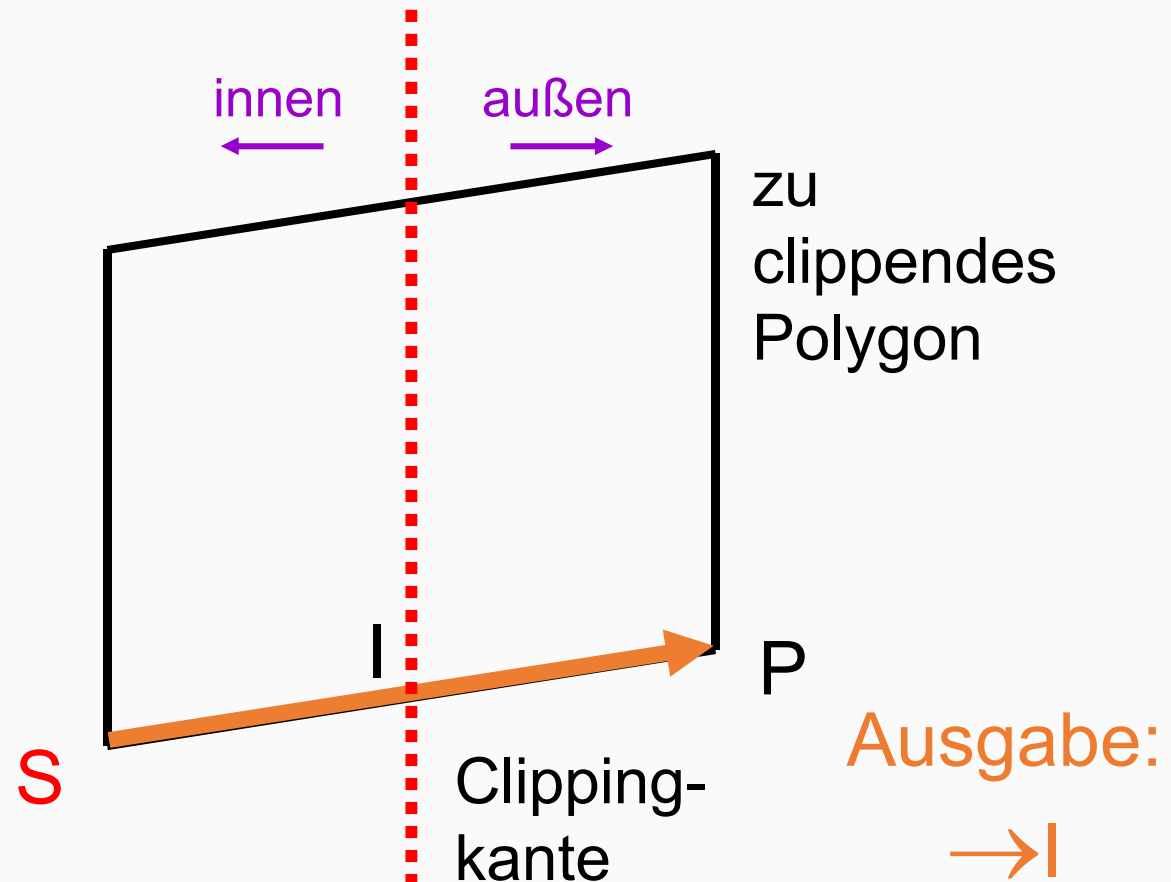
Eingabe:
 $(S) \rightarrow P$



Sutherland-Hodgman-Algorithmus (12)

- Fall 2:
 - S innen
 - P außen

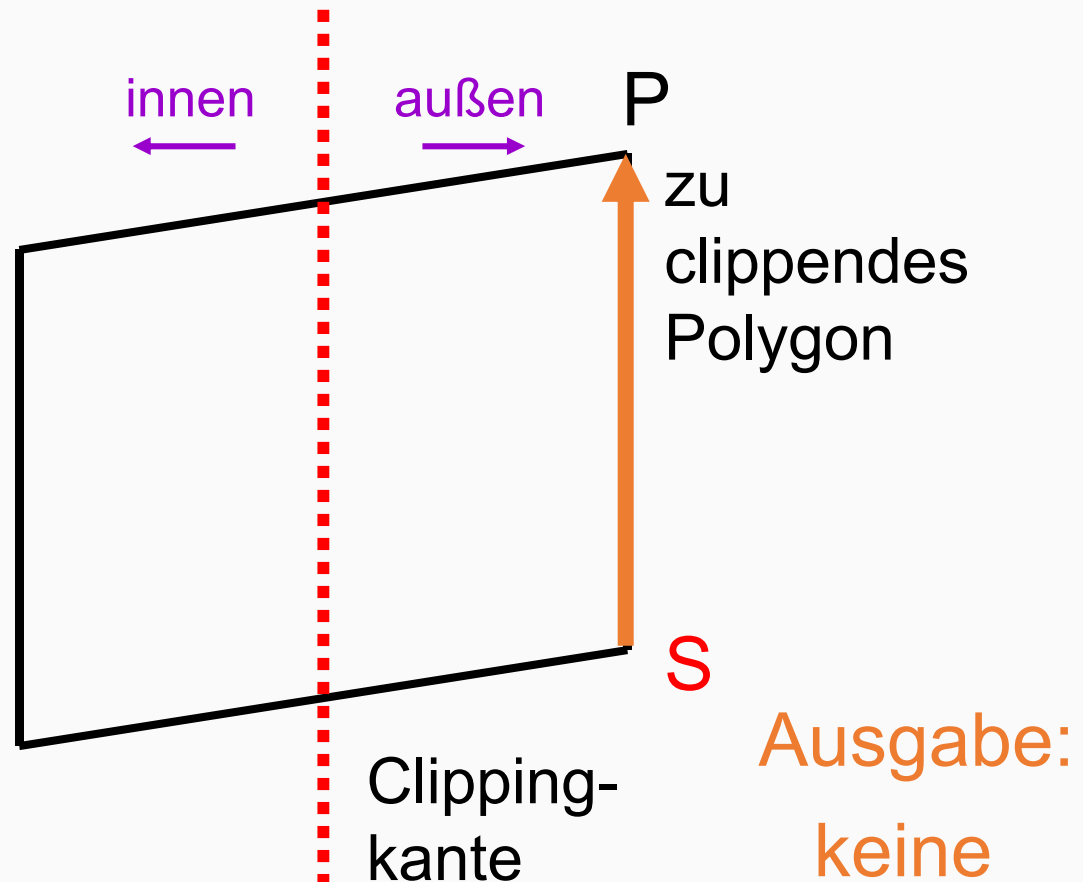
Eingabe:
 $(S) \rightarrow P$



Sutherland-Hodgman-Algorithmus (13)

- Fall 3:
 - S und P außen

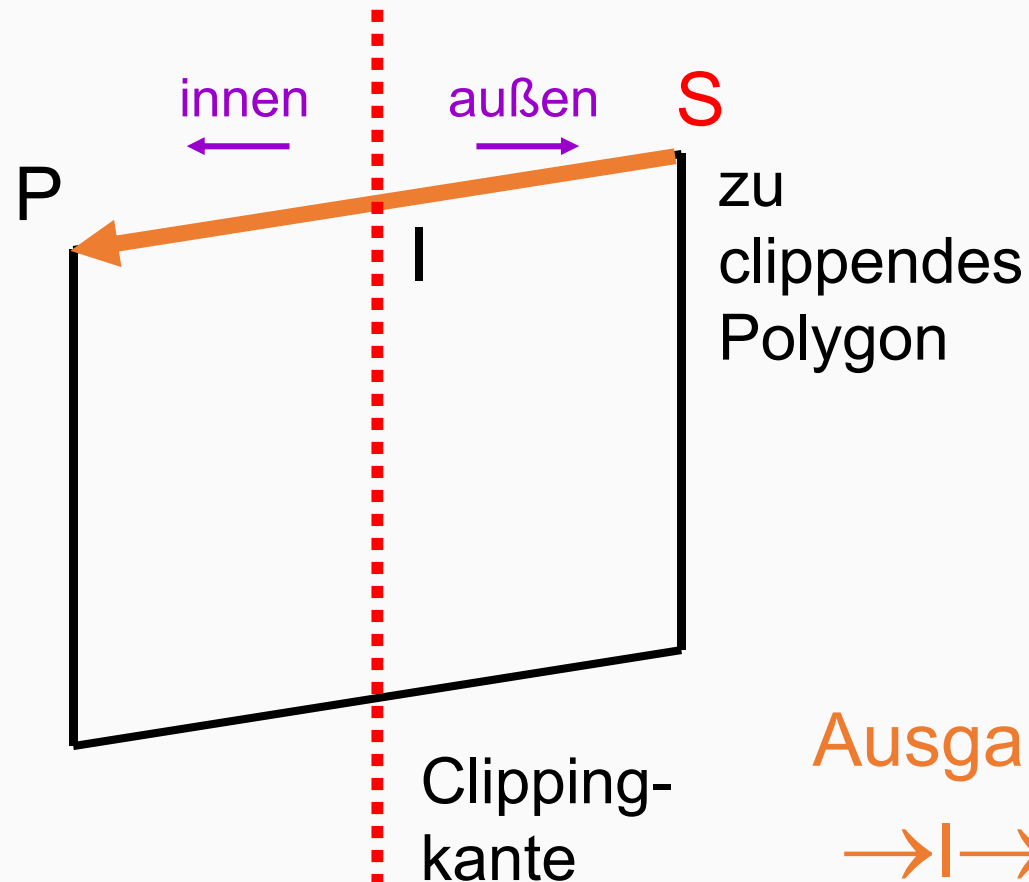
Eingabe:
 $(S) \rightarrow P$



Sutherland-Hodgman-Algorithmus (14)

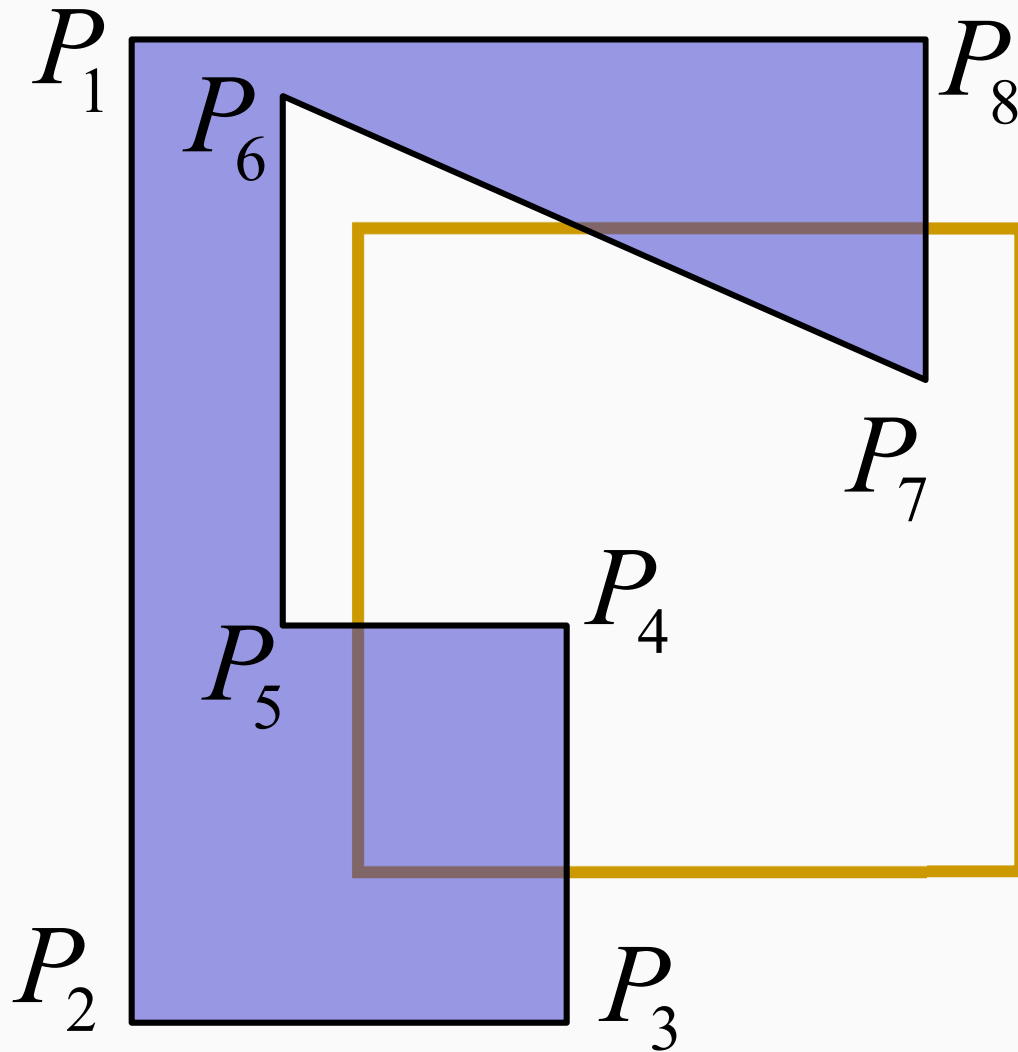
- Fall 4:
 - S außen
 - P innen

Eingabe:
 $(S) \rightarrow P$



Ausgabe:
 $\rightarrow I \rightarrow P$

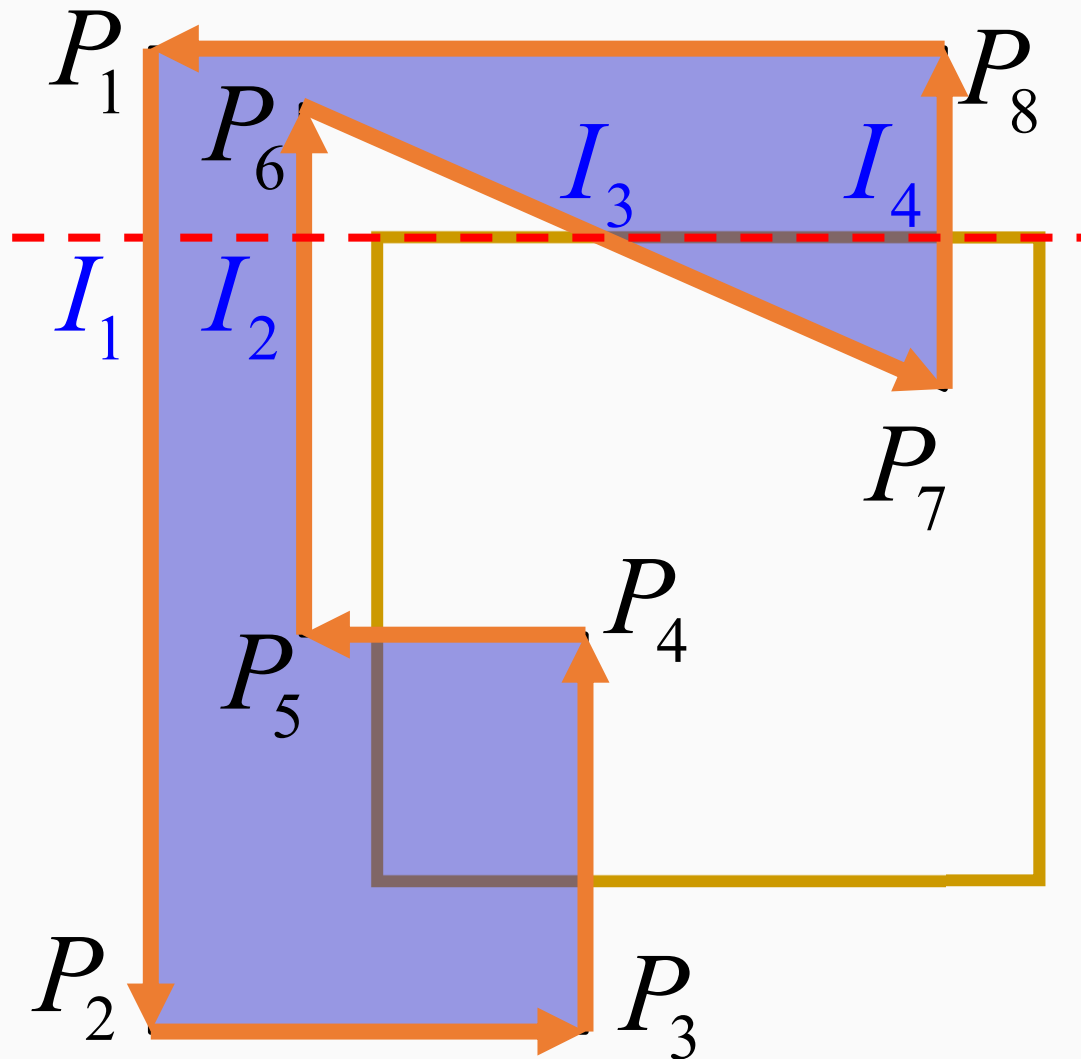
Sutherland-Hodgman-Algorithmus (15)



Eingabe:

$P_1 \ P_2 \ P_3 \ P_4 \ P_5 \ P_6 \ P_7 \ P_8$

Sutherland-Hodgman-Algorithmus (16)



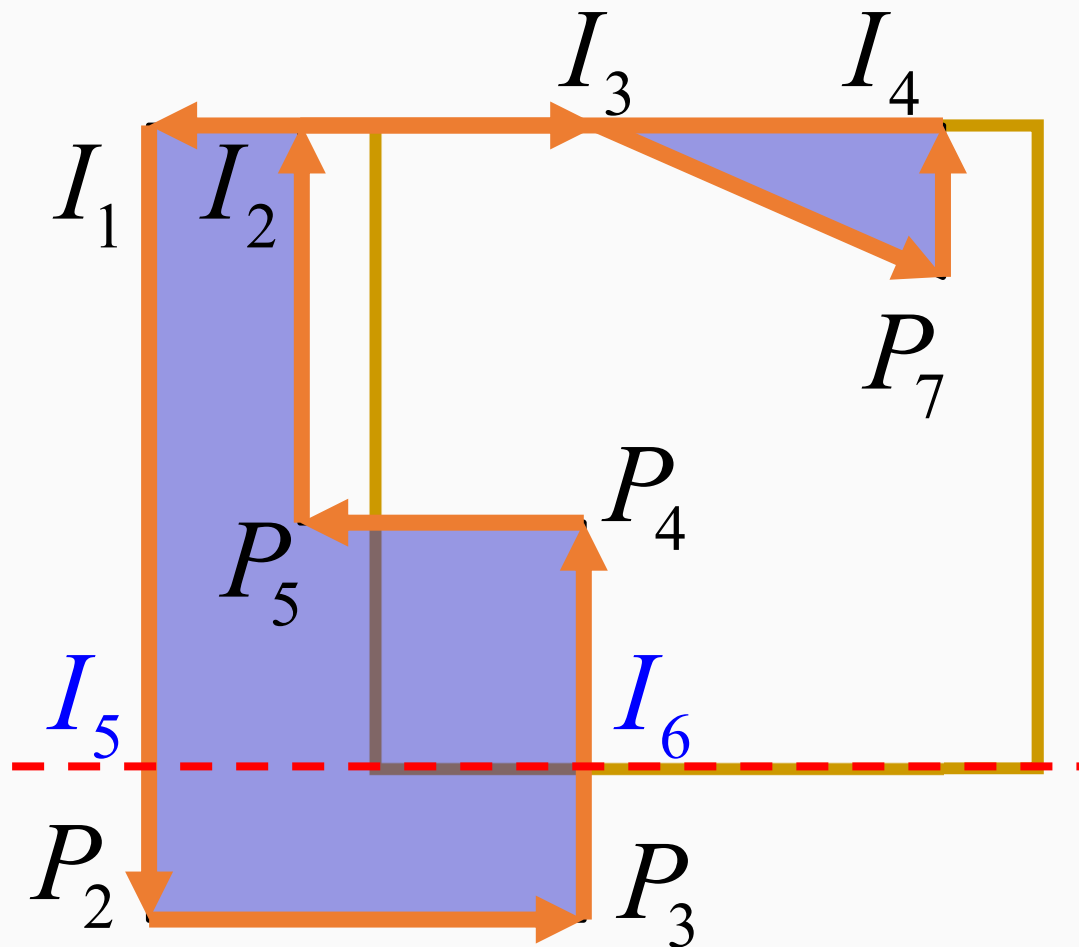
Eingabe:

$P_1 P_2 P_3 P_4 P_5 P_6 P_7 P_8$

Ausgabe:

$I_1 P_2 P_3 P_4 P_5 I_2 I_3 P_7 I_4$

Sutherland-Hodgman-Algorithmus (17)



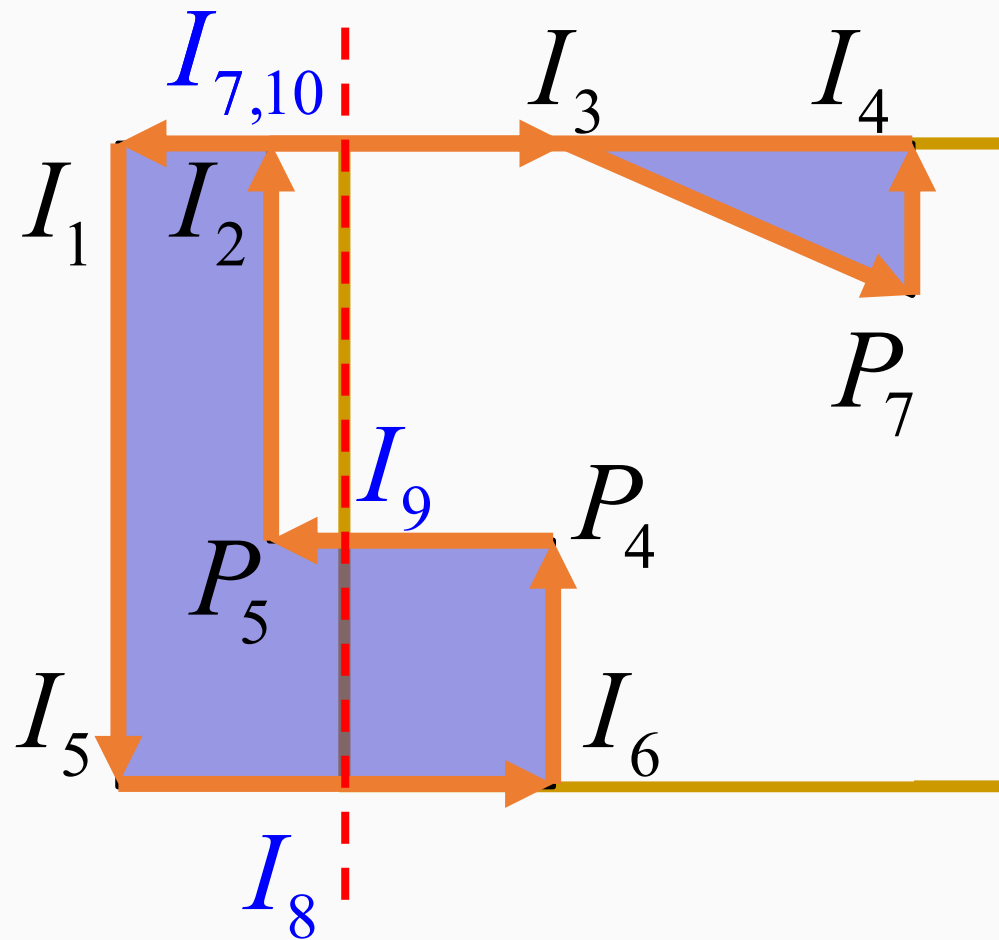
Eingabe:

$I_1 P_2 P_3 P_4 P_5 I_2 I_3 P_7 I_4$

Ausgabe:

$I_1 I_5 I_6 P_4 P_5 I_2 I_3 P_7 I_4$

Sutherland-Hodgman-Algorithmus (18)



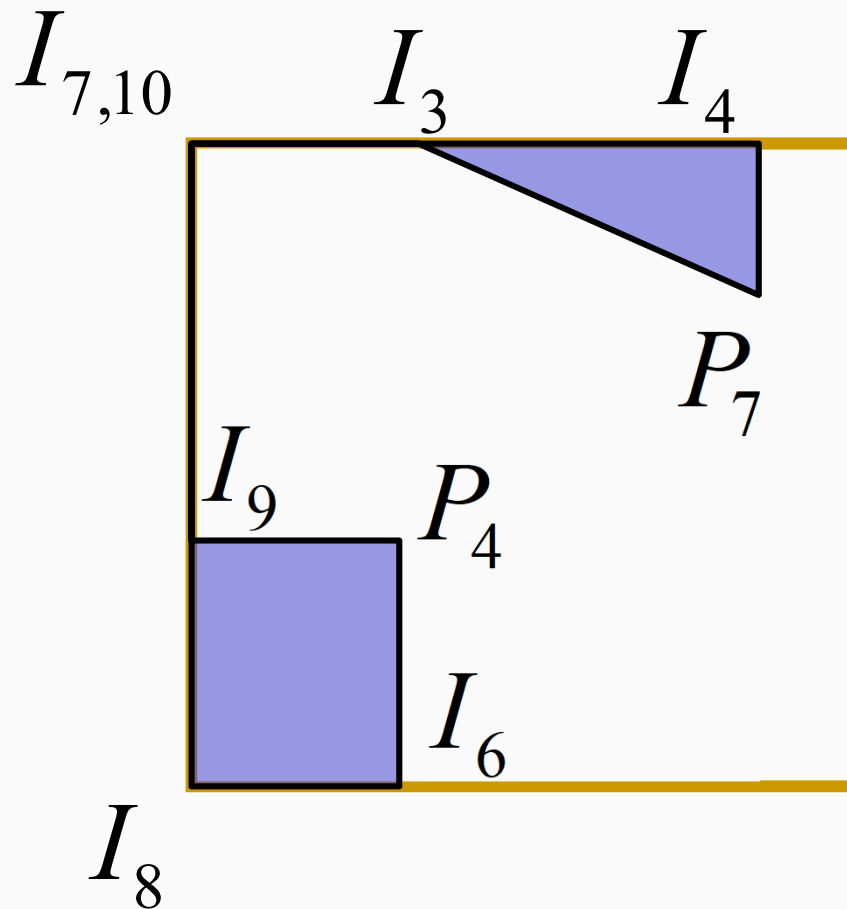
Eingabe:

$I_1 I_5 I_6 P_4 P_5 I_2 I_3 P_7 I_4$

Ausgabe:

$I_7 I_8 I_6 P_4 I_9 I_{10} I_3 P_7 I_4$

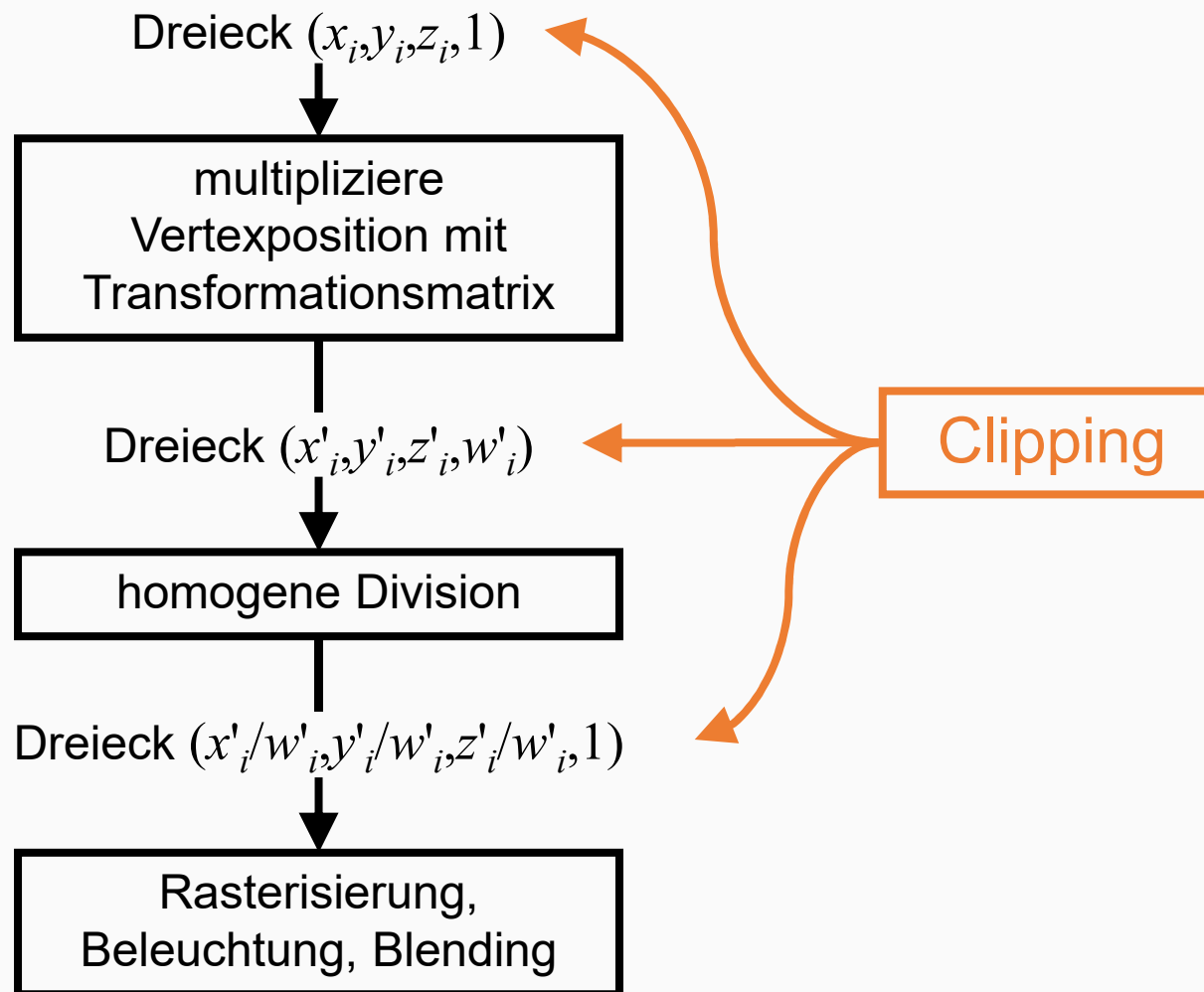
Sutherland-Hodgman-Algorithmus (19)



Ausgabe:

$I_7 I_8 I_6 P_4 I_9 I_{10} I_3 P_7 I_4$

3D-Clipping (1)

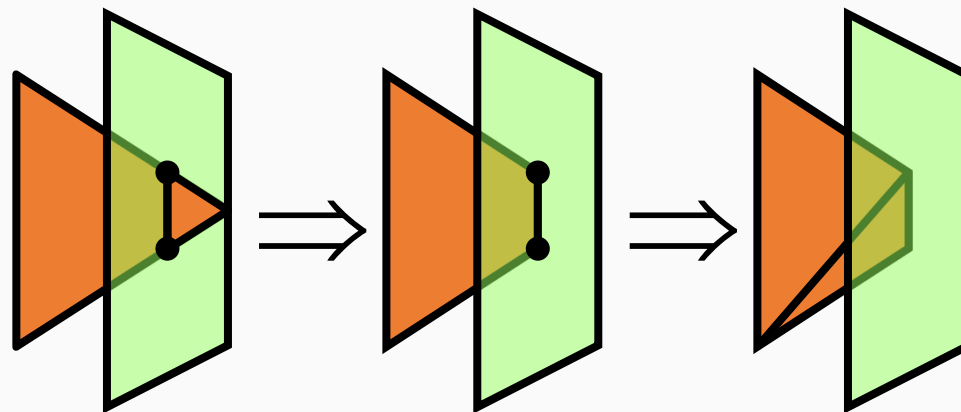


3D-Clipping (2)

- Clipping an drei Stellen in Graphikpipeline möglich:
 - In **Weltkoordinaten** mit den **6 Ebenen des Sichtfrustums**
 - In transformierten Koordinaten **vor der homogenen Division**
 - In Koordinaten des **kanonischen Sichtvolumens**
- Prinzip:
 - Clippe Dreieck nacheinander an jeder Ebene

3D-Clipping (3)

```
for jede der 6 Ebenen  
  if (Dreieck außerhalb der Ebene)  
    break  
  if (Dreieck schneidet Ebene)  
    clippe Dreieck  
    if (Viereck übrig)  
      unterteile in zwei Dreiecke
```



3D-Clipping (4)

- Clipping an der impliziten Ebenengleichung (siehe Kapitel 3)

$$f(\mathbf{p}) = \mathbf{n} \cdot \mathbf{p} + D = 0$$

- Beschreibt eine Gerade in 2D, eine Ebene in 3D und eine Hyperebene in 4D
- Mit parametrischer Liniengleichung:

$$\Leftrightarrow \mathbf{p} = \mathbf{a} + t(\mathbf{b} - \mathbf{a})$$

$$\Rightarrow \mathbf{n} \cdot (\mathbf{a} + t(\mathbf{b} - \mathbf{a})) + D = 0$$

$$\Leftrightarrow t = \frac{\mathbf{n} \cdot \mathbf{a} + D}{\mathbf{n} \cdot (\mathbf{a} - \mathbf{b})}$$

3D-Clipping: Option 1 (vor der Transformation) (1)

- Berechnung der Ebenen am einfachsten aus den 8 Eckpunkten des Sichtvolumens

$$\begin{array}{cccc} (l,b,n)^t & (r,b,n)^t & (l,t,n)^t & (r,t,n)^t \\ (l,b,f)^t & (r,b,f)^t & (l,t,f)^t & (r,t,f)^t \end{array}$$

- Berechnung aus Transformationsmatrix $\mathbf{M}$:

$$\begin{pmatrix} l \\ b \\ f \\ 1 \end{pmatrix} = \mathbf{M}^{-1} \begin{pmatrix} -1 \\ -1 \\ -1 \\ 1 \end{pmatrix} \quad \dots \quad \begin{pmatrix} r \\ t \\ n \\ 1 \end{pmatrix} = \mathbf{M}^{-1} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

3D-Clipping: Option 1 (vor der Transformation) (2)

- Vorteile:
 - Ebenen müssen nur einmal nach Änderung der Transformation berechnet werden
 - Am einfachsten aus den 8 Eckpunkten des Sichtvolumens
- Nachteile:
 - Transformation muss invertierbar sein
 - Ebenengleichungen komplex und damit Schnittpunkttest langsam

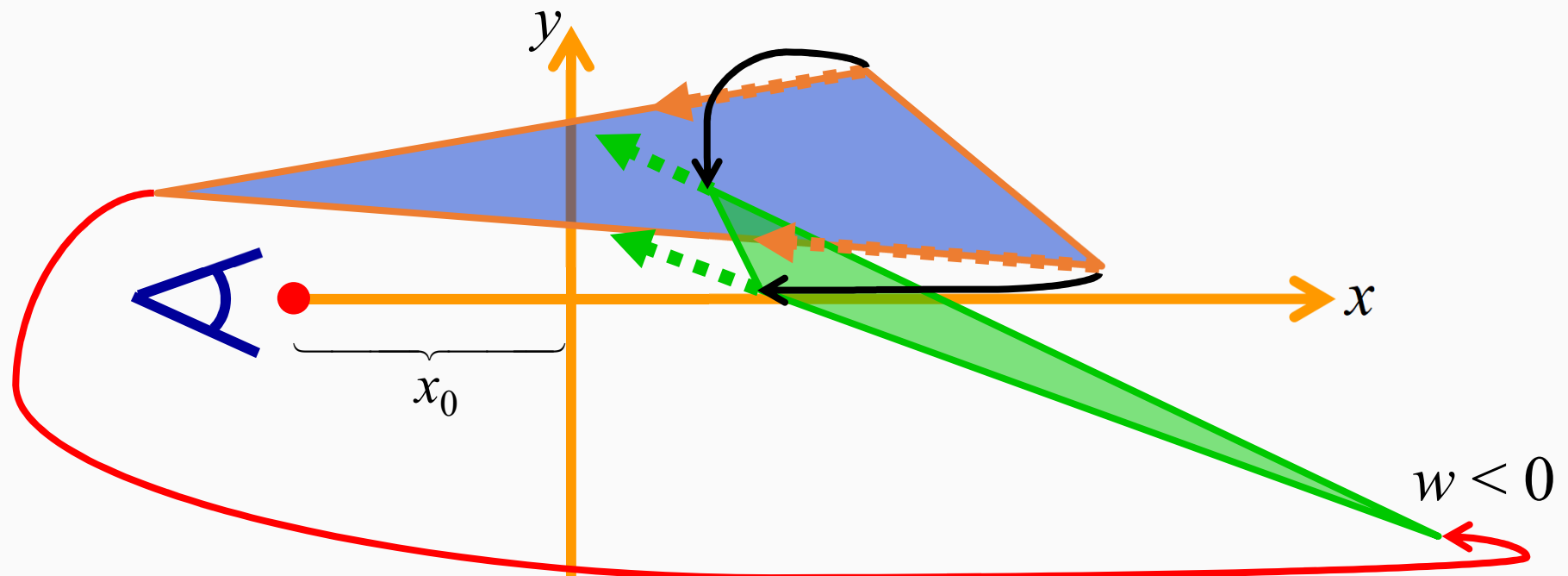
3D-Clipping: Option 3 (nach homogener Division) (1)

- Vorteil:
 - Scheinbar einfachste Methode, denn

$$\left. \begin{array}{l} E_1 : -x - 1 = 0 \\ E_2 : x - 1 = 0 \\ E_3 : -y - 1 = 0 \\ E_4 : y - 1 = 0 \\ E_5 : -z - 1 = 0 \\ E_6 : z - 1 = 0 \end{array} \right\} \text{positiv für alle } P \notin \text{Sichtvolumen}$$

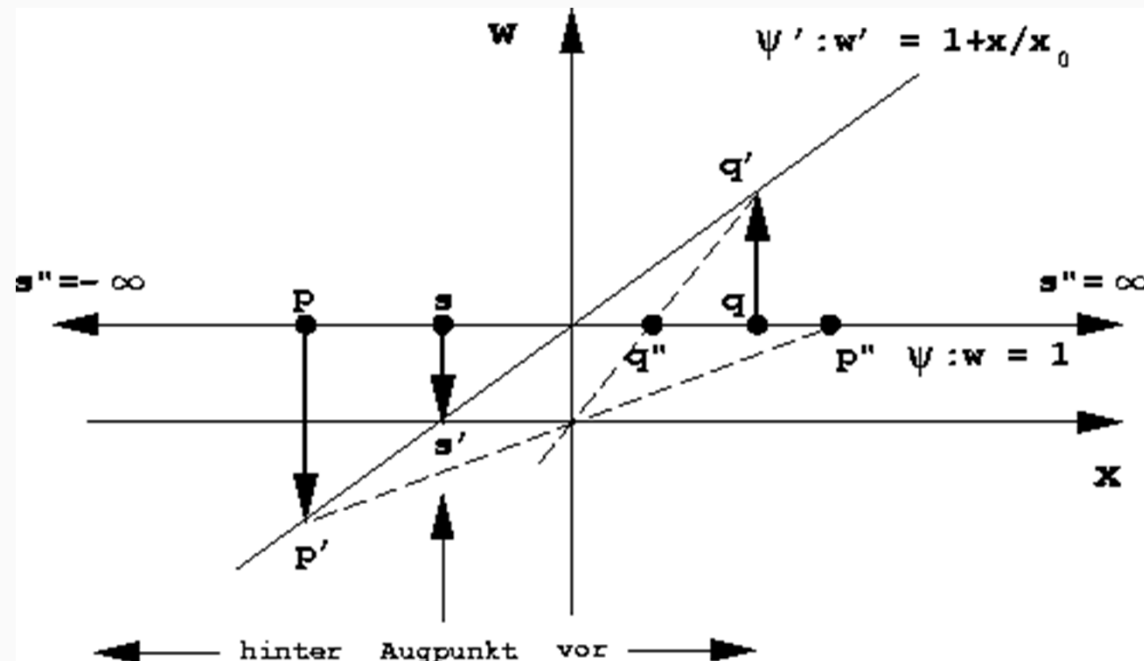
3D-Clipping: Option 3 (nach homogener Division) (2)

- Problem:
 - Wrap-Around, wenn Dreieck teilweise vor und hinter dem Augpunkt



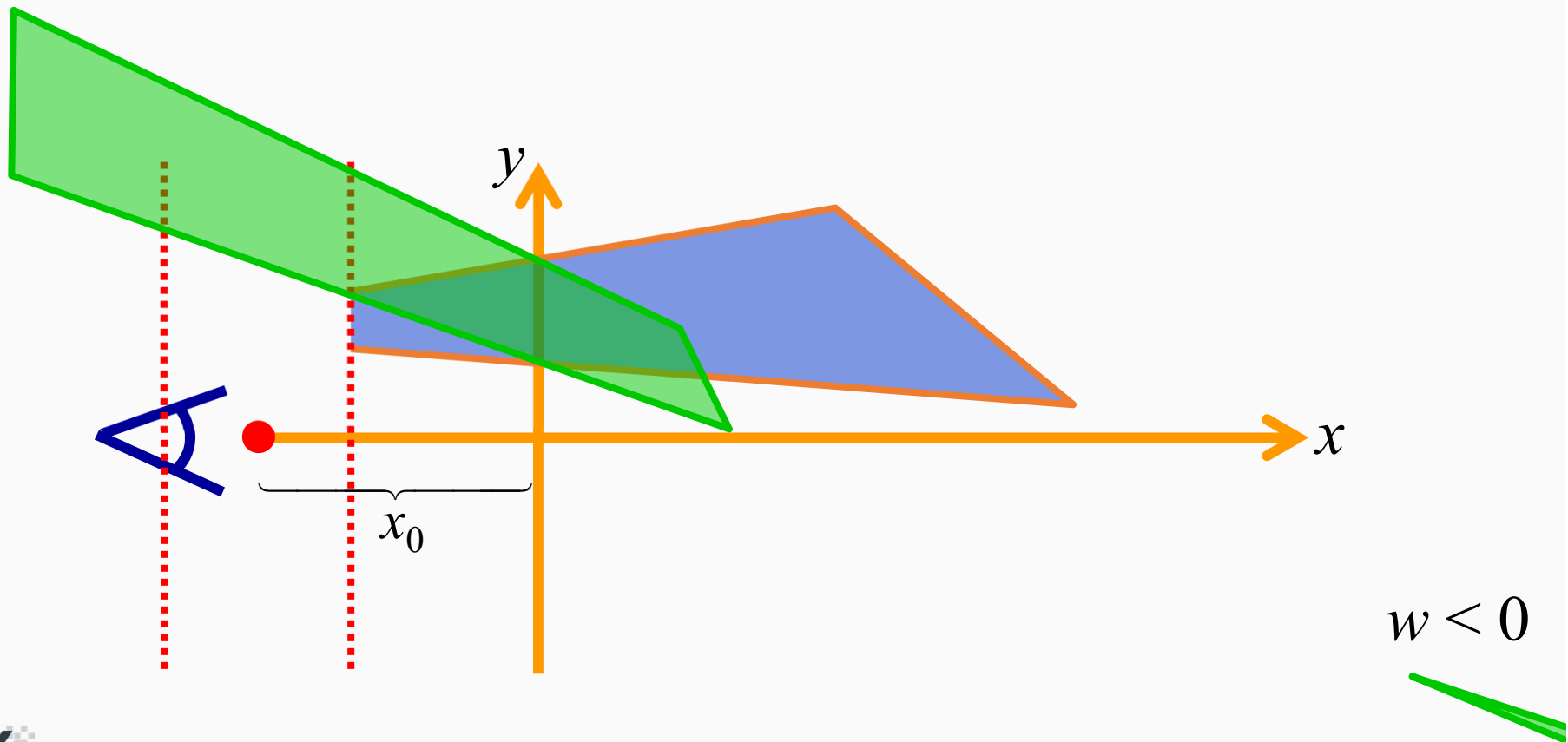
3D-Clipping: Option 3 (nach homogener Division) (3)

- Problem:
 - Wrap-Around, wenn Dreieck teilweise vor und hinter dem Augpunkt
- Grund:
 - Division durch negative w-Koordinate



3D-Clipping: Option 3 (nach homogener Division) (4)

- Lösung:
 - Clipping an den Ebenen $w = \pm \varepsilon$ vor der Division (w-Clip)



3D-Clipping: Option 3 (nach homogener Division) (5)

- Vorteile:
 - Einfache Clipping-Ebenen
 - Unabhängig von Transformation
- Nachteile:
 - Zusätzlicher w-Clip
 - 7, bzw. 8 Clipping-Ebenen
 - Clipping an zwei Stellen in der Pipeline
 - w-Clip vor der homogenen Division
 - Rest nach der Division

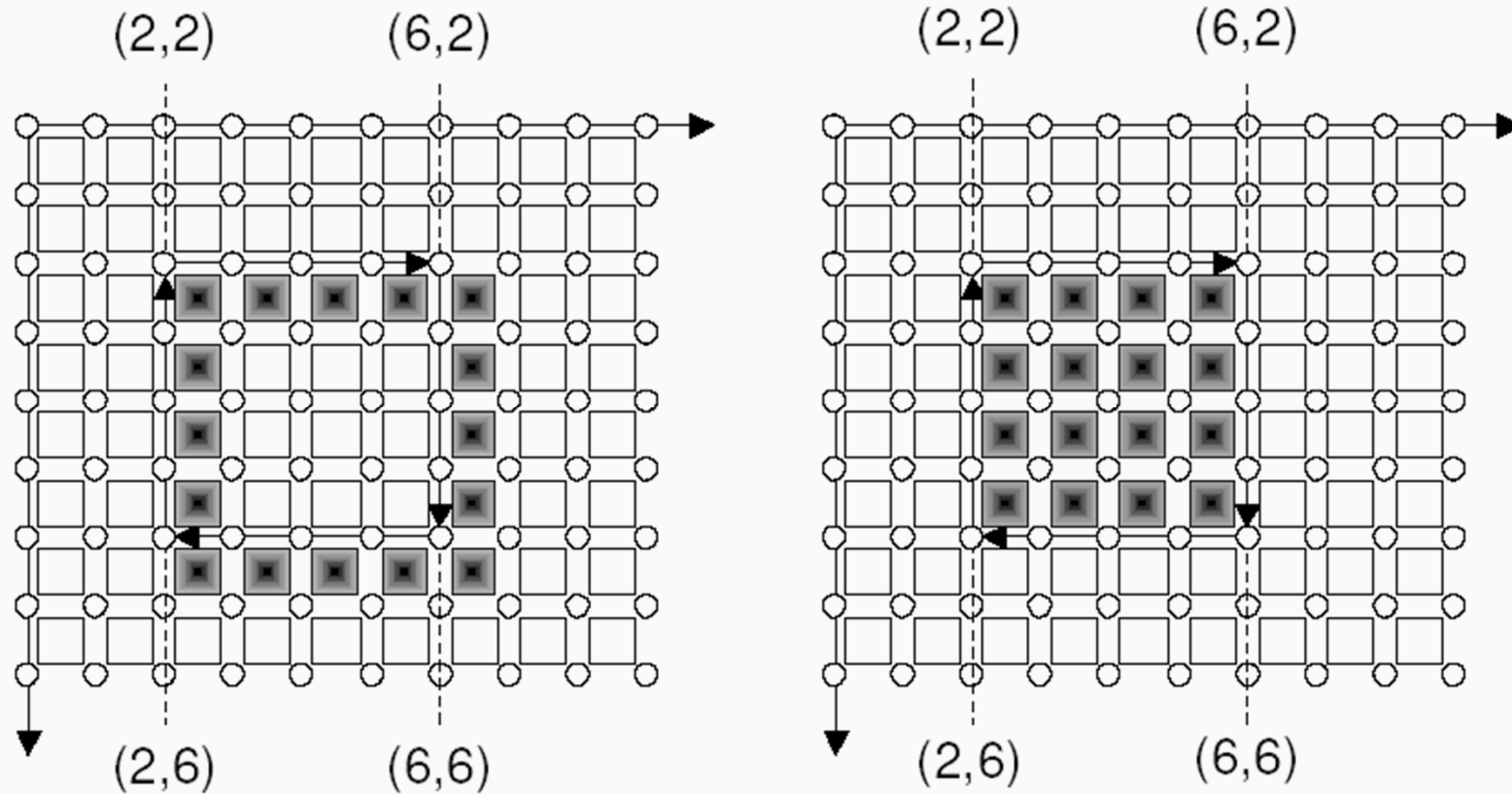
3D-Clipping: Option 2 (vor homogener Division)

- Clipping in 4D an 3D Hyperebenen, mit

$$\left. \begin{array}{l} H_1 : -x - w = 0 \\ H_2 : x - w = 0 \\ H_3 : -y - w = 0 \\ H_4 : y - w = 0 \\ H_5 : -z - w = 0 \\ H_6 : z - w = 0 \end{array} \right\} \text{ positiv für alle } P \notin \text{Sichtvolumen}$$

- Fast so einfach wie Möglichkeit 3, aber kein w-Clip nötig

Unterschied: Umrandung-Füllung



Rasterisierung von Linien (1)

- Herleitung: Betrachte Linien, deren Anfangs- und Endpunkt auf dem Raster liegen:

$$\Delta x = x_1 - x_0, \quad -1 \leq \frac{\Delta x}{\Delta y} \leq 1$$
$$\Delta y = y_1 - y_0,$$

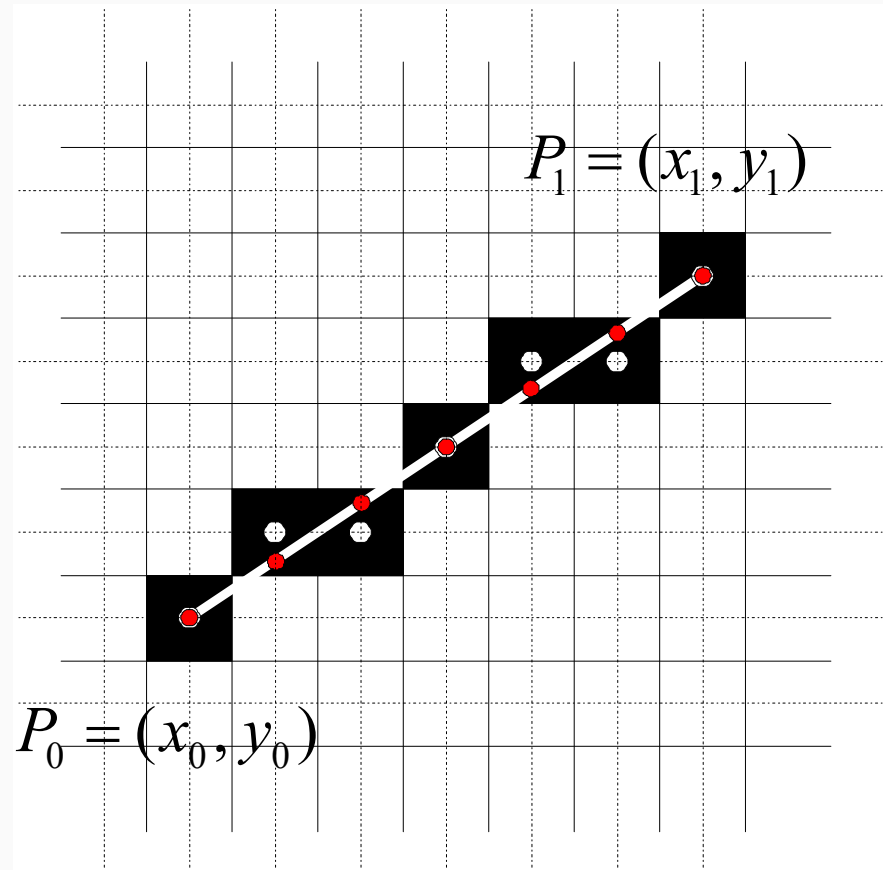
- Berechne:

$$y_i = \frac{\Delta y}{\Delta x} * x_i + b,$$

$$x_i = x_0 + i, \quad i = 1, \dots, \Delta x$$

- Zeichne Pixel an der Stelle

$$(x_i, \text{round}(y_i)) = (x_i, \text{Floor}(\frac{1}{2} + y_i))$$



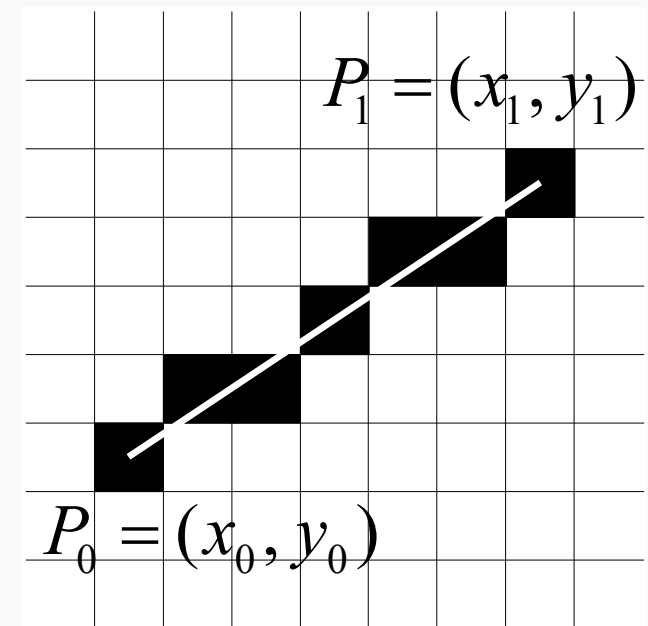
Algorithmus von Bresenham (1)

- Der erste effiziente Algorithmus zum Zeichnen von Linien stammt von Bresenham (1963)
 - Verzichtet auf Berechnung der Steigung
 - Für Pixel (x_i, y_i) wird entschieden, welcher der nächste Pixel ist
- Herleitung: Betrachte Linien mit $0 \leq m \leq 1$

$$\Delta x = x_1 - x_0 \geq 0,$$

$$\Delta y = y_1 - y_0 \geq 0,$$

$$\Delta x \geq \Delta y.$$



Algorithmus von Bresenham (2)

- Fallunterscheidungen zum Erfüllen der Annahmen bei beliebigen Linien:

1. $m = 0$; $m = 1 \Rightarrow$ Trivialfälle

2. $(x_0, y_0) \neq (0, 0) \Rightarrow$ zum Ursprung verschieben

3. $m < 0 \Rightarrow$ an der x -Achse spiegeln

4. $m > 1 \Rightarrow$ an der Geraden $x = y$ spiegeln

Algorithmus von Bresenham (3)

Fall 1: triviale Lösung

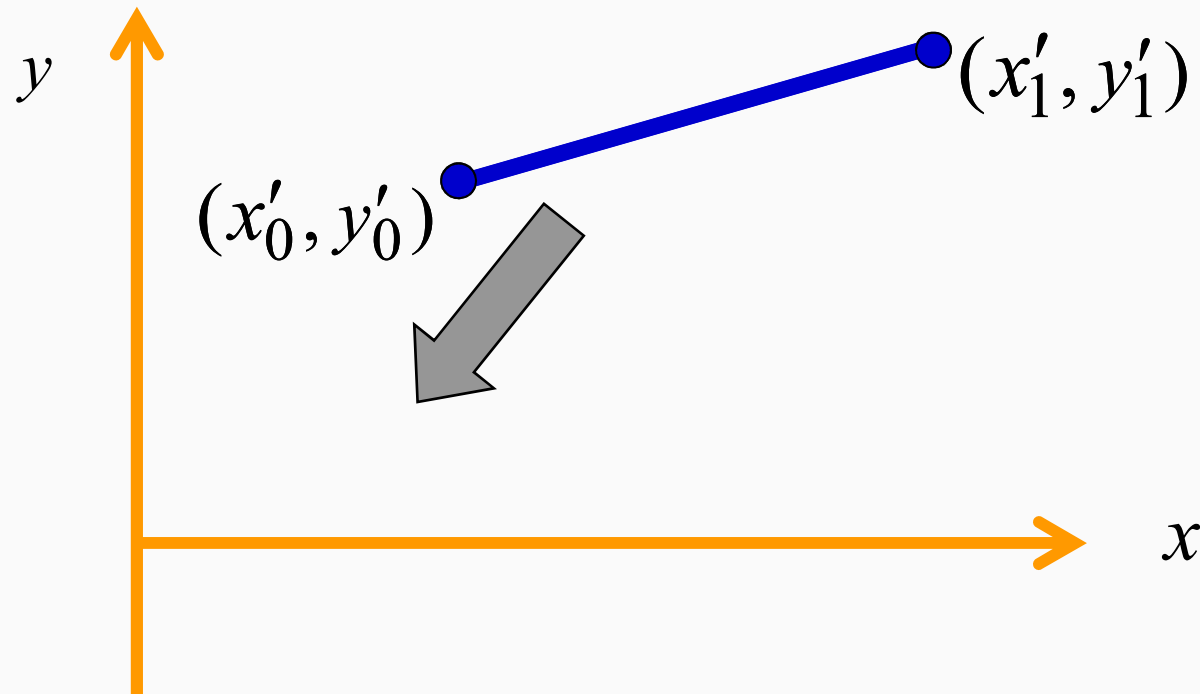
- Bresenham Algorithmus nicht nötig.

a) $m = 0 \Rightarrow$ horizontale Line ($y_i = 0$)

b) $m = 1 \Rightarrow$ diagonale Line ($y_i = x_i$)

Algorithmus von Bresenham (4)

Fall 2: zum Ursprung verschieben



Algorithmus von Bresenham (5)

Fall 2: zum Ursprung verschieben

- Verschiebe (x_0, y_0) zum Ursprung:

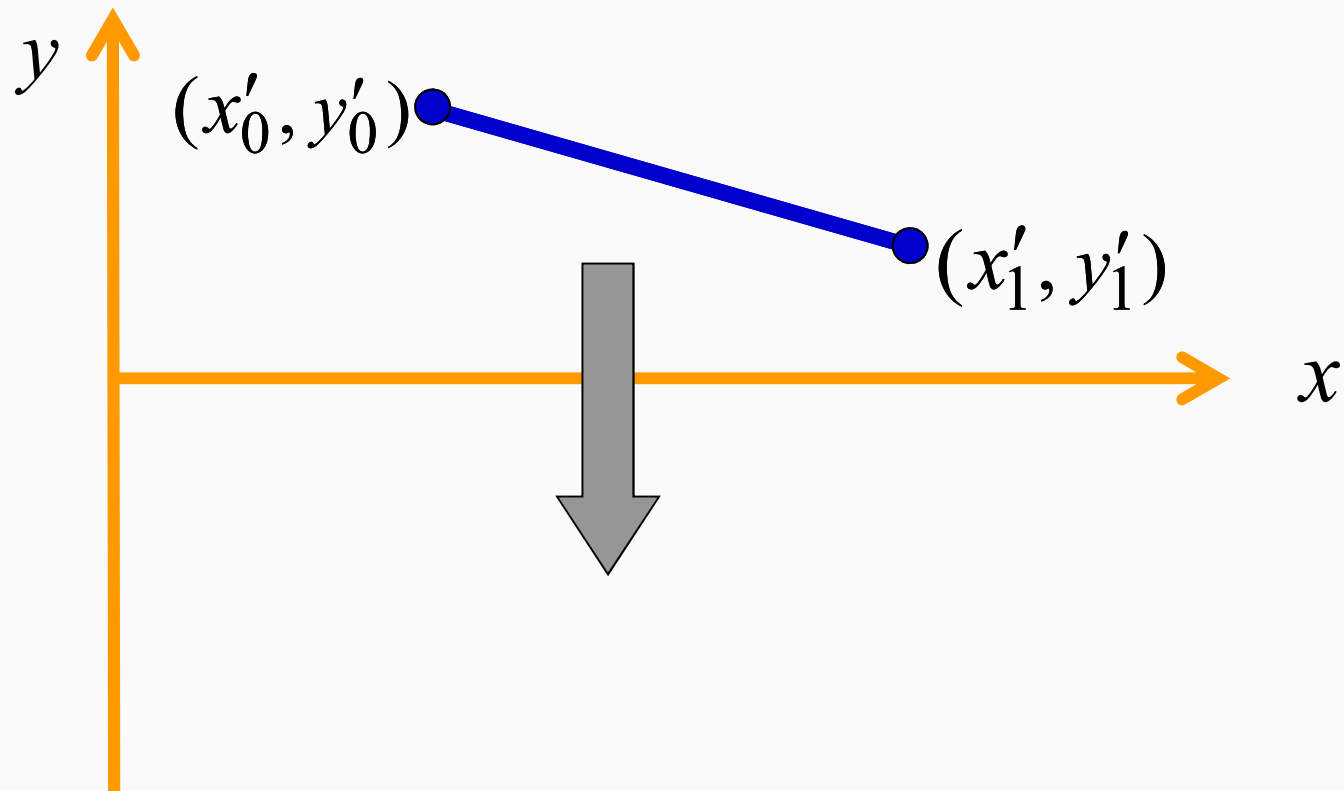
$$(x'_0, y'_0) = (0, 0)$$

$$(x'_1, y'_1) = (x_1 - x_0, y_1 - y_0)$$

- Betrachte im folgenden nur noch Linien, die vom Ursprung ausgehen.

Algorithmus von Bresenham (6)

Fall 3: an der x-Achse spiegeln



Algorithmus von Bresenham (7)

Fall 3: an der x-Achse spiegeln

- Annahme:

$$m < 0$$

- Spiegeln an der x-Achse ($y' = -y$):

$$(x'_0, y'_0) = (x_0, -y_0);$$

$$(x'_1, y'_1) = (x_1, -y_1)$$

Algorithmus von Bresenham (8)

Fall 3: an der x-Achse spiegeln

- Änderung von m :

$$\left. \begin{array}{l} m = \frac{y_1 - y_0}{x_1 - x_0}; \\ m' = \frac{y'_1 - y'_0}{x_1 - x_0} \end{array} \right\} \text{ nach Definition}$$

$$\text{Da } y'_i = -y_i \Rightarrow m' = \frac{-y_1 - (-y_0)}{x_1 - x_0}$$

Algorithmus von Bresenham (9)

Fall 3: an der x-Achse spiegeln

- Änderung von m :

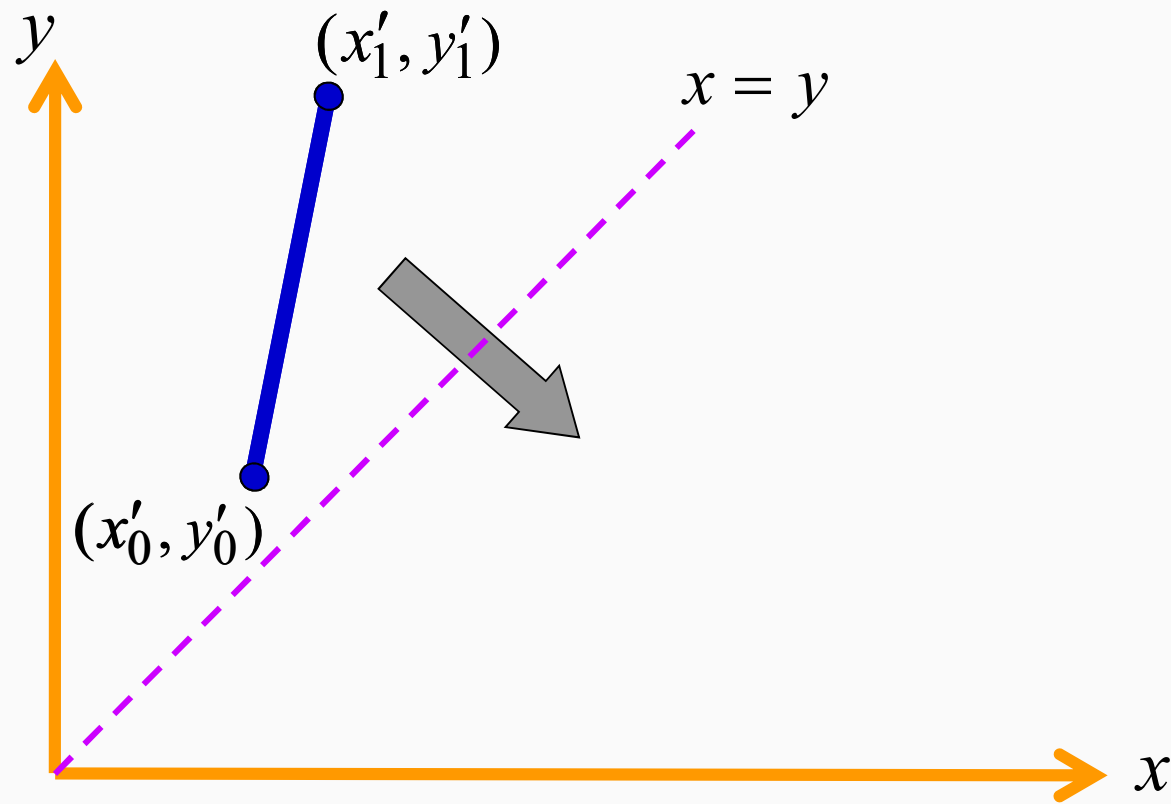
$$m' = -\frac{(y_1 - y_0)}{x_1 - x_0}$$

$$\text{d.h., } m' = -m$$

$$m < 0 \Rightarrow m' > 0$$

Algorithmus von Bresenham (10)

Fall 4: an Gerade $x=y$ spiegeln



Algorithmus von Bresenham (11)

Fall 4: an Gerade $x=y$ spiegeln

$$y = mx + k$$

- vertausche $x \leftrightarrow y$:

$$x' = y, \quad y' = x$$

$$x' = my' + k,$$

$$my' = x' - k$$

Algorithmus von Bresenham (12)

Fall 4: an Gerade $x=y$ spiegeln

- $m' = ?$

$$y' = \left(\frac{1}{m} \right) x' - k',$$

$$m' = \left(\frac{1}{m} \right)$$

$$|m| > 1 \Rightarrow 0 < |m'| < 1$$

Algorithmus von Bresenham (13)

Beschränkte Form:

- Liniensegment im ersten Oktanten mit:
 - $0 < m < 1$
 - $P_0 = (0,0)$

Algorithmus von Bresenham (14)

Liniengleichungen:

Implizit: $F(x, y) = ax + by + c = 0$

Explizit: $y = mx + k$

definiere: $\Delta y = y_1 - y_0$

$$\Delta x = x_1 - x_0$$

somit $y = \left(\frac{\Delta y}{\Delta x} \right) x + k$

Algorithmus von Bresenham (15)

Wir haben, $y = \left(\frac{\Delta y}{\Delta x} \right) x + k$

somit: $\frac{\Delta y}{\Delta x} x - y + k = 0$

oder $(\Delta y)x + (-\Delta x)y + (\Delta x)k = 0$

wobei: $F(x, y) = (\Delta y)x + (-\Delta x)y + (\Delta x)k = 0$

$$a = (\Delta y); \quad b = -(\Delta x); \quad c = k(\Delta x)$$

Algorithmus von Bresenham (16)

Vorzeichen von F :

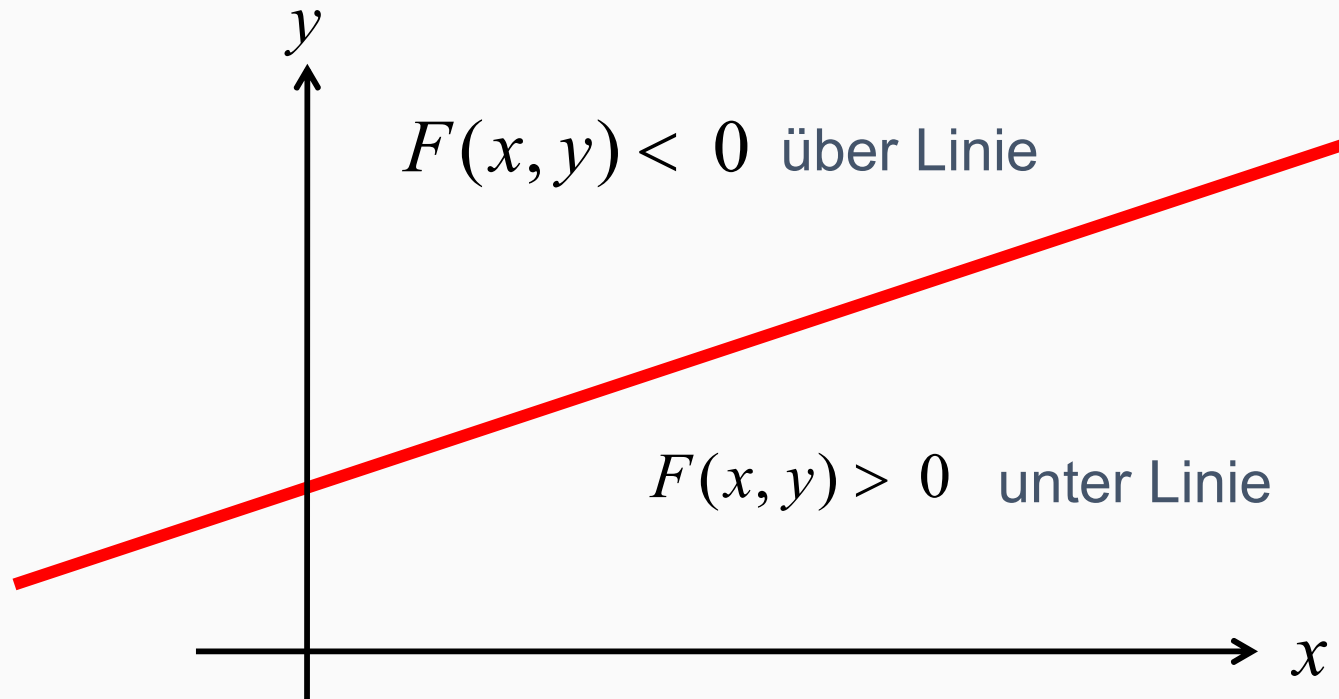
- Es gilt:

$$F(x, y) = \begin{cases} + & : \text{unter Linie} \\ 0 & : \text{auf Linie} \\ - & : \text{über Linie} \end{cases}$$

- Betrachte Extremwerte von y

Algorithmus von Bresenham (17)

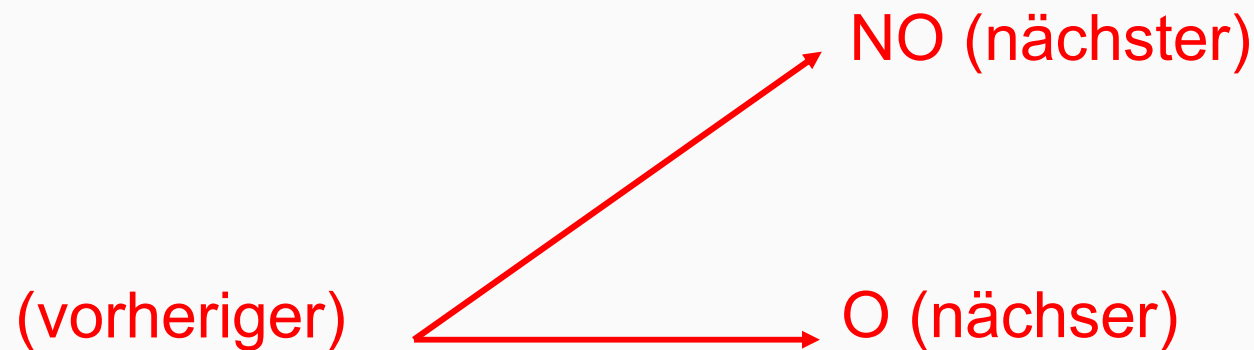
Veranschaulichung



Algorithmus von Bresenham (18)

Schlüssel zum Bresenham Algorithmus:

- „Vernünftige Annahmen“ haben das Problem auf die wiederholte Entscheidung zwischen zwei Möglichkeiten reduziert.



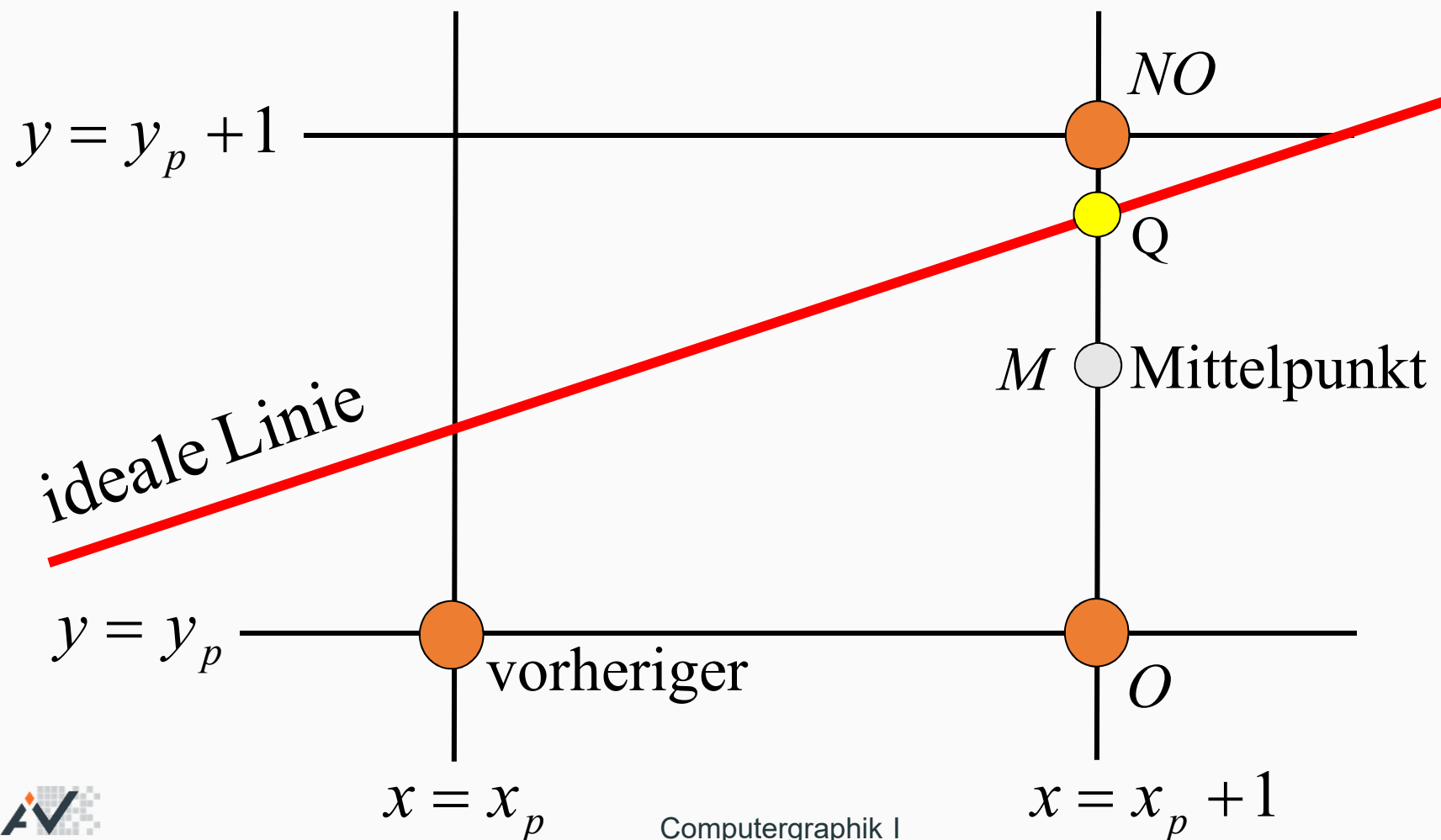
Algorithmus von Bresenham (19)

Entscheidungsvariable d (logisch)

- Definiere eine logische Entscheidungsvariable d mit folgenden Eigenschaften
 - linear in ihrem Verhalten
 - inkrementell aktualisiert (mit Addition)
 - sagt uns, ob wir nach O oder NO gehen

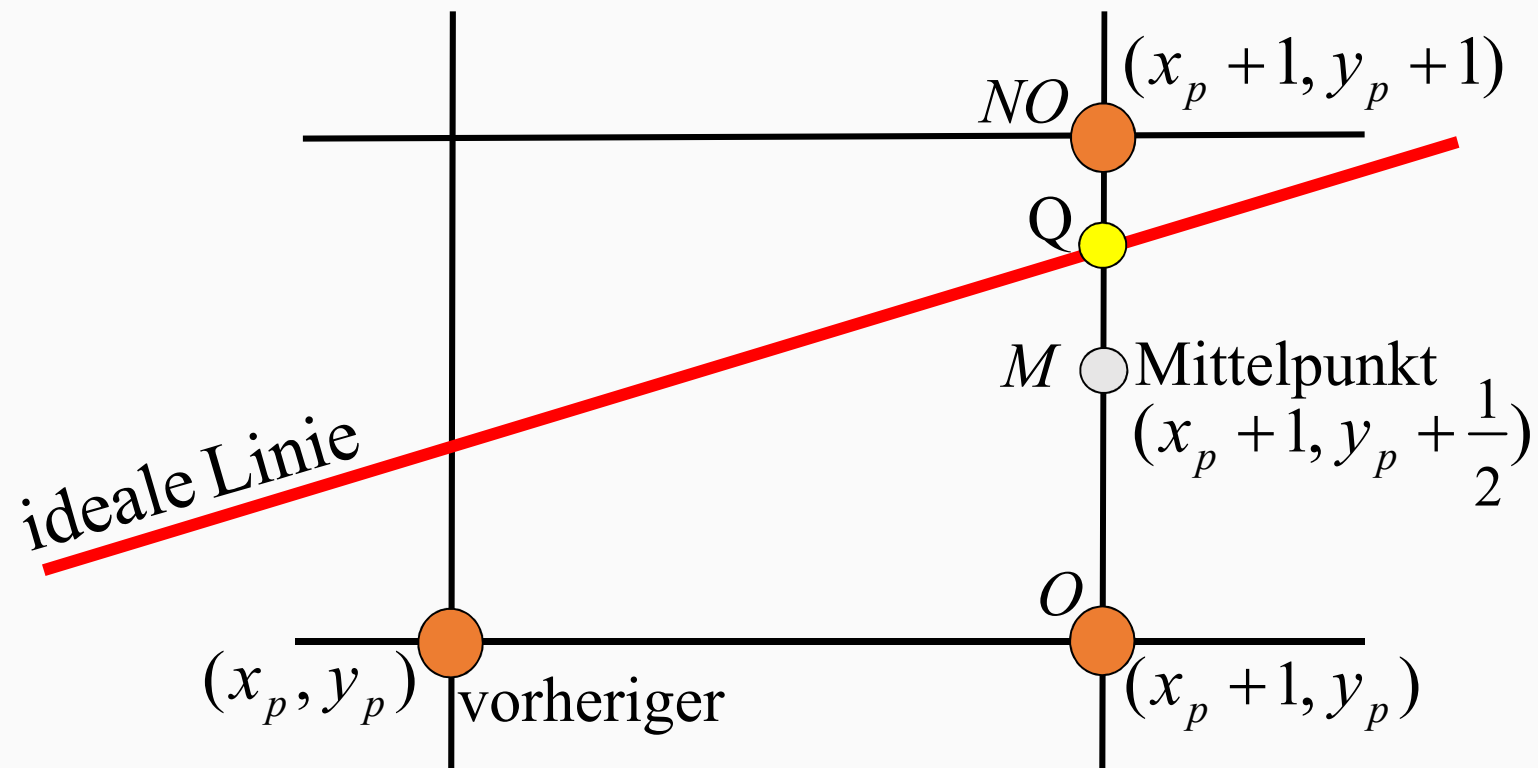
Algorithmus von Bresenham (20)

Beispiel:



Algorithmus von Bresenham (21)

Beispiel:



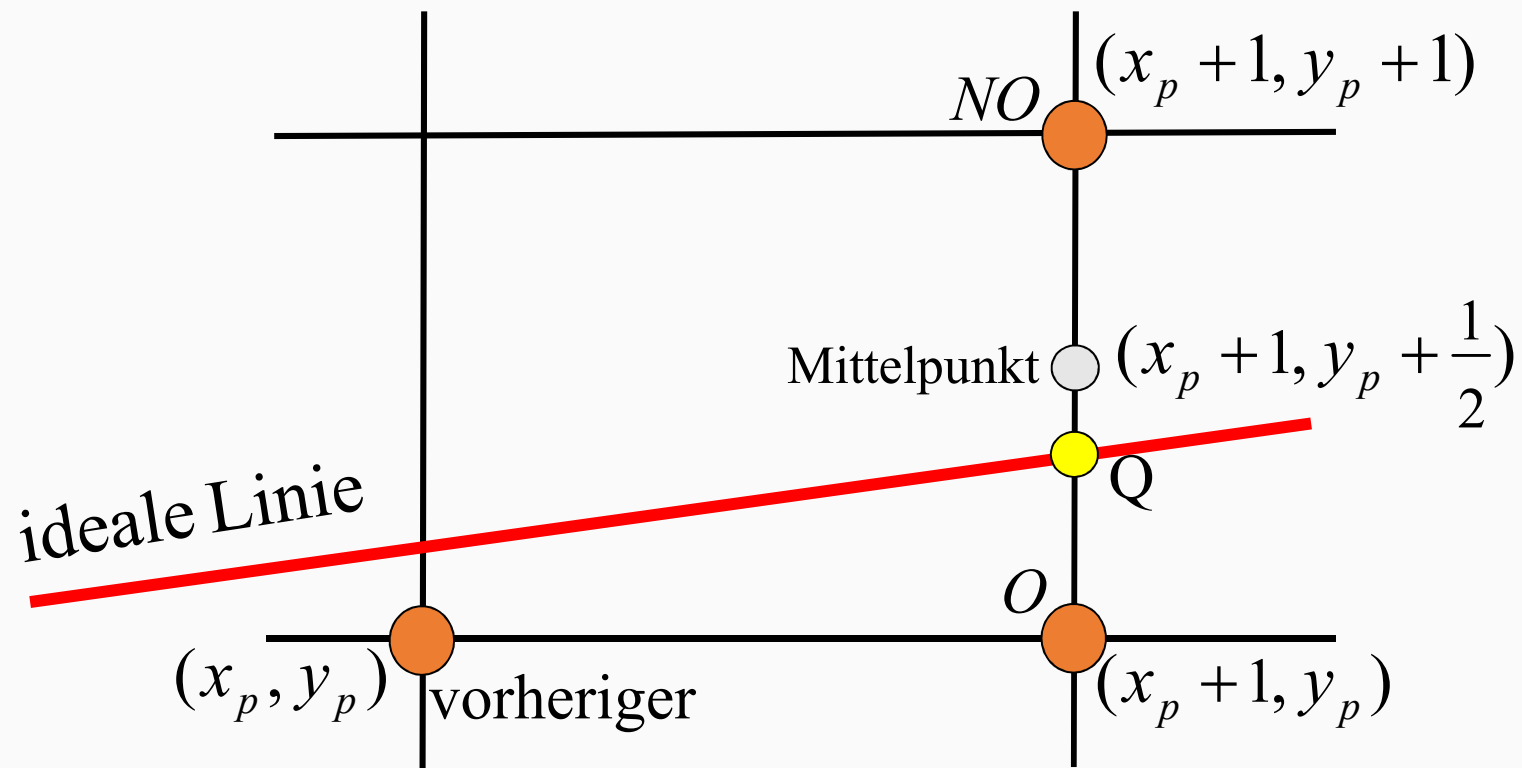
Algorithmus von Bresenham (22)

Beachte folgende Beziehungen:

- Angenommen Q ist oberhalb von M , wie im Beispiel.
- Dann ist $F(M) > 0$ und M unterhalb der Linie
- Also bedeutet $F(M) > 0$, daß die Linie oberhalb von M verläuft.
- Daher müssen wir nach NO , also den y -Wert erhöhen.

Algorithmus von Bresenham (23)

Beispiel 2:



Algorithmus von Bresenham (24)

Beachte folgende Beziehungen:

- Angenommen Q ist unterhalb von M , wie im zweiten Beispiel.
- Dann ist $F(M) < 0$ und M oberhalb der Linie
- Also bedeutet $F(M) < 0$, daß die Linie unterhalb von M verläuft.
- Daher müssen wir nach O , also den y -Wert nicht erhöhen.

Algorithmus von Bresenham (25)

$$M = \text{Mittelpunkt} = (x_p + 1, y_p + \frac{1}{2})$$

- Wir wollen F an M auswerten.
- Mit inkrementeller Entscheidungsvariable d

$$\begin{aligned} \text{Sei } d &= F(x_p + 1, y_p + \frac{1}{2}) \\ d &= a(x_p + 1) + b(y_p + \frac{1}{2}) + c \end{aligned}$$

Algorithmus von Bresenham (26)

Wie wird d benutzt?

daher:
$$d = a(x_p + 1) + b(y_p + \frac{1}{2}) + c$$

$$d = \begin{cases} > 0 \Rightarrow NO & \text{(Mittelpunkt unter idealer Linie)} \\ < 0 \Rightarrow O & \text{(Mittelpunkt über idealer Linie)} \\ = 0 \Rightarrow O & \text{(beliebig)} \end{cases}$$

Algorithmus von Bresenham (27)

Fall 1: O wurde gewählt

$$d_{alt} = a(x_p + 1) + b(y_p + \frac{1}{2}) + c$$

$$O \Rightarrow: \quad x \leftarrow x + 1; \quad y \leftarrow y,$$

$$\begin{aligned} d_{neu} &= F(x_p + 2, y_p + \frac{1}{2}) \\ &= a(x_p + 2) + b(y_p + \frac{1}{2}) + c \end{aligned}$$

Algorithmus von Bresenham (28)

Fall 1: O wurde gewählt

$$d_{neu} - d_{alt} = \left(a(x_p + 2) + b(y_p + \frac{1}{2}) + c \right) - \left(a(x_p + 1) + b(y_p + \frac{1}{2}) + c \right)$$

$$d_{neu} = d_{alt} + a$$

Algorithmus von Bresenham (29)

Whd.: Wir haben, $y = \left(\frac{\Delta y}{\Delta x} \right) x + k$

somit: $\frac{\Delta y}{\Delta x} x - y + k = 0$

oder $(\Delta y)x + (-\Delta x)y + (\Delta x)k = 0$

wobei: $F(x, y) = (\Delta y)x + (-\Delta x)y + (\Delta x)k = 0$

$$a = (\Delta y); b = -(\Delta x); c = k(\Delta x)$$

Algorithmus von Bresenham (30)

Fall 1: $d_{neu} = d_{alt} + a$

$\Delta_O \equiv$ Inkrement wenn O gewählt wurde

Somit: $\Delta_O = a$

Beachte: $a = \Delta y$ (aus Liniengleichungen)

Daher wird $F(M)$ nicht explizit ausgewertet.

Wir addieren einfach $\Delta_O = a$ um d für O zu aktualisieren.

Algorithmus von Bresenham (31)

Fall 2: NO wurde gewählt

$$d_{alt} = a(x_p + 1) + b(y_p + \frac{1}{2}) + c$$

$$NO \Rightarrow: \quad x \leftarrow x + 1; \quad y \leftarrow y + 1,$$

$$\begin{aligned} d_{neu} &= F(x_p + 2, y_p + \frac{3}{2}) \\ &= a(x_p + 2) + b(y_p + \frac{3}{2}) + c \end{aligned}$$

Algorithmus von Bresenham (32)

Fall 2: NO wurde gewählt

$$d_{neu} - d_{alt} = \left(a(x_p + 2) + b(y_p + \frac{3}{2}) + c \right) - \left(a(x_p + 1) + b(y_p + \frac{3}{2}) + c \right)$$

$$d_{neu} = d_{alt} + a + b$$

Algorithmus von Bresenham (33)

Fall 2:

$$d_{neu} = d_{alt} + a + b$$

$\Delta_{NO} \equiv$ Inkrement wenn NO gewählt wurde

Somit: $\Delta_{NO} = a + b$

Beachte: $a = \Delta y$, $b = -\Delta x$ (aus Liniengleichungen)

Daher wird $F(M)$ nicht explizit ausgewertet.

Wir addieren einfach $\Delta_{NO} = a + b$ um d für NO zu aktualisieren.

Algorithmus von Bresenham (34)

Zusammenfassung

- Bei jedem Schritt des Algorithmus müssen wir, abhängig vom Vorzeichen der Entscheidungsvariable d , zwischen O und NO wählen.
- Dann aktualisieren wir d entsprechend:

$$d \leftarrow \begin{cases} d + \Delta_O, & \text{wobei } \Delta_O = \Delta y, \text{ bzw.} \\ d + \Delta_{NO}, & \text{wobei } \Delta_{NO} = \Delta y - \Delta x \end{cases}$$

Algorithmus von Bresenham (35)

Was ist der Startwert von d ?

- erster Punkt: (x_0, y_0)
- erster Mittelpunkt: $(x_0 + 1, y_0 + \frac{1}{2})$
- Was ist der Mittelpunktswert am Anfang?

$$d\left(x_0 + 1, y_0 + \frac{1}{2}\right) = F\left(x_0 + 1, y_0 + \frac{1}{2}\right)$$

Algorithmus von Bresenham (36)

Was ist der Startwert von d ?

$$\begin{aligned} F(x_0 + 1, y_0 + \frac{1}{2}) &= a(x_0 + 1) + b(y_0 + \frac{1}{2}) + c \\ &= \underbrace{(ax_0 + by_0 + c)}_{F(x_0, y_0)} + \left(a + \frac{b}{2} \right) \\ &= F(x_0, y_0) + \left(a + \frac{b}{2} \right) \end{aligned}$$

Algorithmus von Bresenham (37)

Was ist der Startwert von d ?

Beachte: $F(x_0, y_0) = 0$, da (x_0, y_0) auf der Linie

$$\begin{aligned}\text{Daher: } F(x_0 + 1, y_0 + \frac{1}{2}) &= 0 + a + \frac{b}{2} \\ &= \Delta y - \frac{\Delta x}{2}\end{aligned}$$

Algorithmus von Bresenham (38)

Was ist der Startwert von d ?

- Was ändert „ $2\times$ “?

- Gleichung hat dieselbe Nullmenge

$$2F(x, y) = 2(ax + by + c) = 0$$

- Ändert die Steigung der Ebene,
 - bzw. rotiert sie um ihre Nullgerade

Algorithmus von Bresenham (39)

Was ist der Startwert von d ?

Multiplizieren von $F(x_0 + 1, y_0 + \frac{1}{2}) = \Delta y - \frac{\Delta x}{2}$

mit 2 liefert:

$$2F(x_0 + 1, y_0 + \frac{1}{2}) = 2\Delta y - \Delta x$$

$$2F(x, y) = 2(ax + by + c) = 0$$

Also ist der Startwert:

$$d = 2\Delta y - \Delta x$$

Algorithmus von Bresenham (40)

Zusammenfassung

- Startwert: $2\Delta y - \Delta x$
- Fall 1 (O): $d \leftarrow d + \Delta_O$, wobei $\Delta_O = 2\Delta y$
- Fall 2 (NO): $d \leftarrow d + \Delta_{NO}$,
wobei $\Delta_{NO} = 2(\Delta y - \Delta x)$
- wähle: $\begin{cases} O & \text{wenn } d \leq 0 \\ NO & \text{sonst} \end{cases}$

Algorithmus von Bresenham (41)

Beispiel

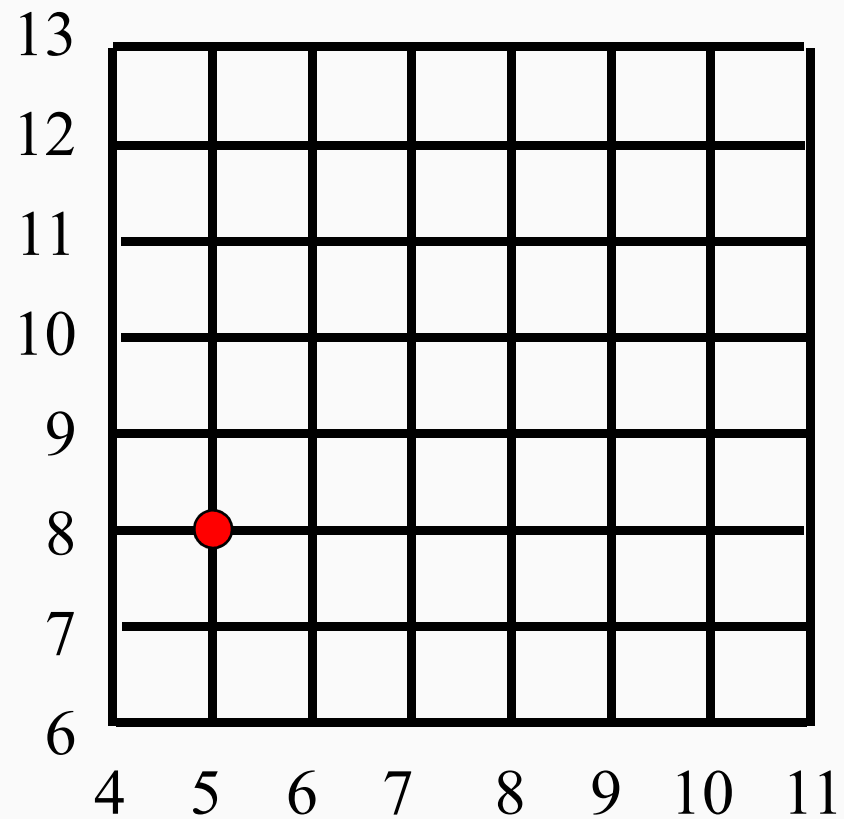
- Endpunkte der Linie:

$$(x_0, y_0) = (5, 8); \quad (x_1, y_1) = (9, 11)$$

$$\Delta x = 4 \quad ; \Delta y = 3$$

Algorithmus von Bresenham (42)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)



Algorithmus von Bresenham (43)

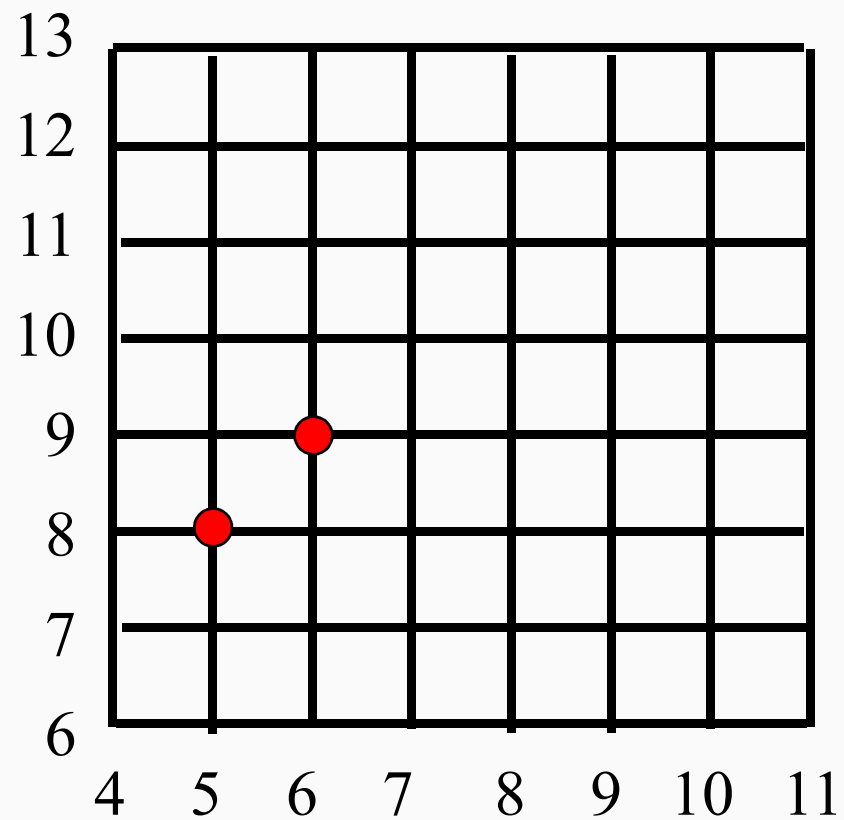
Beispiel ($\Delta x = 4$; $\Delta y = 3$)

- Startwert: $d = 2 \Delta y - \Delta x$
 $= 6 - 4 = 2 > 0$

$$d = 2 \Rightarrow NO$$

Algorithmus von Bresenham (44)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)



Algorithmus von Bresenham (45)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)

- Aktualisierung:

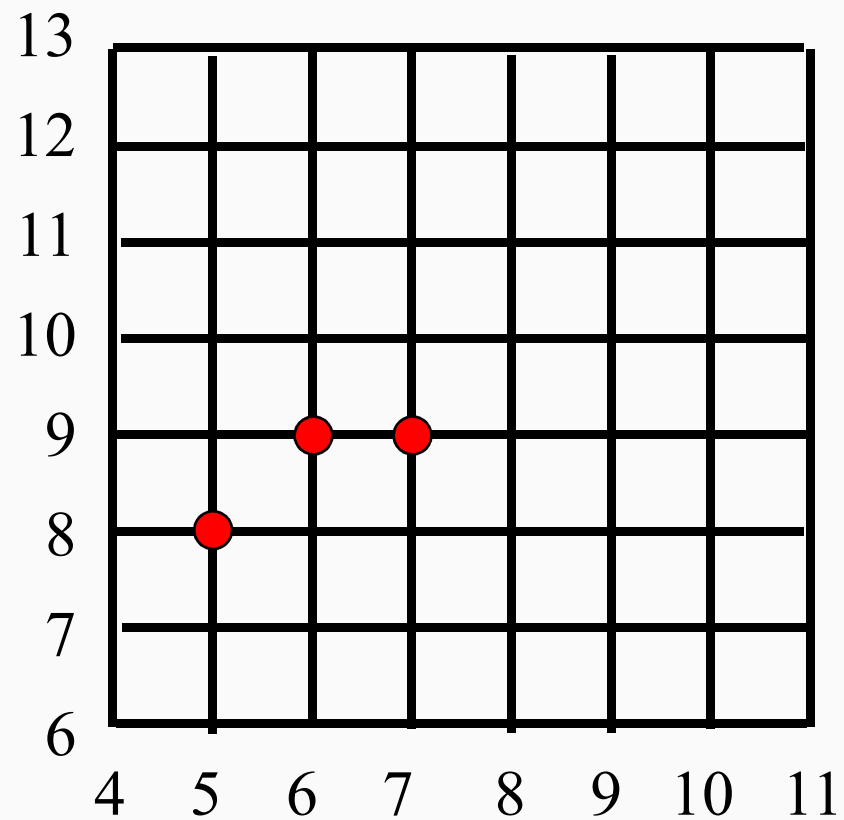
Letzter Schritt war *NO*, also

$$d \leftarrow d + \Delta_{NO} = d + 2(\Delta y - \Delta x)$$

$$d = 2 + 2(3 - 4) = 0 \Rightarrow O$$

Algorithmus von Bresenham (46)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)



Algorithmus von Bresenham (47)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)

- Aktualisierung:

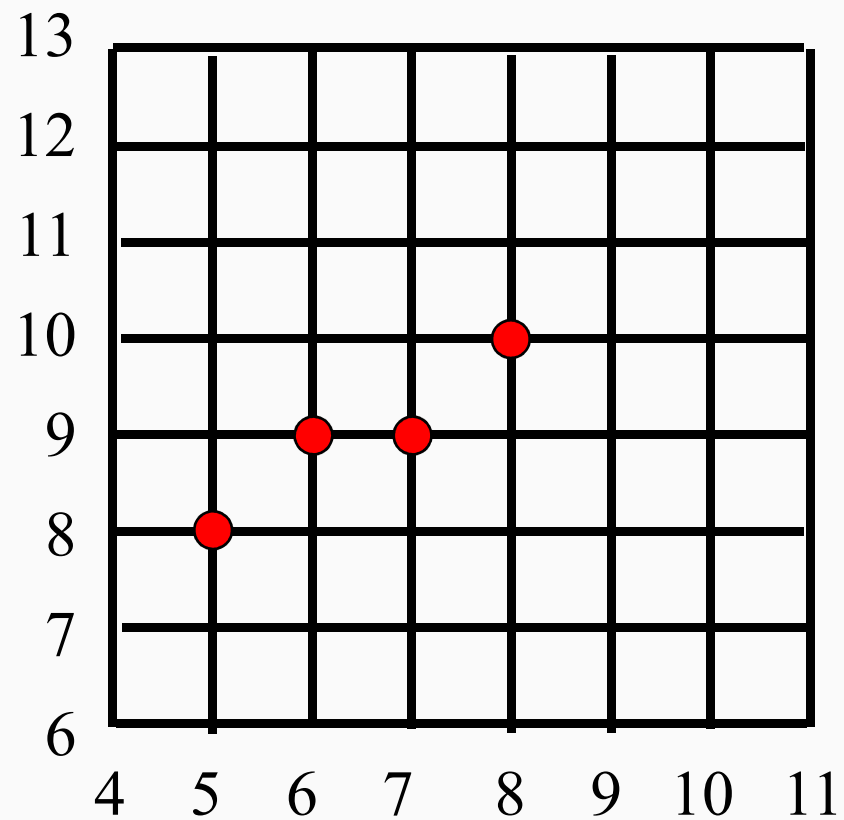
Letzter Schritt war O , also

$$d \leftarrow d + \Delta_o = d + 2\Delta y$$

$$d = 0 + 2 \cdot 3 = 6 \Rightarrow NO$$

Algorithmus von Bresenham (48)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)



Algorithmus von Bresenham (49)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)

- Aktualisierung:

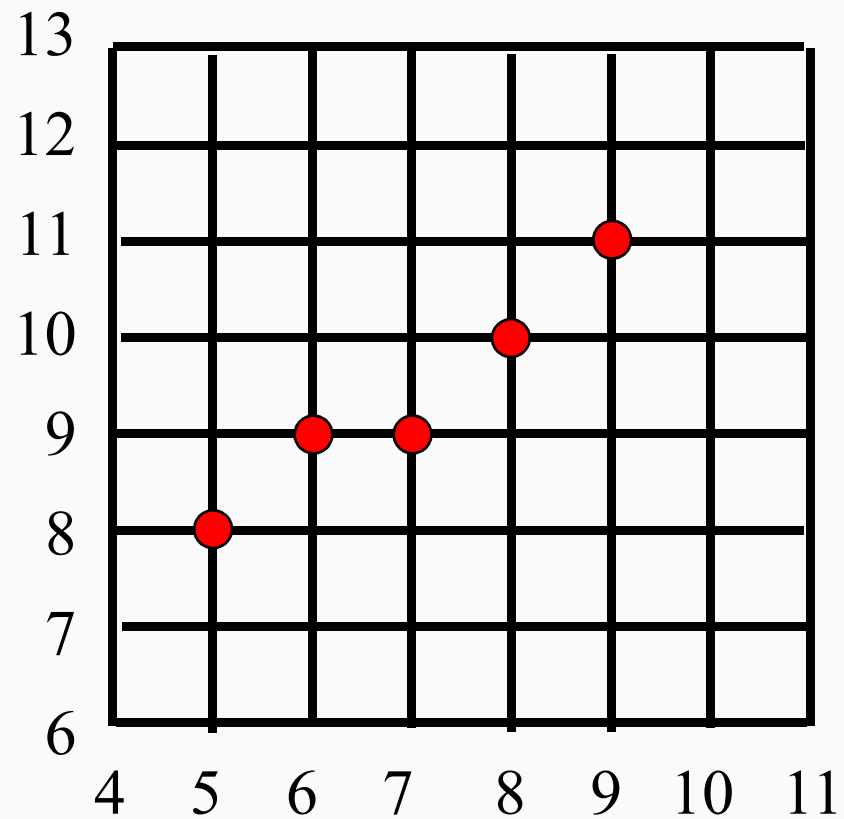
Letzter Schritt war *NO*, also

$$d \leftarrow d + \Delta_{NO} = d + 2(\Delta y - \Delta x)$$

$$d = 6 + 2(3 - 4) = 4 \Rightarrow NO$$

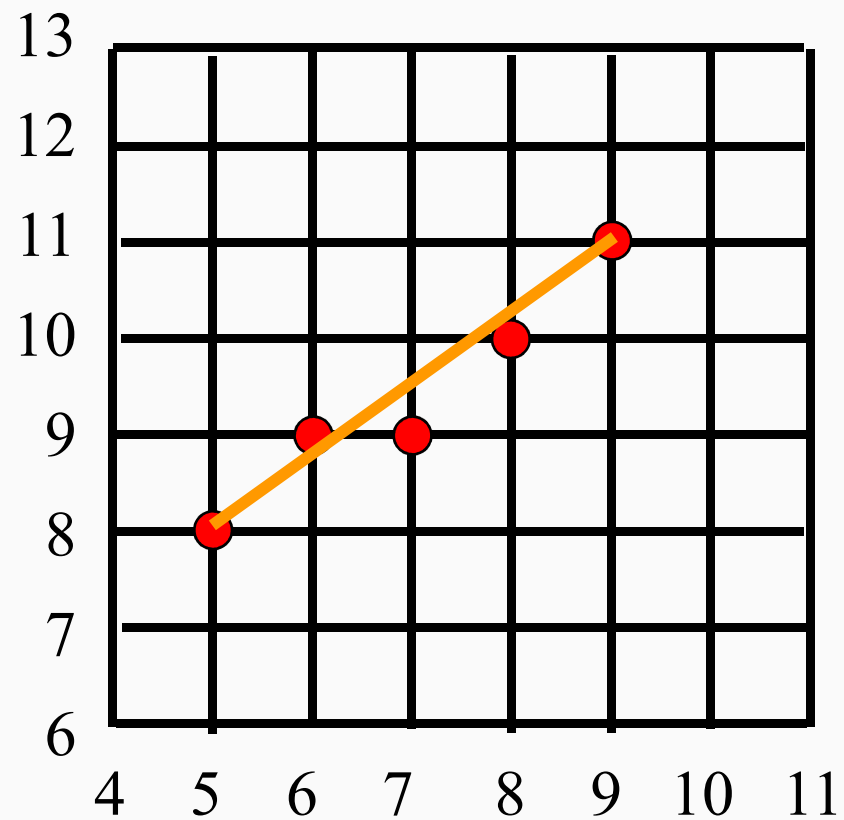
Algorithmus von Bresenham (50)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)



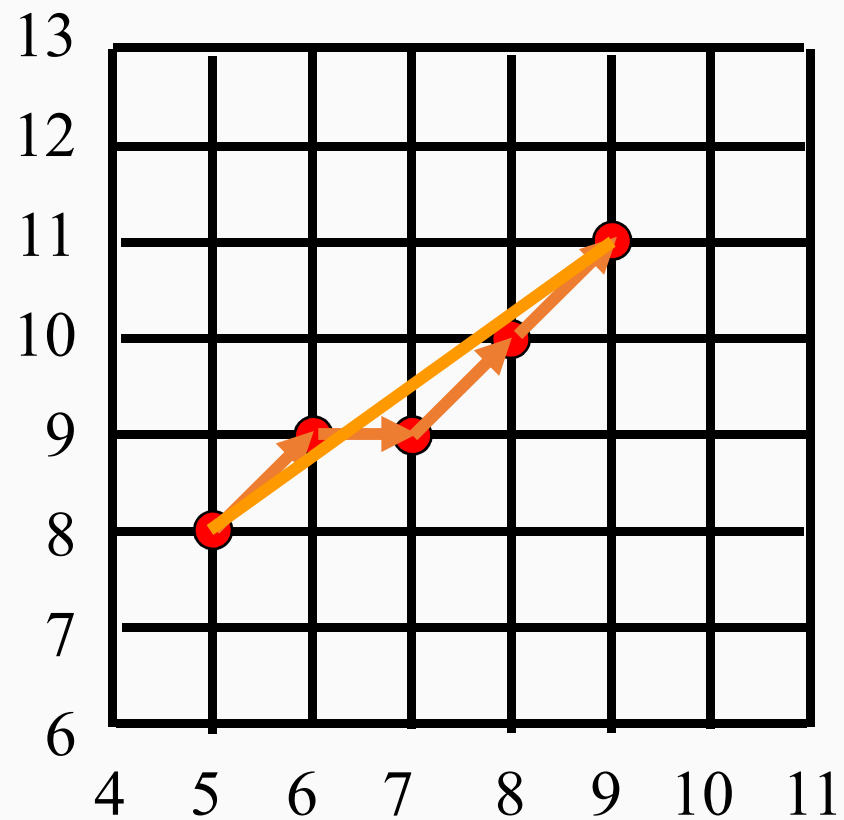
Algorithmus von Bresenham (51)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)



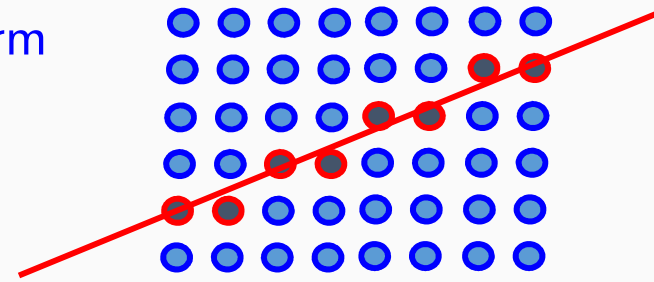
Algorithmus von Bresenham (52)

Beispiel ($\Delta x = 4$; $\Delta y = 3$)

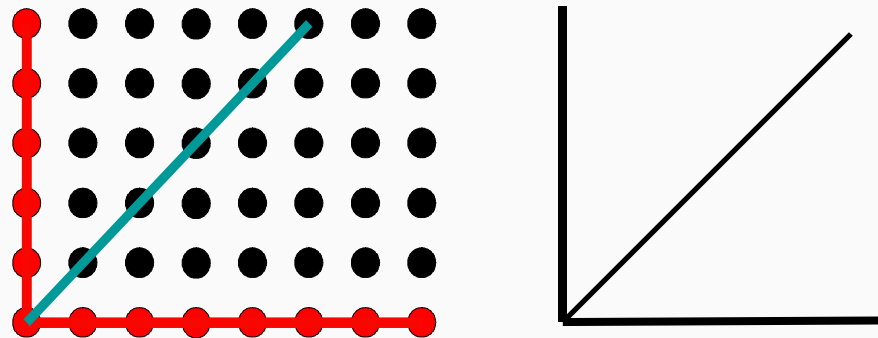


Aliasing

- Beim Zeichnen werden **Näherungspunkte (Aliaspunkte)** ausgewählt
 - Dadurch entsteht eine **Treppenform**
- 

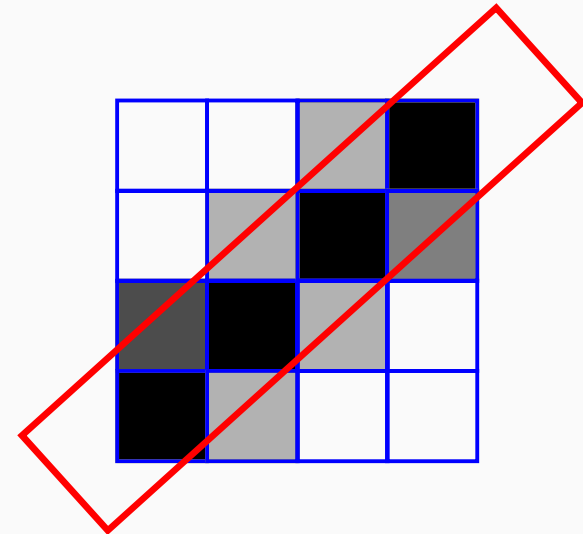


- Je nach Winkel ändert sich die Dicke (Minimum bei 45°)



Ungewichtetes Anti-Aliasing (unweighted area sampling)

- Die Linie wird als Rechteck aufgefasst
- Jedes Pixel entspricht einem Quadrat
 - Die Intensität des Pixels ist proportional zur vom Rechteck überdeckten Fläche des Pixels
 - Die Intensität des Pixels kann nur beeinflusst werden, wenn das Pixel den Rand schneidet
 - Gleiche Anteile erhalten gleiche Intensität, unabhängig vom Abstand zwischen Pixelmitte und Linie



Gewichtetes Anti-Aliasing (weighted area sampling)

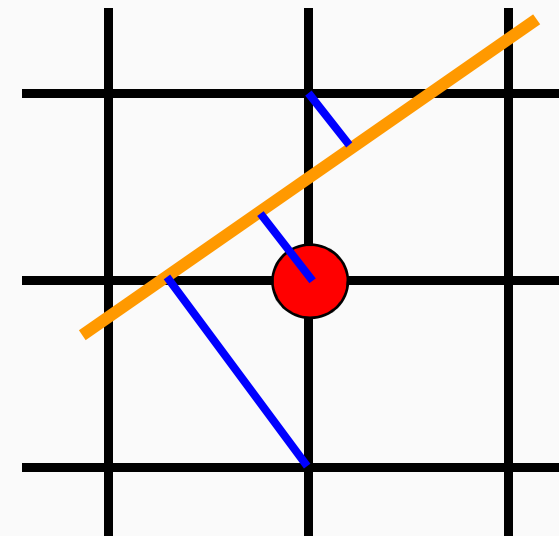
- Gewichtungsfunktion gibt kleineren Bereichen näher an der Pixelmitte eine größere Intensität als in größerer Entfernung.
 - Gewichtungsfunktion (Filterfunktion) ist beim ungewichteten Anti-Aliasing auf 1 normiert und entspricht einer Box.
 - Beim gewichteten Anti-Aliasing entspricht die Filterfunktion einer symmetrischen parabelförmigen Rotationsfunktion.

Anti-Aliasing mit Area Sampling

- Problem:
 - Berechnung der Überdeckung ist aufwendig
 - Um so mehr, wenn eine Filterfunktion benutzt wird
- Gesucht:
 - Einfaches Anti-Aliasing Verfahren, das sich in Bresenham Algorithmus integrieren läßt

Gupta-Sproull Anti-Aliasing (1)

- Idee:
 - Verwende Abstand D zwischen Linie und Pixelmitte zur Abschätzung der Überdeckung
- Vorgehensweise:
 - Berechne D
 - Gehe einen Pixel nach N und berechne D_u
 - Gehe einen Pixel nach S und berechne D_l
 - Je größer D_* , desto weniger Überdeckung



Gupta-Sproull Anti-Aliasing (2)

Inkrementelle Berechnung von D_* :

- Wenn NO gewählt:

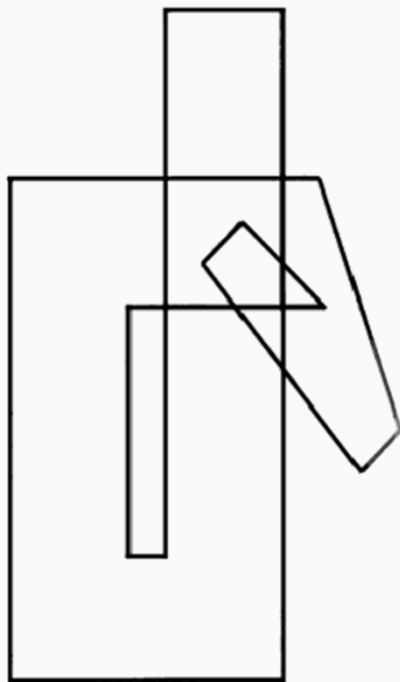
$$D = \frac{d + \Delta x}{2\sqrt{\Delta x^2 + \Delta y^2}} \quad D_u = \frac{\Delta x - d}{2\sqrt{\Delta x^2 + \Delta y^2}} \quad D_l = \frac{d + 3\Delta x}{2\sqrt{\Delta x^2 + \Delta y^2}}$$

- Wenn O gewählt:

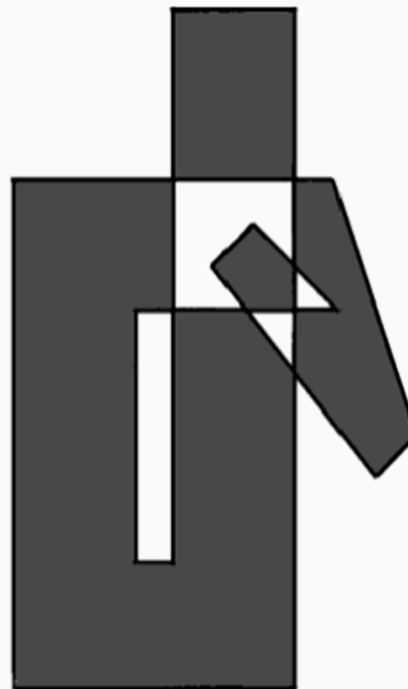
$$D = \frac{d - \Delta x}{2\sqrt{\Delta x^2 + \Delta y^2}} \quad D_u = \frac{d + \Delta x}{2\sqrt{\Delta x^2 + \Delta y^2}} \quad D_l = \frac{3\Delta x - d}{2\sqrt{\Delta x^2 + \Delta y^2}}$$

Füllen von Polygonen (1)

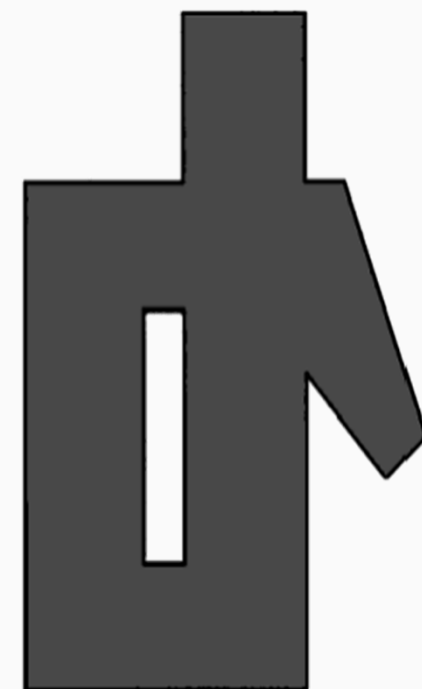
- Definition des zu füllenden Bereiches nicht immer eindeutig



Polygonzug



Paritätsregel

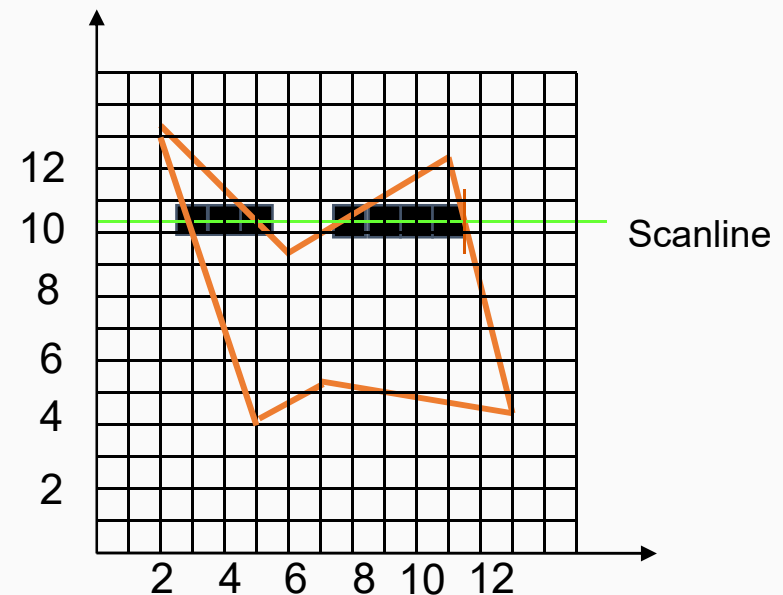


Orientierungsregel

Füllen von Polygonen (2)

Scanline Algorithmus

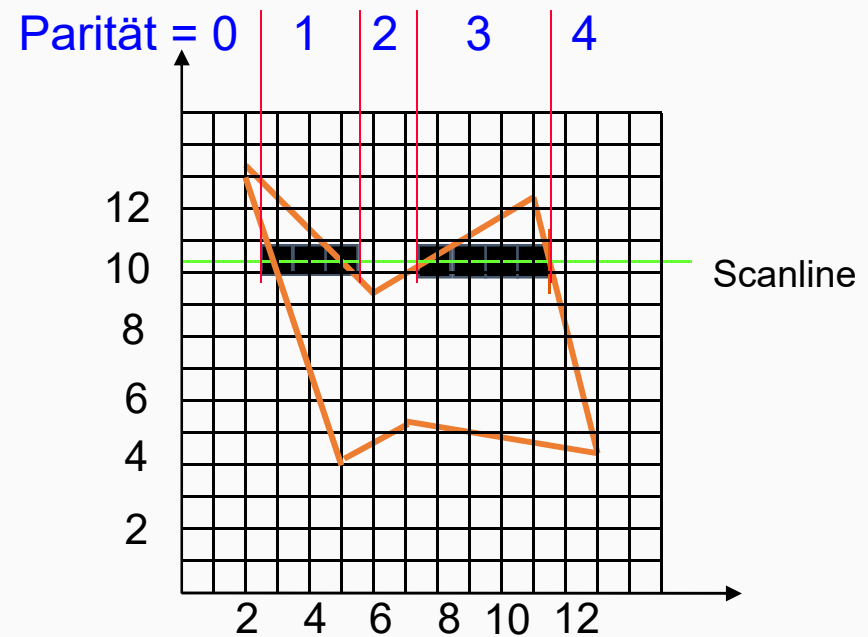
- Durchlaufe Polygon Zeilenweise:
 - Finde alle Schnittpunkte der aktuellen Scanline mit dem Polygon
 - Sortiere Schnittpunkte nach x-Koordinate
 - Fülle alle Pixel zwischen aufeinanderfolgenden Schnittpunktpaaren



Füllen von Polygonen (3)

Scanline Algorithmus

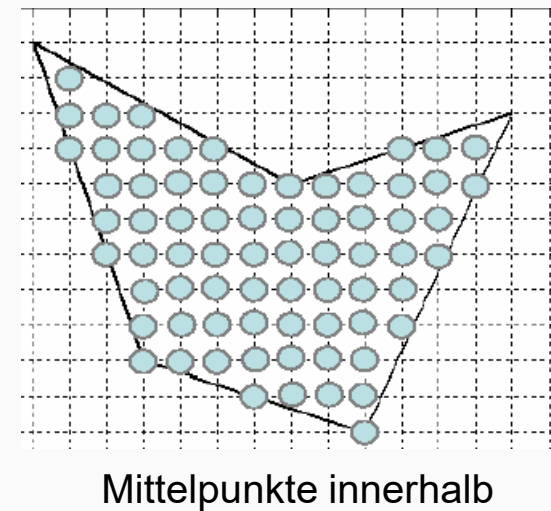
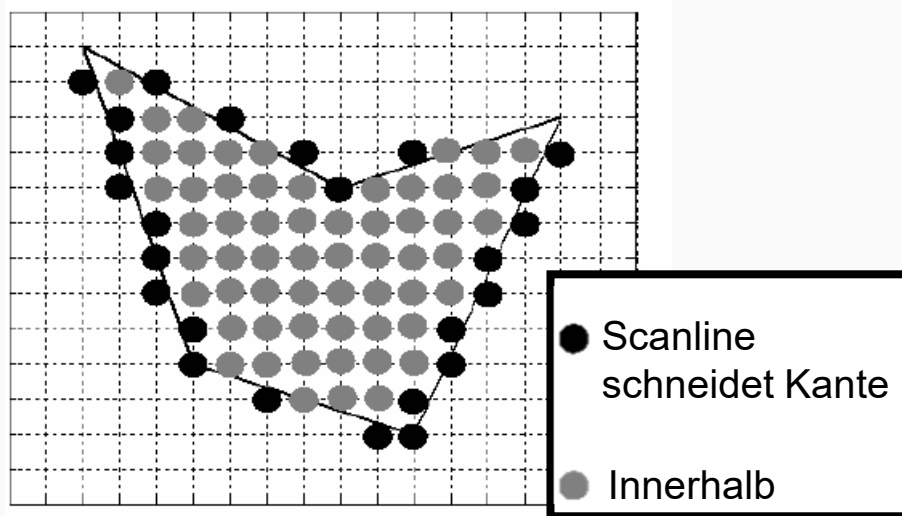
- Füllen nach Paritätsregel:
 - Parität am Anfang = 0
 - An jedem Schnittpunkt Parität um 1 erhöhen
 - Fülle Pixel, wenn Parität ungerade
- Bemerkung:
 - Alle Eckpunkte auf Pixelkoordinaten gerundet



Füllen von Polygonen (4)

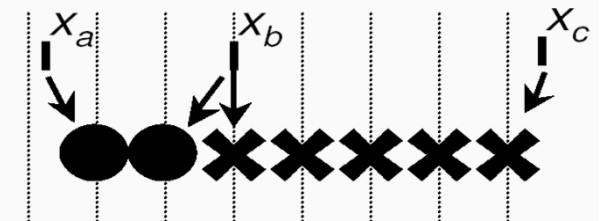
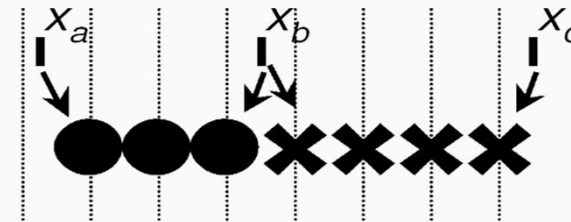
Extrema der Teilstücke (Spans) bestimmen

- Mittelpunktsalgorithmus auf den Kanten
 - Pixel außerhalb des Polygon
- Modifizierter MP-Algorithmus für innere Pixel



Füllen von Polygonen (5)

- Fließkommakordinaten erfordern konsistente Rundung zur Vermeidung von Löchern & Überlapp
 - z.B. Segmente x_a , x_b , x_b , x_c
 - Erster Pixel x_a aufrunden
 - (bleibt bei $\text{frac}(x_a)=0$)
 - Letzter Pixel x_b abrunden
 - (-1 bei $\text{frac}(x_b)=0$)



Füllen von Polygonen (6)

Schnitt mit Fließkommawert

⇒ innen aufrunden,
außen abrunden

Schnitt mit Integerwert

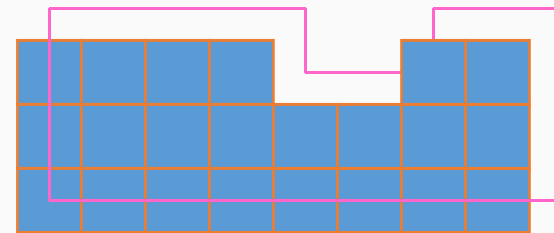
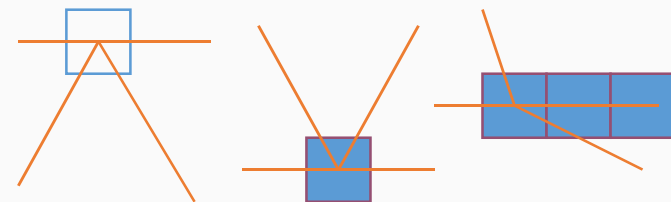
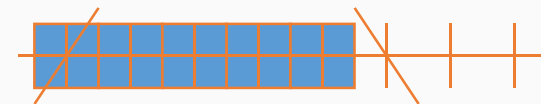
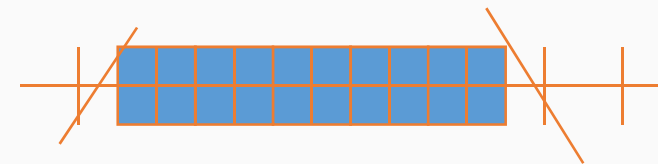
⇒ links innen, rechts außen

Gemeinsamer Kantenpunkt

⇒ nur $y_{\min}$ -Vertex zählen

Horizontale Kanten

⇒ werden ignoriert

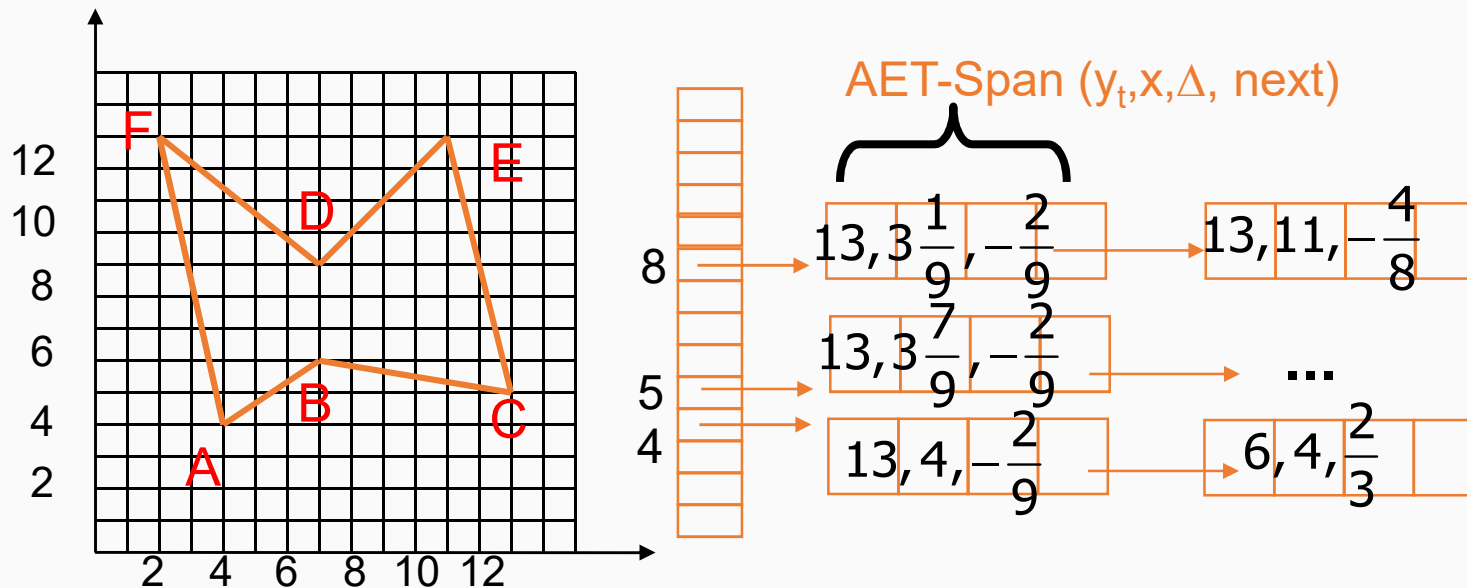


Füllen von Polygonen (7)

Kante (Edge): $(x_b, y_b, x_t, y_t, \Delta x / \Delta y)$

ET (Edge Table): alle Kanten sortiert nach y_b

AET (Active Edge Table): alle Kanten, die von der Scanline geschnitten werden; sortiert nach x-Koordinate



Füllen von Polygonen (8)

$$y = y_b(e_1)$$

$$\text{AET} = \{\}$$

Solange ET und AET nicht leer:

- Verschiebe Kanten mit $y = y_b(e_i)$ aus ET in AET
- Entferne Kanten mit $y = y_t(e_i)$ aus AET
- Sortiere Kanten in AET nach x-Koordinate
- Fülle Pixel anhand der Parität
- Gehe zur nächsten Scanline ($y = y + 1$)
- Aktualisiere x-Koordinaten der Kanten

Pineda Algorithmus (1)

- Grundidee:
 - Rasterisierung von konvexen Polygonen entspricht Clipping der Pixel
 - Dreiecke sind immer konvex
- Wdh.: Liang-Barsky Clipping Algorithmus

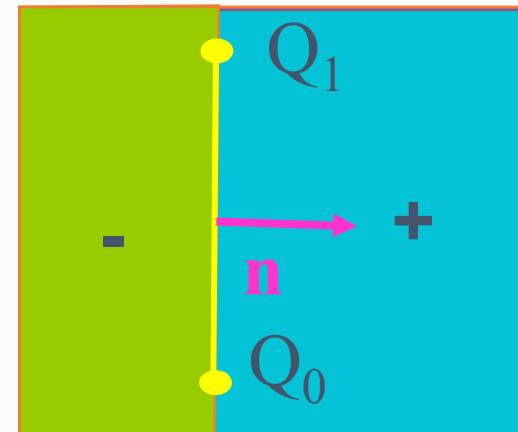
$$Q_0 = (x_0, y_0) \quad Q_1 = (x_1, y_1)$$

$$\mathbf{n} = (\Delta y, -\Delta x) = (y_1 - y_0, x_0 - x_1)$$

$$P = (x, y)$$

$$E(P) = \mathbf{n} \cdot P - \mathbf{n} \cdot Q_0$$

- Liefert Abstand zur Kante, wenn: $\|\mathbf{n}\| = 1$



Pineda Algorithmus (2)

- Übertragen auf Dreiecke:

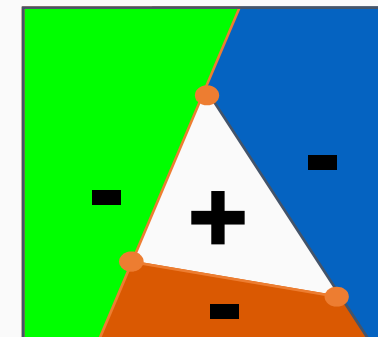
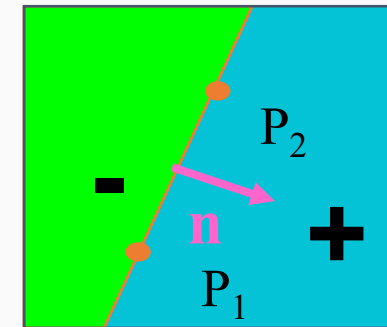
$$P_i = (x_i, y_i)$$

$$\mathbf{n}_i = (\Delta y_i, -\Delta x_i) = (y_{i+1} - y_i, x_i - x_{i+1})$$

$$\mathbf{n}'_i = \frac{\mathbf{n}_i}{\|\mathbf{n}_i\|} = \frac{\mathbf{n}_i}{\sqrt{\Delta x_i^2 + \Delta y_i^2}}$$

$$P = (x, y)$$

$$E_i(P) = \mathbf{n}'_i \cdot P - \mathbf{n}'_i \cdot P_i$$



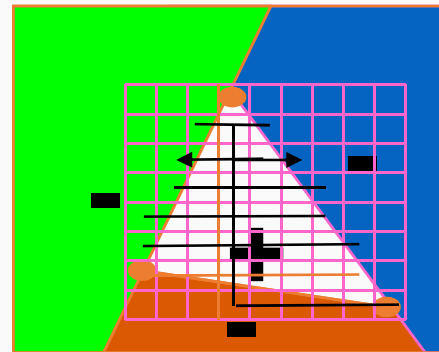
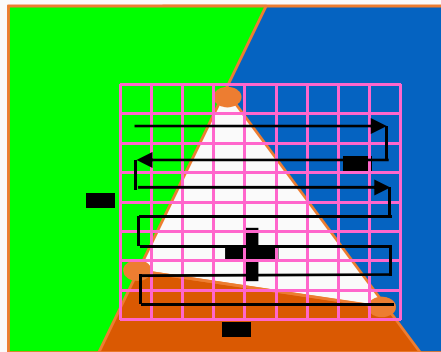
Pineda Algorithmus (3)

- $E_i(P)$ kann inkrementell berechnet werden

$$E_i(x+1, y) = E_i(x, y) + \frac{\Delta y_i}{\sqrt{\Delta x_i^2 + \Delta y_i^2}}$$

$$E_i(x, y+1) = E_i(x, y) - \frac{\Delta x_i}{\sqrt{\Delta x_i^2 + \Delta y_i^2}}$$

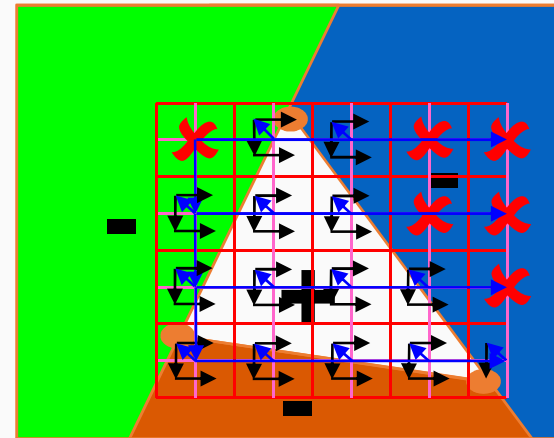
- Traversierung:



Pineda Algorithmus (4)

- Hierarchische Traversierung:

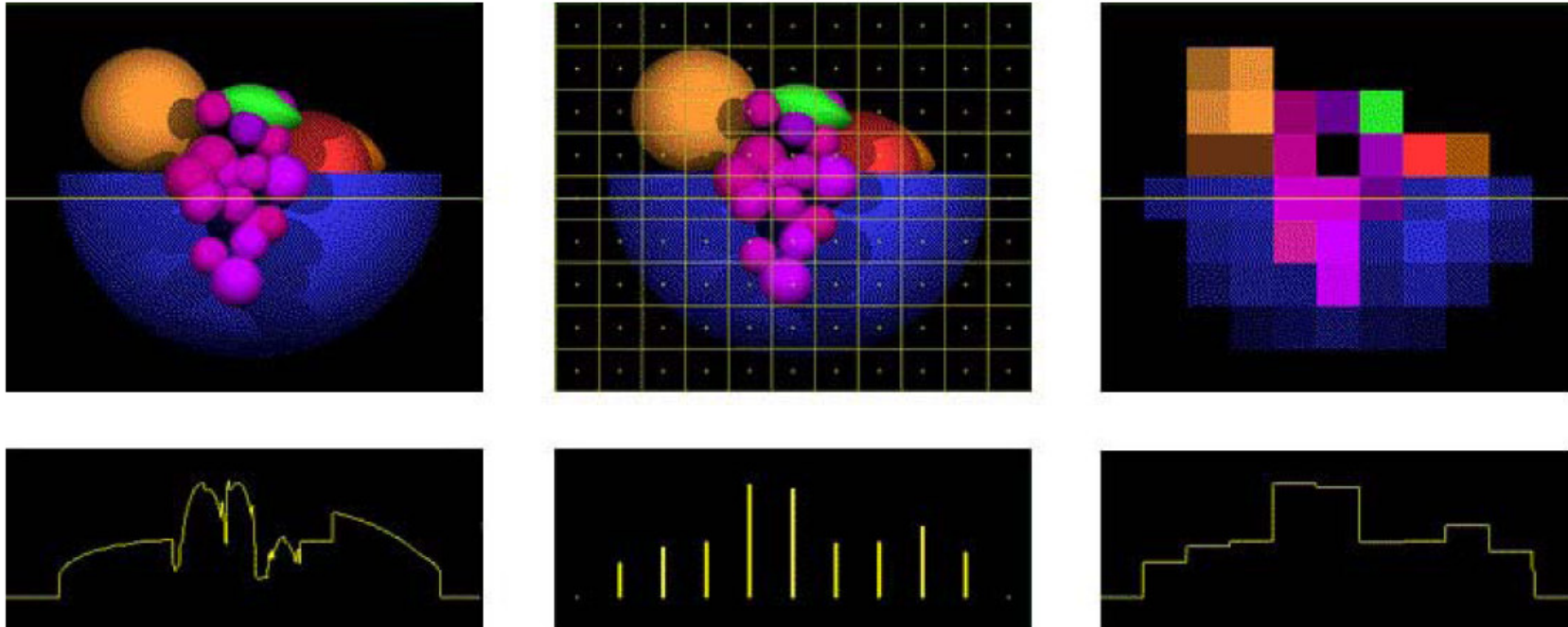
- Abstand ermöglicht
„Überspringen“ von Pixeln



- Integer Pineda Algorithmus:

- Wird $\mathbf{n}_i$ nicht normiert, hat man einen Integer Algorithmus
- $E_i(P)$ ist dann die doppelte Fläche des $\Delta(P, P_i, P_{i+1})$
- Baryzentrische Koordinaten aus den Dreiecksflächen

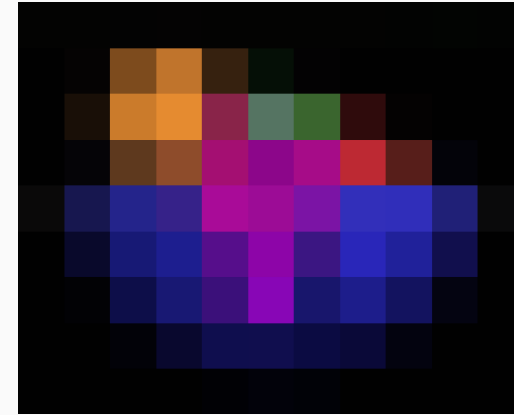
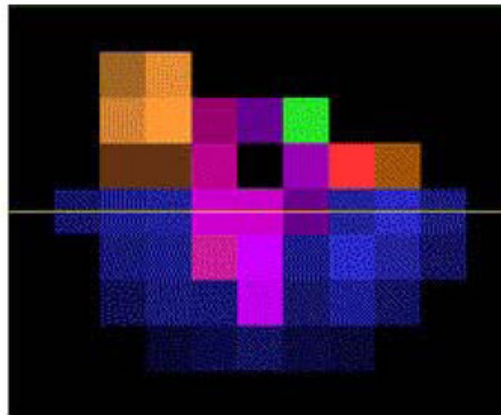
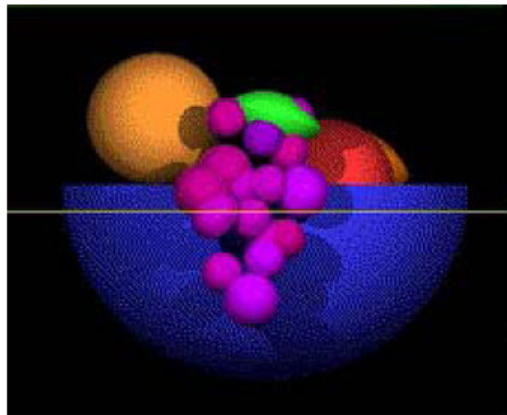
Aliasing bei gefüllten Flächen



- Unten: Helligkeitswerte entlang der Linie im Bild.
- Rechts: Effekt der Abtastung in Pixelmittelpunkten (Rasterisierung auf wenige Pixel)

Anti-Aliasing gefüllter Flächen

- Bestmögliche Darstellung durch Vorfilterung des Bildes
- Einfachste Methode:
 - Abschätzung der überdeckten Fläche (siehe Linien)



Anti-Aliasing von Dreiecken (1)

- Für Dreiecke mit Abstandsfunktion von Pineda Algorithmus möglich

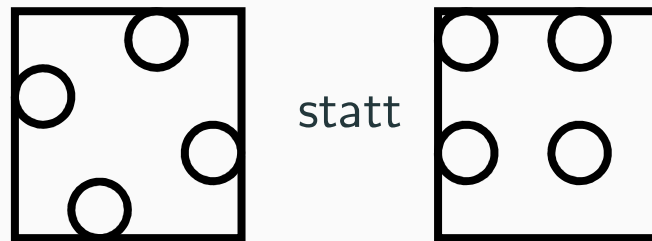
$$A(\mathbf{P}) \approx \prod_i \min\left(1, \max\left(0, E_i(\mathbf{P}) + \frac{1}{2}\right)\right)$$

- Problem: benachbarte Dreiecke
 - Addition ist bei Überschneidung falsch
 - Inverse Multiplikation ist bei gemeinsamen Kanten falsch

$$1 - A(\mathbf{P}) = (1 - A_1(\mathbf{P}))(1 - A_2(\mathbf{P}))$$

Anti-Aliasing von Dreiecken (2)

- Meist durch feinere Abtastung ([Supersampling](#))
 - Bild mit höherer Auflösung rastern
 - Rotierte Samplepositionen, um Helligkeitssprünge zu vermeiden



- Adaptives Anti-Aliasing ([Multisampling](#))
 - Markieren der Kantenpixel als „komplex“
 - Erzeugen und Speichern der Subpixel nur für komplexe Pixel

Anti-Aliasing von Dreiecken (3)



ohne
Anti-Aliasing

4 Samples
pro Pixel 4xAA

16 Samples
pro Pixel 16xAA